OPERATING SYSTEM VERIFICATION

by

Xiangdong Chen

A dissertation submitted to the faculty of
The University of Utah
in partial fulfillment of the requirements for the degree of

Doctor of Philosophy

Department of Kahlert School of Computing

The University of Utah

August 2026

Copyright © Xiangdong Chen 2026

All Rights Reserved

The University of Utah Graduate School

STATEMENT OF DISSERTATION APPROVAL

The dissertation of **Xiangdong Chen**

has been approved by the following supervisory committee members:

Anton Burtsev	, Chair	**April/7/2026** Date Approved
Ryan Stutsman	, Member	**April/7/2026** Date Approved
Jun Xu	, Member	**April/14//2026** Date Approved
Jonathan Appavoo	, Member	**April/8/2026** Date Approved
Chris Hawblitzel	, Member	**April/8/2026** Date Approved

and by **Mary W. Hall** , Chair of

the Department of **Kahlert School of Computing**

and by **Darryl P. Butt** , Dean of The Graduate School.

ABSTRACT

Operating systems (OS) play a pivotal role in modern computing, serving as the vital link between users, applications, and hardware. Formal verification is the only known way to deliver correct, bug-free software. However, despite decades of progress, formal verification remains a highly resource-intensive process, underscoring the inherent challenges of formal verification in low-level systems. This dissertation demonstrates several ways to mitigate the inherent challenges of OS kernel verification: (1) The high manual proof effort by the existing verification frameworks (2) The lack of support for natively verifying low-level system properties in the existing deductive verification frameworks (3) The conceptual gap between concurrent verification frameworks and real-world low-level system programming.

This dissertation presents a range of techniques to overcome (1) and (2) by leveraging Verus, a SMT-based verification framework for Rust, and more carefully modeling the state and the ownerships of the kernel in a non-conventional way. For (3), this dissertation demonstrates that incorporating a range of simple yet powerful concurrency rules from previous rules enables sound and efficient verification of a concurrent OS kernel in a normal programming style.

To my parents and sister.

TABLE OF CONTENTS

ABSTRACT iii

LIST OF FIGURES viii

LIST OF TABLES ix

ACKNOWLEDGMENTS x

Chapters

1 INTRODUCTION 1

1.1 The Challenge of Verified Operating System Kernels 1
1.2 Contributions of This Dissertation 3
1.3 Publications 5
1.4 Intended Audience 5
1.5 Dissertation Structure 5

2 INTRODUCTION TO FORMAL VERIFICATION 7

2.1 Specification, Proof, and Executable Code in Formal Verification . . 7
2.2 Quantifiers 13
2.3 Putting It All Together 13
2.4 The Value of Verified Software 17

3 INTRODUCTION TO VERUS AND LINEAR GHOST TYPES 19

3.1 Introduction to Verus 19
3.2 Introduction to the Verus Pipeline 20
3.3 Introduction to Linear Ghost Types 21
3.4 Triggers 27
3.5 Trigger Loops 28

4 VERUS INTERNALS 30

4.1 The Verus Procedural Macros: `verus!` 30
4.2 From Rust HIR to Verus VIR 33
4.3 Verus Lifetime Checks 37
4.4 VIR to SST (Statement-Oriented Syntax Tree) 38
4.5 Introduction to SMT 39
4.6 Encoding Function Body Checks 41
4.7 Encoding Pre- and Postconditions 43
4.8 Encoding Traits 49
4.9 Compiling to Binary 50

5 IMPLEMENTING ASYNC IN VERUS 51

5.1 Async in Rust 51
5.2 Prerequisite: Return Value Encoding 53
5.3 Prerequisite: Opaque Types 54
5.4 Expected Behavior of Async Function Postcondition 62
5.5 Adding the Future Trait to Vstd 64
5.6 Async Function: HIR to VIR 65
5.7 Rewriting the Postcondition 69
5.8 Encoding Await 70

6 RESEARCH CHALLENGES IN OS KERNEL VERIFICATION 73

6.1 Background of OS Kernel Verification 73
6.2 High Proof Effort 75
6.3 Excessively Long Verification Time 76
6.4 Limitations and Challenges in Linearity-Based Verification 78
6.5 Inherent Limitations of SMT Solvers on Unbounded Data Structures 79
6.6 Complexity of Kernel Invariants 80
6.7 Background: Related Work 81

7 ATMOSPHERE 86

7.1 Introduction 86
7.2 Key Design Decisions 87
7.3 System Architecture 89
7.4 Verification 92
7.5 Trusted Computing Base 111

7.6 Conclusions . . . 113
7.7 Evaluation . . . 114

8 CHALLENGES IN VERIFYING CONCURRENT PROGRAM . . . 128

8.1 Separation Logic . . . 128
8.2 Concurrent Separation Logic . . . 129
8.3 Challenge: Deadlocks . . . 129
8.4 Background: Related Work . . . 130

9 VERIFLAT . . . 139

9.1 Introduction . . . 139
9.2 Verification of Concurrency . . . 140
9.3 System Architecture . . . 155
9.4 Trusted Computing Base . . . 159
9.5 Current Status . . . 160
9.6 Conclusions . . . 160

10 CONCLUSION . . . 162

BIBLIOGRAPHY . . . 163

LIST OF FIGURES

Figures

7.1 Architecture of Atmosphere 91

7.2 Verification time for each function 115

7.3 Atmosphere commit history 119

7.4 Atmosphere Ixgbe driver performance 123

7.5 Atmosphere Performance of the NVMe driver 124

7.6 Atmosphere Performance of Maglev 126

7.7 Atmosphere Performance of Key-value store 127

LIST OF TABLES

Tables

2.1 Spec, Proof, and Exec Code 11

6.1 Proof effort for existing verification projects 76

6.2 Verification time of different systems on CloudLab c220g5 77

7.1 Verification time of different systems on CloudLab c220g5 115

7.2 Proof to code ratio of different systems 116

7.3 Latency of communication and typical system calls (cycles) on c220g5 . 120

ACKNOWLEDGMENTS

This dissertation would not have been possible without the support of many people, to whom I am deeply grateful.

First and foremost, I would like to thank my advisor, Anton Burtsev, for his guidance, patience, trust, and support throughout my doctoral journey. Without him, this would not have been possible.

I would also like to thank my other mentors and colleagues over the years. I am grateful to the members of my dissertation committee—Ryan Stutsman, Jun Xu, Jonathan Appavoo, and Chris Hawblitzel—for their support and insightful feedback.

I also thank Daniel Schoepe, Marianna Rapoport, and Bryan Parno, my internship mentors at Amazon, for a productive summer and the opportunity to learn and work on the internals of Verus.

I would also like to thank all of our labmates and lifelong friends: Vikram Narayanan, Zhaofeng Li, Jerry Zhang, Joshua Tlatelpa-Agustin, and Manvik Nanda. This work would not have been possible without your collaboration and support.

Eight and a half years ago, I came to the United States for my undergraduate studies. Since then, I have spent most of my adult life here with my friends.

Thank you all for your support and companionship—I have never felt lonely or lost in this foreign country.

Finally, I would like to thank my parents and sister. Thank you for always believing in me.

CHAPTER 1

INTRODUCTION

1.1 The Challenge of Verified Operating System Kernels

Despite decades of progress, the development of formally verified operating systems remains a challenging undertaking that demands considerable verification expertise and often requires years of human effort [22], [25], [32].

Kernel correctness depends on numerous low-level invariants governing all internal data structures. These invariants span a wide spectrum: from the lifetimes of manually managed data structures and the correctness of pointer operations in hierarchical structures such as queues, lists, trees, and page tables, to the high-level contracts of the system call interface, and further to cross-cutting properties that capture the global well-formedness of the system as well as high-level semantic invariants.

The seL4 team [32] pioneered the verification of functional correctness and security properties for a high-performance, full-featured L4-family microkernel. Despite numerous careful design decisions, verifying 8,700 lines of C code demanded a heroic effort, consuming 20 person-years and producing 165,000 lines of proof in Isabelle/HOL [32]. Subsequently, CertiKOS [22] emerged as the

first fully verified microkernel supporting multicore concurrency and fine-grained locking. Although implemented in C and assembly and making several compromises in performance and feature richness, CertiKOS required 135,000 lines of Coq (recently renamed to Rocq) to verify 6,500 lines of C and assembly code; the proof itself took two person-years, with an additional two person-years spent constructing the verification framework.

Fortunately, recent advances in programming languages and automated formal reasoning are lowering the threshold for the practical development of verified kernel code [33]. Modern verifiers successfully combine the guarantees of linear type systems with automated proof search based on Satisfiability Modulo Theories (SMT) [33], [34], [37]. Verus [33] is a state-of-the-art Rust verification tool that supports semi-automated reasoning using the Z3 SMT solver as its backend. It seeks to replicate the success of earlier verification frameworks by integrating specifications, proofs, and implementation within a single language. By building upon Rust, Verus inherits its linear ownership and borrow-checking disciplines, which eliminate pointer aliasing and enhance proof scalability. To enable sound reasoning about `unsafe` Rust raw pointers, Verus employs linear ghost types—checked by Rust's compile-time borrow checker—to represent the state of the objects pointed to by raw pointers. Its highly optimized SMT encoding enables rapid verification times. Several recent systems projects [55], [61] have demonstrated Verus's impressive proof automation and verification speed.

1.2 Contributions of This Dissertation

The introductory material and tutorial sections on formal verification and Verus presented in this dissertation are not original contributions. They are included to provide background and guidance for readers who wish to learn these topics.

The contributions of this dissertation are as follows:

1.2.1 Opaque types and async in Verus

My contributions to the open-source project Verus include the implementation of opaque types and support for asynchronous functions. To the best of my knowledge, Verus is the first verification framework to support the verification of asynchronous functions.

1.2.2 Atmosphere

This work leverages Rust and Verus to demonstrate the feasibility of developing a fully verified microkernel, named Atmosphere, with a speed and effort that arguably approach that of commodity unverified development. To manage verification complexity, Atmosphere is deliberately designed as a single-threaded kernel. For this kernel, we verify the full functional correctness of system calls, guarantee the absence of memory leaks, and prove non-interference between system calls originating from untrusted user-level domains. To circumvent limitations of Rust's linear lifetime system in reasoning about memory leaks and to further reduce verification time, Atmosphere incorporates a

range of techniques, including flat ownership and manual memory management. Consequently, Atmosphere achieves a favorable 3:1 proof-to-code ratio and completes a full-system verification run in under 20 seconds. The unique synergy between a linear type system and SMT-based verification invites a re-evaluation of the practical complexity and burden associated with developing verified low-level systems code.

1.2.3 VeriFlat

To overcome the limitation of the big kernel lock in Atmosphere, in this preliminary work, in addition to Rust and Verus, we identify and apply a set of simple, practical yet powerful concurrency rules from prior works to develop a fully verified, concurrent microkernel, VeriFlat, with support for fine-grained locking, resource isolation between different compartments (containers), and better multicore performance than Atmosphere. On top of all the verification guarantees in Atmosphere, including functional correctness and memory leak freedom, VeriFlat is verified to ensure the logical atomicity for all of its system call interfaces and deadlock freedom. Although VeriFlat is a concurrent kernel, the system calls appear completely atomic to the users, and thus, are expressible with Hoare Logic (pre-/postconditions) and identical to those of a sequential counterpart, making the system specification more usable and practical. With even more optimizations than Atmosphere, VeriFlat, in its current development stage, demonstrates a more promising proof scalability and performance than Atmosphere.

1.3 Publications

The content of Chapter 7 is primarily based on the following publications, on each of which I am the first author:

- *Atmosphere: Towards Practical Verified Kernels in Rust* [12]
- *Atmosphere: Practical Verified Kernels with Rust and Verus* [14].

1.4 Intended Audience

The primary aim of this dissertation is to provide the systems community with a practical yet in-depth guide to the formal verification of operating system kernels. It assumes readers possess a basic, and occasionally advanced, understanding of operating system kernels, programming languages, security, and compilers. However, it presumes little to no prior background in formal verification. By the conclusion of this dissertation, readers should be equipped to embark on writing their own formally verified systems software.

1.5 Dissertation Structure

The remainder of this dissertation is organized as follows:

- Chapter 2 introduces the foundational concepts of formal verification to equip readers with the necessary background.
- Chapter 3 presents Verus [33], the verification framework central to this work, and details its integration with Rust's linear type system.

- Chapter 4 delves into the principles of modern SMT-based verification frameworks by examining the internals of Verus, including its front-end procedural macros, SMT encoding strategies, and lifetime system.
- Chapter 5 provides a concrete, real-world example by walking through the process of soundly adding a new type (opaque types) to Verus, as well as detailing the implementation of `async` function support within the framework.
- Chapter 6 provides the background for this work, explaining the motivation and challenges in verifying an OS kernel, along with related work on the verification of sequential programs.
- Chapter 7 presents Atmosphere, the verified sequential kernel built with Verus and Rust, which addresses the challenges mentioned in the previous chapter and reports performance results.
- Chapter 8 covers the background on verifying concurrent programs, identifying challenges unique to verifying a concurrent kernel.
- Chapter 9 presents our ongoing work, VeriFlat, a verified concurrent kernel built with Verus and Rust.
- Chapter 10 concludes this dissertation.

CHAPTER 2

INTRODUCTION TO FORMAL VERIFICATION

2.1 Specification, Proof, and Executable Code in Formal Verification

2.1.1 Function Specification

The specification of a function is a formal description of its expected behavior, encompassing its inputs, outputs, and constraints, without detailing the implementation.

In conventional programming languages, programmers write comments to informally describe the expected behavior of the code. In Verus [33], a function's specification is formally expressed through pre- and postconditions:

```
1  pub fn add(x: usize, y: usize) -> (ret: usize)
2      requires
3          x + y <= usize::MAX,
4      ensures
5          ret == x + y,
6  {
7      x + y
8  }
```

To the caller, the precondition serves as a prerequisite; the function cannot be invoked unless the precondition is satisfied. For the function body, the precondition provides known properties about the inputs, which aid in proving the postcondition. The postcondition provides the caller with a concise yet precise

mathematical description of the function's outcome, conveying information about the return value (and any mutated inputs) without the need to inspect the implementation. In this specific instance, the specification of `add()` asserts: provided `x + y` does not overflow, the return value `ret` is guaranteed to equal the sum of `x` and `y`. Consequently, the caller only needs to verify the absence of overflow before calling `add()`, and the result will always be the correct sum. While this is a minimal example, later sections will introduce functions with more complex logic to better demonstrate the utility of pre- and postcondition-based verification.

If the precondition is omitted:

```
pub fn add(x: usize, y: usize) -> (ret: usize)
    ensures
        ret == x + y,
{
    x + y
}
```

the verifier emits the following error:

```
error: possible arithmetic underflow/overflow
   |
   |     x + y
   |     ^^^^^
```

thereby alerting the programmer to a potential overflow.

If a function's implementation fails to satisfy its declared postcondition:

```
pub fn add(x: usize, y: usize) -> (ret: usize)
    requires
        x + y <= usize::MAX,
    ensures
        ret == x - y,
{
    x + y
```

```
8  }
```

the verifier reports:

```
1  error: postcondition not satisfied
2      |
3      |           ret == x - y,
4      |           ^^^^^^^^^^^^ failed this postcondition
5      | {
6      |       x + y
7      |       ----- at the end of the function body
```

indicating the specific postcondition failure.

2.1.2 Assert

While the verifier can automatically prove simple functions like the one above, more complex functions and specifications often require guidance from the programmer. The `assert` statement allows the programmers to make a local, private query to the verifier about a particular fact without altering the function's public interface. This is useful for guiding the verification process and for debugging proofs.

For example, a programmer might insert an assertion to help diagnose a missing precondition:

```
1  pub fn add(x: usize, y: usize) -> (ret: usize)
2      ensures
3          ret == x + y,
4  {
5      assert((x + y > usize::MAX) == false); // check if overflow is
           impossible
6      x + y
7  }
```

The `assert` function instructs the verifier to validate the provided boolean expression; the verification succeeds only if the expression can be proven always true. In this case, the verifier responds with:

```
1  error: assertion failed
2      |
3      |      assert((x + y > usize::MAX) == false); // check if overflow is
           impossible
4      |             ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^ assertion failed
```

This informs the programmer that the sum `x + y` can, in some cases, exceed `usize::MAX`, which would lead to integer overflow.

2.1.3 Spec and Proof

Verus categorizes code into three modes: `spec`, `proof`, and `exec`.

- `spec` code describes logical properties of the program.
- `proof` code constructs demonstrations that programs satisfy given properties.
- `exec` code is standard Rust code intended for compilation and execution.

The relationships among these modes are summarized in table 2.1:

In the preceding examples, pre- and postconditions are `spec` code, the `assert` statement is `proof` code, and the remaining code is `exec` code. Both `spec` and `proof` code are considered *ghost* code; they are present only during verification and are erased during compilation.

2.1.3.1 Spec Functions

Spec functions encapsulate reusable specification code. For instance:

Table 2.1. Spec, Proof, and Exec Code.

Variable Mode	Where Allowed	Typical Use
Spec	Spec + Proof + Exec	Specifying requirements (pre/postconditions) or abstract properties
Proof	Proof, Exec	Manipulating ghost state; constructing logical arguments
Exec	Exec	Implementing executable functionality

```
pub open spec fn spec_add(x: usize, y: usize) -> int
{
    x + y
}
```

defines a spec function that models ideal integer addition. Programmers can then reference `spec_add`:

```
pub fn add(x: usize, y: usize) -> (ret: usize)
    requires
        x + y <= usize::MAX,
    ensures
        ret == spec_add(x, y),
{
    x + y
}
```

This avoids repeating the logical definition and clarifies the connection between the implementation and its abstract specification.

2.1.3.2 Proof Functions

Proof functions, like their executable counterparts, can have pre- and postconditions. However, while the specifications of an exec function describe

how the state evolves, the specifications of a proof function describe what is assumed and what is guaranteed by a reusable proof, hiding its intermediate steps.

Conceptually, a proof function operates as follows:

```
pub proof fn dummy_proof_fn()
    requires
        condition_A
    ensures
        condition_D
{
    assert(condition_A implies condition_B);
    assert(condition_B implies condition_C);
    assert(condition_C implies condition_D);
}
```

Given `condition_A`, the body of `dummy_proof_fn` establishes the intermediate logical steps, ultimately proving `condition_D`. Other functions can then invoke `dummy_proof_fn` to deduce `condition_A implies condition_D` without reproving the intermediate steps.

2.1.4 Ghost versus Exec Variables

The reader may have observed that expressions like `x + y` in ghost code (i.e., `spec` and `proof` code) do not trigger integer overflow warnings from Verus, unlike in `exec` code. This is because ghost code utilizes a ghost type `int` that represents all integers, unbounded by machine word size. In the `spec_add()` function, the return type is `int`, thereby eliminating overflow in ghost code. Since all ghost constructs are erased prior to compilation, using an unbounded integer type causes no issues.

2.2 Quantifiers

Universal (`forall`) and existential (`exists`) quantifiers allow programmers to specify properties over collections of elements concisely.

The signature below is for a function that adds two integer vectors element-wise:

```
pub fn add_vec(x: Vec<usize>, y: Vec<usize>) -> (ret: Vec<usize>)
    requires
        x.len() == y.len(),
        forall |i: int| 0 <= i < x.len() ==> x[i] + y[i] <= usize::MAX
    ensures
        ret.len() == x.len(),
        forall |i: int| 0 <= i < x.len() ==> ret[i] == x[i] + y[i],
```

Here, the `forall` quantifier in the precondition ensures that no element-wise addition will overflow.

2.3 Putting It All Together

To illustrate how `spec`, `proof`, and `exec` code combine to build a verified component, consider a Verus-implemented `DoublyLinkedList`:

```
struct Node<V> {
    prev: Option<PPtr<Node<V>>>,
    next: Option<PPtr<Node<V>>>,
    payload: V,
}

// Contains head pointer, tail pointer
pub struct DoublyLinkedList<V> {
    // physical data:
    head: Option<PPtr<Node<V>>>,
    tail: Option<PPtr<Node<V>>>,

    // ghost pointers to all the nodes in the linkedlist
    ghost ptrs: Seq<PPtr<Node<V>>>,
    // the ownerships and state of each node in the linkedlist
```

```
16        tracked points_to_map: Map<PPtr<Node<V>>, PointsTo<Node<V>>>,
17    }
```

`Node<V>` represents a list node, parameterized by type `V`. The `DoublyLinkedList<V>` structure includes physical pointers (`head`, `tail`) alongside ghost state: `ghost ptrs` (a sequence of all node pointers) and `tracked points_to_map` (a map tracking ownership and node state). Ghost ownership is discussed further in Chapter 3.

A utility spec function `view` defines the list's abstract state as a sequence of values of type `V`:

```
1  /// Representation of this list as a sequence
2  pub closed spec fn view(&self) -> Seq<V> {
3      Seq::<V>::new(
4          self.ptrs.len(),
5          |i: int| { self.points_to_map[self.ptrs[i]].value().payload },
6      )
7  }
```

Note, in Verus, `.view()` can be shortened to `@`, hence: `self.view()` is equal to `self@`. A pertinent question arises: since `view()` derives order solely from the ghost sequence `self.ptrs`, how do we guarantee that `self.ptrs` matches the concrete pointer linkages? This assurance comes from formally defined invariants.

First, an empty list must have empty `ptrs` and `None` for `head` and `tail`; the first/last elements of the ghost sequence must align with the physical `head` and `tail` when the list is non-empty:

```
1  pub closed spec fn head_tail_well_formed(&self) -> bool {
2      &&&
3      (self.head.is_some() == self.tail.is_some())
4      &&&
```

```
5         (self.head.is_some() == (self.ptrs.len() != 0))
6         &&&
7         (self.head.is_some() ==>
8             (self.ptrs[0] == self.head.unwrap()
9              &&
10             self.ptrs[self.ptrs.len() - 1] == self.tail.unwrap()))
11    }
```

Second, the order in `self.ptrs` must be consistent with the `prev` and `next` pointers of each node:

```
1   pub closed spec fn prev_next_well_formed(&self) -> bool {
2       &&&
3       forall |i: int|
4           0 <= i < self.ptrs.len() - 1
5           ==>
6           (self.points_to_map[self.ptrs[i]].value().next.is_some()
7            &&
8            self.points_to_map[self.ptrs[i]].value().next.unwrap() == self.
                 ptrs[i + 1])
9       &&&
10      forall |i: int|
11          1 <= i < self.ptrs.len()
12          ==>
13          (self.points_to_map[self.ptrs[i]].value().prev.is_some()
14           &&
15           self.points_to_map[self.ptrs[i]].value().prev.unwrap() == self.
                 ptrs[i - 1])
16  }
```

The `forall` quantifier concisely expresses these relational constraints over the entire sequence.

To consolidate these conditions, a top-level well-formedness predicate is defined:

```
1   pub closed spec fn wf(&self) -> bool {
2       &&&
3       self.head_tail_well_formed()
4       &&&
5       self.prev_next_well_formed()
6   }
```

Using these specifications, the public functions of the list can be defined:

```
pub fn new() -> Self
    ensures
        self.wf(),
        self.view() == Seq::<V>::empty(),
{
    // rest of the function body.
}
pub fn push(&mut self, v: V)
    requires
        old(self).wf(),
    ensures
        self.wf(),
        self.view() == old(self).view().push(v),
{
    // rest of the function body.
}
pub fn pop(&mut self) -> (ret: V)
    requires
        old(self).wf(),
        old(self).view().len() != 0,
    ensures
        self.wf(),
        self.view() == old(self).view().pop_tail(),
        ret == old(self).view()[old(self).view().len() - 1],
{
    // rest of the function body.
}
```

The `new()` function creates a new instance of the doubly-linked list and initializes it into a well-formed state. The `push()` and `pop()` functions require the list to be well-formed beforehand and guarantee it remains so afterward, and also precisely describe the state change to the abstract `view()`. Callers can therefore reason purely in terms of the abstract sequence model, without delving into implementation details.

The implementation of these functions employs advanced Verus features to manage Verus linear ghost permissions and will be detailed in later chapters.

This example outlines a general methodology for constructing verified software:

- Define the concrete state.
- Define an abstract model capturing the abstract state that the users care about. In this case, users would most likely want the doubly-linked list to behave like a sequence of type `V`
- Formally define invariants linking the concrete state to the abstract model.
- Define the supported transitions (interfaces) – in this case, the `exec` functions.
- Implement and verify the `exec` functions against these specifications.

Developing a verified sequential OS kernel follows this same pattern, albeit with significantly greater complexity in managing memory, hardware interaction, and system-wide invariants.

2.4 The Value of Verified Software

The principal value of verified software extends beyond machine-checked correctness or a reduced trusted computing base (TCB). Its true power lies in abstracting complex implementation that contains machine-level details into a mathematical, direct, and auditable form.

In the linked list example, the critical invariants are encapsulated within the `wf()` predicate and verified at function boundaries. In addition, the operational specification (pre-/postcondition) of each `exec` function describes the behaviors of the interfaces. These invariants and operational specifications are trusted and

together form the TCB of the linked list. Provided these invariants and operational specifications accurately capture all necessary correctness conditions, the correctness of the executable code follows. As demonstrated, auditing these formal specifications is substantially easier than analyzing the low-level source code, as they express core logic while abstracting away implementation details, control flow branches, machine/architecture-specific details and edge-case handling.

For complex verified systems, the ratio of trusted specification code to executable code is often 1:2 or lower. Since specifications are inherently easier to audit and less prone to errors, formal verification provides a pragmatic pathway to achieving high levels of security and assurance in software systems with reduced effort.

CHAPTER 3

INTRODUCTION TO VERUS AND LINEAR GHOST TYPES

3.1 Introduction to Verus

Verus is a verification tool for Rust that enables semi-automated reasoning by leveraging an SMT solver [33]. As demonstrated in the previous chapter, Verus co-locates proofs and executable code within a single language, a model proven highly successful by earlier verification frameworks. However, Verus distinguishes itself in several key aspects.

First, rather than relying on a verification-centric language, Verus extends Rust, a language already designed for safe, low-level systems development. Previous successful frameworks (e.g., Dafny [35]) define their own programming languages, which subsequently require significant effort to be transpiled to conventional systems languages or binaries.

Second, Verus benefits from Rust's linear type system, which simplifies the reasoning about heap objects, raw pointers, and concurrency. Rust's ownership system eliminates pointer aliasing and enforces thread safety, requiring no additional verification effort for these guarantees.

Third, Verus prioritizes verification speed over feature richness and aggressively optimizes its SMT encoding throughout the entire verification pipeline. Prior frameworks often frustrate developers due to excessively long verification times. A project with as few as 10k lines of code can sometimes take hours to verify. While this is partially attributable to the overhead of checking pointer aliasing and object ownership, their less optimal SMT encoding strategy is also a major factor.

3.2 Introduction to the Verus Pipeline

The Verus pipeline, from proof-annotated source code to a verified and compiled binary, consists of the following steps:

1. The `verus!` procedural macro transforms all ghost code (both `spec` and `proof`) into standard Rust syntax.
2. The Rust compiler (`rustc`) compiles the transformed code into High-Level Intermediate Representation (HIR) and performs type checking.
3. Verus erases all ghost code (except linear ghost values) and sends the erased code back to `rustc` for lifetime checking.
4. After lifetime checking, Verus lowers the HIR from (2) into Verification Intermediate Representation (VIR). Following a series of checks and lowering, it emits queries to the SMT solver.
5. Upon successful verification, the procedural macro erases all remaining ghost code from the original source and invokes `rustc` to compile the final executable binary.

3.3 Introduction to Linear Ghost Types

3.3.1 Rust Linear Ownership

Rust's ownership system is its most distinctive feature among low-level programming languages, enabling memory safety without garbage collection. The *linear* aspect of this system means each value has exactly one owner at any time, and ownership must be explicitly transferred. Note that Rust's ownership system guarantees safety but not the absence of memory leaks, which are still possible in safe Rust. The three core rules are:

- Each value has exactly one owner.
- When the owner goes out of scope, the value is dropped and its memory is freed.
- Multiple immutable references or a single mutable reference may exist, but references cannot outlive the value they refer to.

These rules are enforced at compile time by Rust's borrow checker, incurring no runtime overhead.

3.3.2 Linearity in Formal Verification

Consider a scenario where Rust hypothetically allowed multiple mutable references:

```
1 pub fn foo() {
2     let mut x: usize = 0;
3     let mut_ref_1 = &mut x;
4     let mut_ref_2 = &mut x;
5     * mut_ref_2 = 1;
6 }
```

A single write, `* mut_ref_2 = 1;`, would change the state of multiple logical entities: not only `mut_ref_2`, but also `mut_ref_1` and `x`. This would force the verifier (or the programmers to write proofs to manually) to track and propagate state changes across all aliases, introducing significant verification complexity and overhead. In contrast, when at most one mutable reference to a value exists, only that reference and the underlying value need to be tracked and updated upon a write.

Rust's linearity similarly reduces the complexity of reasoning about concurrency. If a value is being mutated in one thread, it cannot be accessed from another, eliminating data races at compile time. We will cover the verification of concurrent code in later chapters; our present focus remains on sequential programs.

3.3.3 Linear Ghost Types (Tracked Permissions)

Rust's strict linearity can be overly restrictive for data structures like doubly-linked lists or trees. In a doubly-linked list, a node cannot be owned by both its predecessor and successor. While reference-counted smart pointers (e.g., `Rc<T>` and `RefCell<T>`) can be used, they introduce non-zero runtime and space overhead and are essentially still pointers and reintroduce the pointer aliasing problems that Rust's linearity elegantly avoids. Verus addresses this through *linear ghost types*, which decouple state and ownership from physical pointers and references.

As the name suggests, linear ghost types are ghost variables that adhere to Rust's ownership rules. Their core principles are:

- A value can be converted into a single linear ghost permission.
- The lifetime and borrowing of these permissions are checked by Rust's borrow checker, not the verifier.
- The linear ghost permission owns the value and maintains its state.
- The permission uses the value's memory address as its unique identifier.
- Raw pointers to the value carry only the address.

To access an object by dereferencing the raw pointer, the program must first prove it holds a valid ghost permission for that address. Any state change from the access is then reflected solely on the ghost permission, avoiding the need to propagate updates to aliases.

3.3.4 Linear Ghost Types Example: Doubly Linked List

Recall the doubly-linked list definition from the previous chapter:

```
pub struct DoublyLinkedList<V> {
    // physical data:
    head: Option<PPtr<Node<V>>>,
    tail : Option<PPtr<Node<V>>>,
    // ghost pointers to all the nodes in the linkedlist
    ghost ptrs: Seq<PPtr<Node<V>>>,
    // the ownerships and state of each node in the linkedlist
    tracked points_to_map: Map<PPtr<Node<V>>, PointsTo<Node<V>>>,
}
```

Here, `PPtr<T>` is a safe wrapper around the raw pointer, carrying only the address. `Seq<V>` is a ghost (but non-linear) data structure of unlimited size. `PointsTo<T>` is the linear ghost permission. `perm.addr()` and `perm.value()` can

be used to access the address and state of the object under the linear ghost permission (`perm`).

The `tracked` keyword indicates that this data structure follows Rust's linear ownership rules and is purely ghost—it will be erased before compilation. Consequently, ghost code can utilize data structures beyond the limits of physical machines. For instance, `points_to_map` is a tracked `Map<K, V>` of unlimited size holding all tracked permissions for the list nodes.

As mentioned previously, `ptrs` maintains the abstract sequence of node addresses, while `points_to_map` governs the tracked permissions, using the address (`PPtr<T>`) as the key. Therefore, proving that `points_to_map` correctly holds the permissions for all nodes in the list establishes the list's correctness.

We introduce the following invariant for this purpose:

```
pub closed spec fn perms_wf(&self) -> bool {
    // Here we focus on just the safety, not memory leak free.
    &&&
    forall |pptr: PPtr<Node<V>>|
        self.ptrs@.contains(pptr)
        ==>
        self.points_to_map@.contains_key(pptr)
        &&
        self.points_to_map@[pptr].is_init()
        &&
        self.points_to_map@[pptr].addr() == pptr.addr()
}
```

In short, this invariant ensures that (1) the tracked map owns the tracked permission for every node in `ptrs` , (2) each permission is initialized, and (3) each permission's address matches its key. Adding this invariant to `wf()` :

```
pub closed spec fn wf(&self) -> bool {
    &&&
```

```
        self.head_tail_well_formed()
        &&&
        self.prev_next_well_formed()
        // Add the new invariant
        &&&
        self.perms_wf()
}
```

With this, we can complete the `push_empty()` function body:

```
// First case of push(): the linked list is empty.
pub fn push_empty(&mut self, v: V)
    requires
        old(self).wf(),
        old(self).view() == Seq::empty(),
    ensures
        self.wf(),
        self.view() == old(self).view().push(v),
{
    // Allocate a node to contain the payload.
    let node = Node::<V> { prev: None, next: None, payload: v };

    // Convert it into a tracked permission.
    let (ptr, Tracked(points_to)) = PPtr::<Node<V>>::new(node);

    // Update head and tail pointers.
    self.tail = Some(ptr);
    self.head = Some(ptr);

    // Update the ghost sequence.
    proof {
        self.ptrs@ = self@.ptrs.push(ptr);
    }

    // Update the tracked map.
    proof {
        self.points_to_map.borrow_mut().tracked_insert(ptr, points_to);
    }

    // Verify that invariants still hold.
    assert(self.wf());
}
```

This function allocates a node, consumes it to obtain a tracked permission via `PPtr::new`, and then transfers ownership of that permission to the list's `points_to_map`.

For the non-empty case:

```
// Second case of push(): the linked list is not empty.
// Appends the new value to the tail.
pub fn push_non_empty_tail(&mut self, v: V)
    requires
        old(self).wf(),
        old(self).view() != Seq::empty(),
    ensures
        self.wf(),
        self.view() == old(self).view().push(v),
{
    // Remember the old tail.
    let old_tail_op = self.tail;
    // Per the invariants, if the list is not empty, tail is Some.
    assert(old_tail_op.is_some()) by {
        assert(self.head_tail_well_formed());
    };
    let old_tail = old_tail_op.unwrap();

    // Allocate a new node.
    let node = Node::<V> { prev: old_tail_op, next: None, payload: v };

    // Convert it into a tracked permission.
    let (ptr, Tracked(points_to)) = PPtr::<Node<V>>::new(node);

    // Update the tail pointer.
    self.tail = Some(ptr);

    // Update the 'next' field of the old tail node.
    // First, take the tracked permission of the old tail node.
    let tracked mut old_tail_pointsto: PointsTo<Node<V>> =
        self.ghost_state.borrow_mut().points_to_map.tracked_remove((self
            .ghost_state@.ptrs.len() - 1) as nat);
    // Take the node out of the permission by proving address match and
        initialization. (this set .is_init () to false)
    assert(old_tail.addr() == old_tail_pointsto.addr() && old_tail_pointsto
        .is_init ()) by {
        old(self).perms_wf();
    };
```

```
36        let mut old_tail_node = old_tail.take(Tracked(&mut old_tail_pointsto));
37
38        // Set the 'next' pointer of the old tail.
39        old_tail_node.next = Some(ptr);
40        // Put the old tail node into the permission (this set .is_init() to
              true) and return the permission to the map.
41        // Note: 'is_init()' is now set back to true.
42        old_tail.put(Tracked(&mut old_tail_pointsto), old_tail_node);
43        proof {
44            self.points_to_map.borrow_mut().tracked_insert(old_tail,
                  old_tail_pointsto);
45        }
46
47        // Update the ghost sequence.
48        proof {
49            self.ptrs@ = self@.ptrs.push(ptr);
50        }
51
52        // Insert the new node's permission.
53        proof {
54            self.points_to_map.borrow_mut().tracked_insert(ptr, points_to);
55        }
56
57        // Verify that invariants still hold.
58        assert(self.wf());
59    }
```

This process illustrates how the tail is updated: the invariants guarantee the existence and state of the old tail's permission. The function temporarily takes ownership of this permission, extracts and modifies the node, and then restores the permission map, finally inserting the permission of the new node.

3.4 Triggers

Triggers are a mechanism in Z3 (and other SMT solvers) that control when quantified formulas (`forall` and `exists`) are instantiated during solving. They are crucial for efficiency, as indiscriminate instantiation can lead to infinite or

exponential case explosion. Z3 uses E-matching (equality matching modulo equality) to match patterns against ground terms.

Consider the following code:

```
spec fn is_even(x: usize) -> bool {
    x % 2 == 0
}
proof fn foo(seq: Seq<usize>, x: usize)
    requires
        forall |i: usize|
            #![trigger seq.contains(i)]
            seq.contains(i) ==> is_even(i),
        seq.contains(x),
{
    assert(is_even(x)); // Passes
}
```

In Verus, `#![trigger expression]` annotates the trigger for a `forall` statement. In the precondition, we declare all integers in `seq` are even, using `seq.contains(i)` as the trigger. When Z3 encounters the ground term `seq.contains(x)`, it matches the trigger and instantiates the quantifier, producing another ground term `seq.contains(x) ==> is_even(x)`. The assertion `is_even(x)` is then proven. Using `is_even(i)` as the trigger would also work, though instantiation would occur only when Z3 sees a term matching that pattern.

3.5 Trigger Loops

Triggers can inadvertently cause infinite instantiation loops. For example:

```
proof fn foo(seq: Seq<usize>, x: int)
    requires
        forall |i: int|
            #![trigger seq[i]]
            0 <= i < seq.len() - 1 ==> seq[i] < seq[i + 1],
        0 <= x < seq.len() - 10,
```

```
7  {
8      assert(seq[x] < seq[x + 3]); // Passes
9  }
```

Here, seeing `seq[x]` triggers instantiation, yielding `seq[x] < seq[x + 1]`. This new term contains `seq[x + 1]`, triggering another instantiation, and so on. For a bounded range, this chain eventually terminates, allowing the proof to succeed.

However, with a larger offset:

```
1  proof fn foo(seq: Seq<usize>, x: int)
2      requires
3          forall |i: int|
4              #![trigger seq[i]]
5              0 <= i < seq.len() - 1 ==> seq[i] < seq[i + 1],
6          0 <= x < seq.len() - 10000,
7  {
8      assert(seq[x] < seq[x + 9999]); // Fail
9  }
```

Z3 employs internal heuristics to limit recursion depth, attempting to mitigate such trigger loops. Nevertheless, these heuristics cannot eliminate all overhead. Trigger loops involving multiple quantifiers can still cause exponential or infinite instantiation cascades, making Z3 extremely slow or causing it to run out of memory—a well-known challenge in SMT solving.

CHAPTER 4

VERUS INTERNALS

This chapter delves into the internals of Verus, covering the transformations and lowerings from Verus source code to its final verification state. It addresses advanced topics in SMT-based verification frameworks and is intended for readers with interests extending beyond OS kernel verification.

4.1 The Verus Procedural Macros: `verus!`

Recall from Chapter 1 the following Verus code example:

```
1 pub fn add(x: usize, y: usize) -> (ret: usize)
2     requires
3         x + y <= usize::MAX,
4     ensures
5         ret == x + y,
6 {
7     x + y
8 }
```

Readers familiar with Rust will notice that `(ret: usize)`, `requires`, and `ensures` are not part of Rust's syntax tree. Indeed, submitting this code directly to the Rust compiler results in a syntax error:

```
1 error: expected one of `!`, `(`, `)`, `+`, `,`, `::`, or `<`, found `:`
2   |
3   | pub fn add(x: usize, y: usize) -> (ret: usize)
```

```
4     |                                                    ^ expected one of 7 possible
          tokens
```

Since Verus relies on the Rust compiler for type checking and borrow checking, the most practical approach – instead of completely rewriting the ever evolving Rust compile – is to retrofit Verus syntax into Rust syntax. The following subsection illustrates as an example of Verus's rewriting of `requires` / `ensures` clauses and return values.

4.1.1 Rewriting Requires/Ensures Clauses and Return Values

For the `requires` / `ensures` clauses and return values `ret`, the `verus!` macro wraps their contents into two Rust closures with special names, placed at the beginning of the function body:

```
1  pub fn add(x: usize, y: usize) -> usize
2  {
3      verus_builtin::requires(|| {
4          x + y <= usize::MAX
5      });
6      verus_builtin::ensures(|ret| {
7          verus_builtin::constrain_type(ret, add(x, y));
8          ret == x + y
9      });
10     x + y
11 }
```

The functions `verus_builtin::requires`, `verus_builtin::ensures`, and `verus_builtin::constrain_type` are defined internally by Verus as follows:

```
1  pub fn requires<A>(_a: A) {
2      unimplemented!();
3  }
4  pub fn ensures<A>(_a: A) {
5      unimplemented!();
6  }
```

```
7  pub fn constrain_type<T>(_x: T, _y: T) -> bool {
8      unimplemented!();
9  }
```

Later, during the HIR-to-VIR lowering stage, Verus reassembles the contents of `requires` and `ensures` back into the function's pre- and postconditions. The purpose of this rewriting is to encode the pre- and postconditions with valid Rust syntax while preserving type information.

For instance, because the `requires` closure is inserted at the beginning of the function, the variables `x` and `y` refer to the function's input parameters and inherit their types. If a type error occurs, as in the following:

```
1  pub fn add(x: bool, y: usize) -> usize
2  {
3      verus_builtin::requires(|| {
4          x + y <= usize::MAX
5      });
6      x + y
7  }
```

Rust will catch the type error because a `bool` cannot be added to an `usize`:

```
1  error[E0599]: no method named ‘spec_add‘ found for type ‘bool‘ in the
       current scope
2    |
3    |          x + y <= usize::MAX,
4    |          ^^^^^ method not found in ‘bool‘
```

For the `ensures` clause, the return value must also be incorporated. Since the return value is not in scope at the beginning, Verus defines an input parameter `|ret|` named after the return value as the input of the closure in `verus_builtin::ensures` and uses the internal `constrain_type()` function to forcibly set its type before the Rust compiler processes the rest of the postcondition.

4.2 From Rust HIR to Verus VIR

After the procedural macro transforms the Verus source code, `rustc` lowers the Abstract Syntax Tree (AST) into Rust's High-Level Intermediate Representation (HIR) and performs name resolution and type checking.

4.2.1 What Constitutes Rust HIR?

The following transformations occur during the AST-to-HIR conversion:

- Macros are fully expanded.
- Syntactic sugar is fully desugared.
- Each expression is assigned a unique identifier (HIR ID).

Steps (1) and (2) are straightforward: they essentially expand the source AST completely. To understand (3), consider this code:

```
1 fn bar() {
2       let x = 233;
3       let x = 2333;
4       let y = x;
5 }
```

To the naked eye, the first and second `x` appear identical even though they are completely different values. In HIR, however, after name resolution, each variable receives a unique identifier:

```
1 fn bar(hir_id :(0) ) () {
2       let x@1 (hir_id :(0:0) ) = const 233;
3       let x@2 (hir_id :(0:1) ) = const 2333;
4       let y@1 (hir_id :(0:3) ) = Path(x@2)(hir_id :(0:2) ) ;
5 }
```

Thus, the `let` expression for `y` correctly refers to `x@2`.

Although Rust HIR itself contains no type information, `rustc` provides interfaces (e.g., `typeck()`) to type-check a given scope. For the above example, Verus first type-checks the entire function `bar` and then queries the type of any expression within the function body using its HIR ID. For instance, to determine the type of `y`, Verus queries the type of the `Path` expression with HIR ID `0:2`.

4.2.2 Why Verify at the HIR Level?

Most Rust-based verification and static analysis frameworks [2], [30], [40], [48], [56] operate on Mid-level Intermediate Representation (MIR), as its design is more suitable for program analysis. However, Verus primarily operates on HIR for performance reasons, despite the additional complexity of handling language-specific features like traits, generics, and polymorphism. HIR is closer to the source code than MIR, retaining more of the program's original logic and fewer machine-level details. The rationale is that as more lowering has been performed to the original code, more details unrelated to verification are introduced and the harder it is to extract the original logic of the program.

To understand this concept better, consider an extreme scenario in which we lower the source code all the way to the assembly code:

```
1 pub fn add(x: usize, y: usize) -> usize
2 {
3     x + y
4 }
```

The logic is straightforward: the return value is the sum of the two inputs. However, if this code is lowered all the way to assembly:

```
1 push %rbp
2 mov %rsp,%rbp
3 mov %edi,−0x4(%rbp)
4 mov %esi,−0x8(%rbp)
5 mov −0x4(%rbp),%edx
6 mov −0x8(%rbp),%eax
7 add %edx,%eax
8 pop %rbp
9 ret
```

The code includes multiple stack relative load/store operations, stack push/pop, saving/restoring stack base pointer, and finally a `add` instruction, which is the only thing we care about. Extracting the original logic becomes extremely difficult after introducing machine-level details. Even if the verifier no longer needs to reason about pointer aliasing or type errors at this stage, verification complexity increases substantially.

4.2.3 Lowering HIR to Verus VIR

After HIR, Verus lowers Rust HIR into Verus Verification Intermediate Representation (VIR). During this phase, Verus performs the following steps:

1. Reassembles Verus-specific language constructs (e.g., pre-/postconditions) from Rust HIR.
2. Gathers type information and source spans for each VIR expression from the type-checking results.
3. Constructs the entire crate, including functions, structs, and traits.
4. Performs mode checking.
5. Removes unnecessary items (e.g., unused dependencies from other crates).

For step (1), using our earlier example of `verus_builtin::requires` and `verus_builtin::ensures`, during HIR-to-VIR lowering, Verus identifies these special function calls, extracts the pre-/postconditions as well as other Verus-specific items, and restores them to the original form while removing the placeholder calls.

For step (2), when converting HIR expressions to VIR, Verus attaches the corresponding type information and source span (the original location in the source file) to each VIR expression. These spans are used for error reporting, as seen in the earlier error message for `x + y`.

For step (3), recall that the smallest unit of verification in Verus is a function. In VIR, each function includes its signature, pre-/postconditions, body, etc. After reassembling verification-specific items, each function (without loops) is converted into a single verification unit: given the preconditions and all the operations in the body, the verifier checks all assertions and the postcondition.

For performance reasons, each loop, by default, is treated as a separate verification unit. The loop invariants have to be proven before the loop and after each iteration and also act as the pre- and postcondition of the loop – any other state changes in the loop body are invisible to outside.

For step (4), recall that variables in Verus have three modes (`spec`, `proof`, and `exec`), each with different allowed operations. Mode checking enforces these rules.

After all well-formedness checks, Verus reduces the verification scope. Since a crate may `use` items from other crates, verifying all items from all crates is

unnecessary. To achieve step (5), Verus first includes all items from the current crate (ensuring they will all be verified) and then recursively includes all items used by the current crate. Note that Verus does not include the bodies of functions from other crates—only their pre-/postconditions, as those functions are verified within their own crates.

4.3 Verus Lifetime Checks

As mentioned earlier, Verus relies on the Rust compiler for lifetime checking. However, unlike regular Rust code where all values follow Rust's ownership rules, Verus values in `spec` mode do not adhere to these rules, for two reasons: (1) specification code does not affect the program's lifetime behavior, and (2) proofs sometimes require accessing, referencing, or copying values in ways that violate Rust's lifetime rules (e.g., pushing a copy of a `PPtr<V>` into `self.ptrs` in the doubly linked list example). Note that *tracked* (linear ghost) values can influence lifetime behavior and thus memory safety; consequently, all tracked values reside in `proof` mode and must follow Rust's lifetime rules.

For lifetime checking, after HIR-to-VIR lowering, Verus uses mode information to erase `spec` code from HIR and then invokes Rustc's lifetime checker. Specifically, Verus builds Typed HIR (THIR)—an intermediate representation between HIR and MIR—by attaching type information from `typeck()` to HIR while ignoring HIR expressions with mode `spec`. Verus then calls `mir_borrowck` from Rustc to perform lifetime checking on this modified THIR.

4.4 VIR to SST (Statement-Oriented Syntax Tree)

4.4.1 Review: Expressions versus Statements

An expression is a piece of code that evaluates to a value, such as:

- An operation: `a + b * 3`
- A function call: `f(x + 1)`
- A Boolean expression (common in spec code): `x > 5`.

A statement is a complete unit of execution that performs an action (often with side effects), such as:

- Variable assignment: `let y = x > 5;`
- Any expression followed by a semicolon: `return;`

Rust's expression-oriented design allows expressions to contain statements and thus mutate state. However, SMT solvers primarily operate on logical formulas (side-effect-free expressions), not imperative statements. The main goal of the VIR-to-SST stage is to convert Rust-style expressions (which may have side effects) into SMT-style expressions (side-effect-free), augmented with additional statements that encode state changes. Additional checks are also performed at the SST stage before final conversion to SMT queries.

Generally, the VIR-to-SST stage transforms VIR expressions—which may contain statements and side effects—into a sequence of statements paired with a side-effect-free expression that returns a value. Consider the following block expression containing a statement:

```
1  let mut y = 0;
2  let x = {
```

```
3         y += 1; // statement
4         y + 233
5     };
6     assert(x == 234);
```

This code is converted into:

```
1   let y@0!0 = 0;
2   let y@0!1 = y@0!0 + 1;
3   let x@0 = y@0!1 + 233;
4   assert(x@0 == 234);
```

The original block expression `{ y += 1; y + 233 }` contains a statement. In SST form, the block expression is broken down into two statements:

`let y@0!1 = y@0!0 + 1;` and `let x@0 = y@0!1 + 233;`.

In addition, in `y@0!0`, `@0` is a unique identifier for the variable `y`; if another `y` were declared, it would be named `y@1`. The `!0` is a version number: because SMT does not support mutable variables, each modification creates a new SMT value with an incremented version number.

4.5 Introduction to SMT

4.5.1 Sorts, Functions, and Datatypes in SMT

In Satisfiability Modulo Theories (SMT), a `sort` is similar to a type in programming languages. It defines the domain from which constants, variables, or term values are drawn. For example, `Bool` is a predefined sort in Z3, corresponding to Rust's `bool`. To define a new sort without parameters:

```
1 (declare-sort Person 0)
```

Here, `Person` is the sort name, and `0` indicates it takes no parameters.

In SMT, a `fun` (function) is closer to a mathematical function than a programming language procedure: it maps input values (of specific sorts) to an output value (of a specific sort). Functions are central for encoding logical constraints and system models. A function declaration looks like this:

```
(declare-fun age (Person) Int)          ; Maps a Person to an Int
```

This function is declared but not defined; the solver only knows that `age` is some mapping from `Person` to `Int`. Its exact values are uninterpreted until constrained. A function can also be defined as a macro:

```
(define-fun isAdult ((p Person)) Bool
    (>= (age p) 18) ; Uses the 'age' function in its definition
)
```

Here, `(isAdult p)` is equivalent to `(>= (age p) 18)`.

To constrain a function:

```
(declare-const bob Person)          ; declare a person bob
(assert (= 20 (age bob)))           ; set bob's age
```

This sets the output of `age` for input `bob` to 20.

SMT datatypes are structured types defined with `declare-datatype`, similar to algebraic data types in functional languages. They provide constructors that create values of the datatype, possibly containing other values. For example:

```
(declare-datatype Person
  ( (person (name String) (age Int) (height Real)) )
)
; Usage: (person "Bob" 20 6)
```

Here, `Person` is the datatype name, and `person` is a constructor with three fields: `name`, `age`, and `height`.

4.6 Encoding Function Body Checks

In Verus, a function is the smallest unit of verification. The SMT encoding of each function consists of four main parts:

1. Variable declarations.
2. Introduction of preconditions.
3. The function body.
4. Postcondition checks.

Consider the following Verus function:

```
fn foo(x: usize) -> (ret: usize)
    requires
        x <= 100,
    ensures
        ret <= 101,
{
    assert(x != 101);
    return x;
}
```

Here our verification goal is: given the preconditions of the function, check if all the assertions and postconditions are satisfied.

To do this, Verus produces the following SMT code:

```
(push)
;; parameters
(declare-const ret! Int)
(declare-const x! Int)
(declare-const tmp%1 Bool)
(assert (uInv SZ x!)) ; integers size of x
;; precondition
```

```
(assert(<= x! 100))

;; assertion failed
(declare-const %%location_label%%0 Bool)

;; postcondition not satisfied
(assert
    (not (=>
            (= tmp%1
                (not (= x! 101))
            )
            (and
                (=> %%location_label%%0 tmp%1)
                (=>
                    tmp%1
                    (=>
                        (= ret! x!)
                        (=> %%location_label%%1 (<= ret! 101))
                    )
                )
            )
        )
    )
)
(check-sat)
(pop)
```

Verus first declares the variables. In this case, since no variable is mutable, each variable only has one version: `(declare-const ret! Int)` and `(declare-const x! Int)`. Immediately after, Verus uses `(assert(uInv SZ x!))` to set the range of `x` to the range of Rust `usize`. Then, Verus introduces the preconditions: `(assert(<= x! 100))`.

To check the function body and postconditions, Verus introduces two location labels, `%%location_label%%0` and `%%location_label%%1`, for the assertion and postcondition, respectively. The outermost `not (=> ...)` structure implements a proof by contradiction: it asks whether there exists an `x!` such that

either the assertion or the postcondition is false. If such an `x!` exists, some `%%location_label%%` will be set to true. Z3 can generate counterexamples, where a true `%%location_label%%` indicates the location of the failed assertion or postcondition.

The `push` and `pop` commands create a local SMT scope for verifying a single function. After `pop`, all local values declared after `push` go out of scope.

4.7 Encoding Pre- and Postconditions

To expose pre- and postconditions to callers of `foo`, Verus generates a function to check the precondition:

```
(declare-fun req%foo. (Int) Bool)
(declare-const %%global_location_label%%0 Bool)
(assert
 (forall ((x! Int)) (!
   (= (req%foo. x!) (=>
     %%global_location_label%%0
     (<= x! 100)
   ))
)))
```

Simultaneously, Verus generates an unchecked postcondition for callers:

```
(declare-fun ens%foo. (Int Int) Bool)
(assert
 (forall ((x! Int) (ret! Int)) (!
   (= (ens%foo. x! ret!) (and
     (uInv SZ ret!)
     (<= ret! 101)
   ))
 )
))
```

Now, if we call `foo` in Verus code:

```
let z = foo(1);
```

Verus generates the following SMT to check the precondition on input `1` and ensure the postcondition holds for the return value `z`:

```
(declare-const z@ Int)
;; precondition not satisfied
(declare-const %%location_label%%0 Bool)
(assert
  (not (and
    (=> %%location_label%%0 (req%foo. 1))
    (=>
      (ens%foo. 1 z@)))))
```

First, Verus checks the precondition similarly to an `assert` with `(=> %%location_label%%0 (req%foo. 1))`. Note that Verus does not recheck the type of `1`, as type checking is already performed by Rustc. After the precondition, Verus invokes `ens%foo` to constrain `z@`.

4.7.1 Encoding Rust Structs

Structs in Rust are custom data types that group related data. They are a primary tool for creating complex, structured data in Rust. Consider a simple Rust struct:

```
struct Struct {
    x: usize,
    y: bool,
}
```

Verus first defines a global sort `Type` to represent all types in Verus:

```
(declare-sort Type 0)
```

For the struct `Struct`, Verus defines a datatype, accessor functions, and a type constant:

```
(declare-datatypes (Struct. 0)
    ((Struct./Struct (Struct./Struct/?x Int) (Struct./Struct/?y Bool)))
)
(declare-fun Struct./Struct/x (Struct.) Int)
(declare-fun Struct./Struct/y (Struct.) Bool)
(declare-const TYPE%Struct. Type)
```

This SMT code defines a datatype `Struct.` with two fields, a constructor `Struct./Struct`, two accessor functions, and a constant `TYPE%Struct.` of sort `Type` representing the struct's type.

Notice that Verus defines its own accessor functions rather than using the built-in field accessors. This is due to Z3's trigger handling: default field functions are prone to trigger loops, which can cause verification timeouts.

If we initialize a value of type `Struct` in Rust:

```
let st = Struct {
    x: 1,
    y: false,
};
```

Verus produces the following SMT:

```
(declare-const st@ Struct.)
(assert (= st@ (Struct./Struct 1 false)))
```

This declares a constant `st@` of datatype `Struct.` and sets it equal to the constructed value `(Struct./Struct 1 false)`.

4.7.2 Polymorphism

4.7.2.1 Poly, Type, and Decoration

Real-world SMT encoding for Rust verification is more complex due to polymorphism, which SMT lacks direct support for. To support polymorphism, Verus introduces new sorts: `Poly` to represent values and `Dcr` (decoration) to represent type modifiers:

```
(declare-sort Poly 0)
(declare-sort Dcr 0)
```

Polymorphism is encoded by coercing expressions to and from the universal `Poly` type via `Box` and `Unbox` coercions.

Verus also introduces a predicate to connect `Poly` values to their types:

```
(declare-fun has_type (Poly Type) Bool)
```

A decoration in Verus can be one of: `Ref`, `Mut Ref`, `Sized`, `Box`, `Rc`, `Arc`, `Ghost`, `Tracked`, `Never`, or `const ptr`. For example, Verus defines a function `TRACKED` to generate a `Tracked<T>` decoration from a decoration of type `T`:

```
(declare-fun TRACKED (Dcr) Dcr)
```

If we create a `Tracked` value from a value of type `T`:

```
fn foo<T>() {
    let tracked_t = Tracked(T);
}
```

Verus produces:

```
(declare-const T&. Dcr) ; decoration of T
(declare-const T& Type) ; type of T
```

```
(declare-const t! Poly) ; the value of t
(declare-const tracked_t&. Dcr) ; decoration of tracked_t
(declare-const tracked_t& Dcr) ; type of tracked_t
(assert (= tracked_t&. (TRACKED T&.))) ; decoration of tracked_t is a
    Tracked T
(assert (= tracked_t& T&)) ; type of tracked_t is still T
```

This illustrates how Verus encodes a `Tracked` decoration for a polymorphic variable.

4.7.2.2 Encoding Polymorphism with Poly

Consider a polymorphic Rust struct:

```
struct StructT<T> {
    x: usize,
    y: bool,
    t: T
}
```

The type parameter `T` cannot be directly expressed in SMT, as it can be any type with any decoration. Verus generates the following SMT code:

```
(declare-datatypes (StructT. 0)
    (StructT./StructT (StructT./StructT/?x Int) (StructT./StructT/?y Bool)
        (StructT./StructT/?t Poly))
)
(declare-fun StructT./StructT/x (StructT.) Int)
(declare-fun StructT./StructT/y (StructT.) Bool)
(declare-fun StructT./StructT/t (StructT.) Poly)
(declare-fun TYPE%StructT. (Dcr Type) Type) ; get the type of StructT
(declare-fun Poly%StructT. (StructT.) Poly) ; Box StructT into a Poly
(declare-fun %Poly%StructT. (Poly) StructT.) ; Unbox a Poly into a StructT
```

Note that field `t` has type `Poly`. Verus also generates helper functions `Poly%StructT.` and `%Poly%StructT.` to box and unbox `StructT` values. For a

`StructT` instantiated with type `V`, `TYPE%StructT.` takes the type and decoration of `V` and returns an SMT value of sort `TYPE` representing the instantiated type.

Verus also generates a series of axioms to ensure correctness. For example:

```
(assert
 (forall ((T&. Dcr) (T& Type) (x Poly))
  (!
   (=>
    (has_type x (TYPE%StructT. T&. T&))
    (has_type (StructT./StructT/t (%Poly%StructT. x)) T&))
)))
```

This axiom ensures that for any boxed `StructT` value `x` instantiated with type `T`, if we unbox `x` and access field `t`, it has type `T`.

If we initialize `StructT` with a polymorphic type `T`:

```
fn foo<T>(t: T) {
    let st_t = StructT {
        x: 1usize,
        y: false,
        t: t,
    };
}
```

Verus produces:

```
(declare-const st_t@ StructT.)
(declare-const T&. Dcr)
(declare-const T& Type)
(declare-const t! Poly)
(assert (has_type t! T&))
(assert (= st_t@ (StructT./StructT 1 false t!)))
```

Here, a value of type `StructT.` is declared and initialized with the constructor, passing `1`, `false`, and `t!`. The `has_type` predicate ensures `t!` has type `T`.

4.8 Encoding Traits

Traits are Rust's mechanism for defining shared behavior, similar to interfaces but more powerful. They enable polymorphism and code reuse by specifying required functionality. Consider the following Verus trait with a single trait function:

```
trait Tr {
    spec fn spec_add(self, v: Self) -> Self;
}
```

For the above trait, Verus generates:

```
(declare-fun tr_bound%Tr. (Dcr Type) Bool)
(declare-fun Tr.spec_add.? (Dcr Type Poly Poly) Poly)
```

The predicate `tr_bound%Tr` indicates whether a type satisfies the trait bound; the function `Tr.spec_add.?` defines the trait method `spec_add`. Since by the function signature, `self` and `v` are both `usize`, `Tr.spec_add.?` only takes one pair of declaration and type as the first two parameters, and two parameters of `Poly` for `self` and `v`.

If we implement `Tr` for `usize`:

```
impl Tr for usize {
    spec fn spec_add(self, v: Self) -> Self {
        (self + v) as usize
    }
}
```

Verus generates a type-specific implementation:

```
(assert
(forall ((self! Poly) (v! Poly)) (!
    (= (Tr.spec_add.? $ USIZE self! v!) (I (uClip SZ (Add (%I self!) (%I v
        !)))))
```

```
4 )))
```

`Tr.spec_add.? $ USIZE self! v!` is the SMT form of `spec_add(self: usize, v: usize)`. `$` is the default declaration for `Sized` types. `USIZE` is the built-in predefined `TYPE` for Rust `usize`. `I` and `%I` are the built-in functions to box and unbox integers in Rust into and from `Poly`. `uClip` is another built-in function that clips an integer into a given range. Since we pass in `SZ`, we model the behavior of `as usize`.

If we declare a generic type `T` with trait bound `Tr`:

```
1 fn foo<T: Tr>(t: T) {
2     // function body
3 }
```

Verus generates:

```
1 (declare-const T&. Dcr)
2 (declare-const T& Type)
3 (declare-const t! Poly)
4 (assert (sized T&.))
5 (assert (tr_bound%Tr. T&. T&))
6 (assert (has_type t! T&))
```

This asserts that `T` is sized and satisfies `Tr`.

4.9 Compiling to Binary

After successful verification, Verus invokes the procedural macro again to erase all proof code completely. The resulting pure Rust code is then compiled by `rustc` into binary.

CHAPTER 5

IMPLEMENTING ASYNC IN VERUS

In this chapter, we will cover the details of the implementation of asynchronous (async) functions in Verus. To the best of our knowledge, Verus is the first formal verification framework that supports verifying async functions. Readers may take this chapter as a practical guide for modifying their own fork of Verus or other verification frameworks.

5.1 Async in Rust

An async function returns a `Future`, which is a value representing a computation that may complete later. The function body is *lazy*—it does not run until the future is actively polled. When a regular Rust function returns, its stack frame is freed, all local variables go out of scope, and any references passed to the function are released. Unlike regular Rust functions, async functions do not actually return at the point of invocation. Instead, they spawn a user-level, lightweight thread-like task, which is managed by the async runtime. Paired with `Future` is the `await` keyword. When a program calls `await`, it yields execution to other threads or async tasks, enabling other tasks to run while waiting for an operation to complete. `await` returns only when the async task finishes.

Async functions in Rust are particularly useful when handling many I/O-bound tasks concurrently without the overhead of OS threads, notably in servers, network clients, and GUI applications. Rust's implementation is uniquely valuable because it permits custom async runtimes, provided they implement the `Future` trait.

5.1.1 Rust Async Lifetime Rules for Verification

Because the body of an async function is lazy, its lifetime behavior differs from that of synchronous functions. Consider the following Rust functions:

```
1  fn sync_add(x: &mut usize) {
2      *x += 1;
3  }
4  fn async_add(x: &mut usize) {
5      *x += 1;
6  }
7  fn foo() {
8      let mut x = 0;
9      sync_add(&mut x); // x is mutably borrowed
10     let y = x;        // OK: borrowing ends after sync_add() returns
11
12     let future = async_add(&mut x); // x is mutably borrowed again
13     let z = x;        // Fail: borrowing ends only after the future is
           awaited
14     future.await;     // async_add() ends after await returns
15     let z = x;        // OK
16 }
```

Rust's lifetime rules handle async functions without significant additional complexity: references passed to an async function call remain owned by the call until `await` returns.

This rule drastically reduces the verification complexity of async code: it turns a concurrent async program into a conceptually sequential one with

delayed function return points. Verus does not need to reason about borrowings across different async function calls or about data races.

5.2 Prerequisite: Return Value Encoding

In older versions of Verus, the return value type was encoded by explicitly declaring the argument type in the closure passed to `verus_builtin::ensures`:

```
1  pub fn add(x: usize, y: usize) -> usize
2  {
3      verus_builtin::requires(|| {
4          x + y <= usize::MAX
5      });
6      verus_builtin::ensures(|ret: usize| {
7          ret == x + y
8      });
9      x + y
10 }
```

This approach poses a problem for async functions because their return type is an opaque type (`impl Future`): the actual type is determined by the async runtime and is guaranteed to satisfy the `Future` trait. Rust does not allow opaque types as explicit type annotations for closure arguments. Yet, we need to fix the return value type to perform type checking for the postcondition inside the closure body.

Therefore, as shown in Chapter 4, we introduced a new built-in function `verus_builtin::constrain_type()`, which takes two arguments of the same type. In the closure, we omit the type annotation for the return value and instead call `constrain_type()` to restrict the type of the return value. The first argument to `constrain_type()` is the return variable define as the (only) argument in the closure. The second argument is the return value of a call to the current function (a

recursive call to self) with the same arguments, in which the returned value of the call has the same type as the return value (because it is indeed the return value).

The resulting code is the following:

```
1  verus_builtin::ensures(|ret| {
2      verus_builtin::constrain_type(ret, add(x, y));
3      ret == x + y
4  });
```

The Rust compiler infers that the type of `ret` is equal to that of `add(x, y)` (`usize`) and performs type checking on the rest of the postcondition based on this fact.

5.3 Prerequisite: Opaque Types

Rust's opaque types refer to the `impl Trait` syntax (e.g., `impl Future` is one application), which allows a function to return a type that implements a trait without exposing the concrete type. This section covers the necessary changes to support opaque types in Verus.

5.3.1 Opaque Types in Rust HIR

In Rust HIR, each opaque type has a constructor, and every value of that opaque type instantiates this constructor to obtain an instance. In HIR, the constructor is defined as:

```
1  pub struct OpaqueTy<'hir> {
2      pub hir_id: HirId,
3      pub def_id: LocalDefId,
4      pub bounds: GenericBounds<'hir>,
5      pub origin: OpaqueTyOrigin<LocalDefId>,
6      pub span: Span,
7  }
```

The `bounds` defines all the trait bounds of the opaque type, and can be instantiated if the opaque type takes generic arguments. Specifically, in `impl Future<Output = T>`, `Output = T` is a bound associated with `Future` and needs a generic argument `T` to instantiate. Note: `impl Future` itself is another bound on this opaque type. Thus, for each opaque type defined by a function's return, Rustc creates a new opaque constructor as if it were a completely new type (in some sense, it is). For values of opaque types, their types are represented as:

```
pub struct AliasTy<TyCtxt<'tcx>> {
    pub args: TyCtxt::GenericArgs,
    pub def_id: TyCtxt::DefId,
     ...
}
```

Here, `def_id` refers to the opaque type constructor, and `args` are the generic parameters passed to the *function call* that returns the opaque type. Why does Rustc design it this way? The reasoning is straightforward: the type of the opaque type is purely dependent on the generic arguments passed to the function call. If the same function is called with the same generic arguments, the return values are guaranteed to have the same type, even when they are opaque.

Consider the following Rust code:

```
trait Tr {
    type Ty;
}
fn foo<T>() –> impl Tr<Ty = T> {
    // Function body omitted
}
fn bar<T,V>(){
    let opaque_value_1 = foo::<T>();
    let opaque_value_2 = foo::<V>();
}
```

Opaque type `impl Tr<Ty = T>` takes one generic parameter to instantiate, since `foo<T>` takes one generic argument. `opaque_value_1` has `T` as the `args`, and `opaque_value_2` has `V` as the `args`. As a result, when instantiated, `opaque_value_1` is `impl Tr<Ty = T>` and `opaque_value_2` is `impl Tr<Ty = V>`.

For nested opaque types, consider the following Rust code (assume type checking is passed):

```
1  trait Tr {
2      type Ty;
3      fn dummy_add(self, v: Self) -> Self;
4  }
5  fn foo() -> impl Tr<Ty = impl Tr> {
6      // Function body omitted
7  }
```

In HIR, two opaque types are defined: `OpaqueTy_1` for `impl Tr<Ty = impl Tr>` and the inner `OpaqueTy_2` for `impl Tr`. For `OpaqueTy_1`, it takes no generic arguments to instantiate and has two bounds: one bound requires `OpaqueTy_1` to satisfy `Tr`, and another bound requires the associated type `Ty` of the trait bound `Tr` to be `OpaqueTy_2`. `OpaqueTy_2` similarly takes no generic arguments but has a single bound requiring it to satisfy trait `Tr`.

5.3.2 Type Checking Opaque Types in Rust

When the return value is converted to an opaque type at function exit, Rust performs all necessary type checking to ensure that the return value satisfies the trait bounds of the opaque type. If the return value does not satisfy those bounds, Rustc emits an error:

```
1  trait Tr {
```

```
2       type Ty;
3       fn dummy_add(self, v: Self) -> Self;
4   }
5   impl Tr for usize {
6       // trait impl omitted
7   }
8   fn foo() -> impl Tr {
9       true
10  }
11  error[E0277]: the trait bound 'bool: Tr' is not satisfied
12      |
13      | fn foo() -> impl Tr {
14      |                ^^^^^^^ the trait 'Tr' is not implemented for 'bool'
15      |       true
16      |       ---- return type was inferred to be 'bool' here
```

When a trait function is invoked on a value of an opaque type, Rustc also performs the necessary type checking:

```
1   trait Tr {
2       type Ty;
3       fn dummy(self);
4   }
5
6   trait Tr2 {
7       type Ty;
8       fn dummy2(self);
9   }
10
11  impl Tr for bool {
12      // trait impl omitted
13  }
14  fn foo() -> impl Tr<Ty = impl Tr> {
15      true
16  }
17
18  fn baz() {
19      let opaque_v = foo();
20      opaque_v.dummy2();
21  }
```

Trigger the following error:

```
error[E0599]: no method named ‘dummy2‘ found for opaque type ‘impl Tr<Ty = impl Tr>‘ in the current scope
  |
  |       opaque_v.dummy2();
  |                 ^^^^^^
```

These type checking rules for opaque types eliminate the need for additional type checking during verification, thereby drastically reducing verification complexity.

5.3.3 Encoding Opaque Types in Verus

To simplify verification and stay close to Rust's internal structures, Verus's implementation of opaque types follows the same principles.

First, Verus lowers the HIR opaque type constructor into VIR:

```
pub struct OpaqueTypeX {
    pub name: Path,
    pub typ_params: Typs,
    pub typ_bounds: GenericBounds,
}
```

In VIR, each opaque type records its unique name (according to its defining path), the generic type parameters required for instantiation, and its (uninstantiated) generic bounds. Each instance of an opaque type in VIR is represented as:

```
pub enum TypX {
    ...
    Opaque {
        def_path: Path,      // path of the opaque type constructor
        args: Typs,          // arguments for instantiating the opaque type constructor
    },
}
```

It records the path of the opaque type constructor it refers to, and the arguments used to instantiate that constructor.

To mimic the "instantiation" behavior of Rust's opaque type instances, for the Rust code example above, Verus generates the following SMT functions:

```
;; Opaque-Type-Constructors
(declare-const DCR%foo.opaque0.opaque0. Dcr)
(declare-const TYPE%foo.opaque0.opaque0. Type)
(assert
 (and
  (tr_bound%Tr. DCR%foo.opaque0.opaque0. TYPE%foo.opaque0.opaque0.)
  (sized DCR%foo.opaque0.opaque0.)
))
(declare-const DCR%foo.opaque0. Dcr)
(declare-const TYPE%foo.opaque0. Type)
(assert
 (and
  (and
   (= DCR%foo.opaque0.opaque0. (proj%%Tr./Ty DCR%foo.opaque0. TYPE%foo.opaque0.))
   (= TYPE%foo.opaque0.opaque0. (proj%Tr./Ty DCR%foo.opaque0. TYPE%foo.opaque0.))
  )
  (tr_bound%Tr. DCR%foo.opaque0. TYPE%foo.opaque0.)
  (sized DCR%foo.opaque0.)
))
```

First, the constants `DCR%foo.opaque0.opaque0.` and `TYPE%foo.opaque0.opaque0.` refer to the decoration and type of `OpaqueTy_1` (the innermost opaque type `impl Tr`). Note: it is named `opaque0.opaque0.` since it is the first opaque type (`Ty = impl Tr`) defined in the first opaque type of the function (`impl Tr<Ty = impl Tr>`). Since `OpaqueTy_1` takes no arguments to instantiate, Verus declares two constants instead of functions. Verus then generates two axioms declaring the trait bounds of `OpaqueTy_1`: it satisfies `Tr` and, by default, the `Sized` trait. Second, for `OpaqueTy_0` (the outer opaque type `impl Tr<Ty = impl Tr>`), Verus declares an additional axiom setting the

associated type `Ty` (which can be accessed by `proj%%Tr./Ty DCR%foo.opaque0. TYPE%foo.opaque0.` and `proj%Tr./Ty DCR%foo.opaque0. TYPE%foo.opaque0.`) to be equal to `DCR%foo.opaque0.opaque0.` and `TYPE%foo.opaque0.opaque0.`.

If we create a value of type `OpaqueTy_0`:

```
let opaque_value = foo();
```

Verus generates the following SMT code:

```
;; Function-Specs foo
(assert
 (forall ((no%param Int) (%return! Poly)) (!
   (= (ens%foo. no%param %return!) (has_type %return! TYPE%foo.
       opaque0.))
)))

(declare-const opaque_value@ Poly) ; opaque_value is a Poly
(assert (ens%foo. 0 opaque_value@))

; opaque_value's type is set by the postcondition of foo
```

Since a value of an opaque type is guaranteed to be polymorphic, Verus uses `Poly` for `opaque_value`. Its decoration and type are set by the ensures function of `foo()`.

Now consider an opaque type with a type parameter:

```
fn foo<T>(t: T) -> impl Tr<Ty = T> {
    // Function body omitted
}
// create a value of opaque type with argument usize
let opaque_value = foo::<usize>(1usize);
```

Verus generates the following SMT code:

```
;; Opaque-Type-Constructors
```

```
(declare-fun DCR%foo.opaque0. (Dcr Type) Dcr)
(declare-fun TYPE%foo.opaque0. (Dcr Type) Type)
;; Axioms
(assert
 (forall ((T&. Dcr) (T& Type)) (!
   (and
    (and
     (= T&. (proj%%Tr./Ty (DCR%foo.opaque0. T&. T&) (TYPE%foo.opaque0. T&. T&)))
     (= T& (proj%Tr./Ty (DCR%foo.opaque0. T&. T&) (TYPE%foo.opaque0. T&. T&)))
    )
    (tr_bound%Tr. (DCR%foo.opaque0. T&. T&) (TYPE%foo.opaque0. T&. T&))
    (sized (DCR%foo.opaque0. T&. T&))
   )
)))

;; Function-Specs foo
(declare-fun ens%foo. (Dcr Type Poly Poly) Bool)
(assert
 (forall ((T&. Dcr) (T& Type) (t! Poly) (%return! Poly)) (!
   (= (ens%foo. T&. T& t! %return!) (has_type %return! (TYPE%foo.opaque0. T&. T&)))
)))

(declare-const opaque_value@ Poly)
(ens%foo. $ USIZE (I 1) opaque_value@)
```

Here, `DCR%foo.opaque0` and `TYPE%foo.opaque0` both take one type argument to instantiate. Therefore, `DCR%foo.opaque0. T&. T&` and `TYPE%foo.opaque0. T&. T&` return the instantiated decoration and type of `impl Tr<Ty = T>`. And the axioms ensure that if we take the projection type `Ty` from the opaque type (`impl Tr<Ty = T>`) by calling `proj%%Tr./Ty` and `proj%Tr./Ty` on it, we get `T` back. In the postcondition, `ens%foo.` uses the type of the first argument (the generic parameter `T`) to instantiate the opaque type and sets the type of the return value with the `has_type` function. In our example

where we `opaque_value` is returned from `foo::<usize>`, Verus first defines it as `opaque_value@`, a `Poly`, and calls `ens%foo.` with `$ USIZE` as the first two arguments (which are the decoration and type of the generic parameter `T`) to set the type of `opaque_value@` to the instantiated opaque type.

5.4 Expected Behavior of Async Function Postcondition

As mentioned in previous sections, lifetimes of async functions are handled by Rustc. For an async function, we only need to soundly capture the **delayed postcondition**. Consider the following Verus code:

```
1  async fn async_add_one(x: &mut usize) -> (ret: ())
2      requires
3          old(x) < usize::MAX,
4      ensures
5          x == old(x) + 1,
6  {
7      *x += 1;
8  }
9
10 async fn foo() {
11     let mut x = 0;
12     let f = async_add_one(&mut x);
13     assert(x == 1); // spec code bypasses lifetime checks but this should
           fail, as the state of x is unknown
14     f.await;
15     assert(x == 1); // passes
16 }
```

We need to accomplish two things:

1. Invalidate the state of any mutable references passed to the async function.
2. Delay the postcondition until `await` is called on the future.

Fortunately, (2) is already handled by Verus's treatment of mutable values. Each time a mutable value is mutated, a fresh SMT variable is created to represent its

new version. The state of the new version is set by the postcondition. Thus, if the postcondition is delayed, the state of the new version of the variable remains unknown to the SMT solver, which matches the expected behavior. We therefore only need to address (1).

Note: technically speaking, by the official definition in Rustc, all the mutable values passed to an async function call remain *unchanged* until the future is awaited. However, since these mutable values cannot be accessed by anything outside the async function call itself, it should be fine to give them *unknown* state. This is subject to change in the future.

Once again, Rust's implementation of async naturally lowers the verification burden. The future variable (`f`) returned by calling the async function is an ideal candidate for binding a *prophecy variable* [1]. Prophecy variables are a technique in formal verification used to reason about future events or nondeterministic choices. They "predict" or "record" information about execution to assist in proving correctness properties.

To model the future behavior of an invocation `Inv` of an async function with postcondition `Pos` :

1. Introduce a Boolean prophecy variable `Pro` with an uninterpreted (undefined) value. `Pro` indicates whether `Inv` has returned.
2. Rewrite the postcondition of the async function from `Pos` to `Pro ==> Pos` .
3. When `await` returns, bind `Pro` to true, thereby revealing that `Pos` holds.

Of course, `Pro` must follow all the rules of prophecy variables to ensure soundness.

5.5 Adding the Future Trait to Vstd

The `Future` trait is a native trait from Rust's standard library `std`, external to Verus. It must be imported into Verus's standard library `vstd`.

In Verus, one can import an external trait as follows:

```
#[ verifier :: external ]
trait Tr {
    fn f(&self);
    type X;
}

trait ExTr {
    type ExternalTraitSpecificationFor: Tr;

    fn f(&self)
        requires
            // Precondition omitted
        ensures
            // Postcondition omitted
        ;
    type X;
}
```

Here, the specially named associated type `ExternalTraitSpecificationFor` specifies which trait is being specified. With this specification in place, verified code can use the trait `Tr` and its members `f` and `X`; any use of `T::f` will be verified against the specification provided by `ExTr::f`.

Using `ExternalTraitSpecification`, Verus adds `std::future::Future` to `vstd` as follows:

```
#[ verifier :: external_trait_specification ]
pub trait ExFuture {
    type ExternalTraitSpecificationFor: core::future::Future;
    type Output;
}

pub trait FutureAdditionalSpecFns<T>: Future<Output = T> {
```

```
8       spec fn view(&self) -> T;
9       #[ verifier :: prophetic ]
10      spec fn awaited(&self) -> bool;
11  }
12
13  impl<V, T: Future<Output = V>> FutureAdditionalSpecFns<V> for T {
14      uninterp spec fn view(&self) -> V;
15      uninterp spec fn awaited(&self) -> bool;
16  }
```

Here, Verus uses `FutureAdditionalSpecFns` to add two additional spec functions for `Future`. `awaited()` is the prophecy variable for `Pro`; it evaluates to true iff the future has been `await`ed. It is uninterpreted so that the verifier does not know its value unless explicitly set. `view()` is another uninterpreted variable that is set equal to the return value when the future completes.

5.6 Async Function: HIR to VIR

5.6.1 Async Function in Rust HIR

Fortunately, after desugaring, an async function is not too far away from its original form. A simple async function in Rust:

```
1  async fn async_foo(x: usize) -> u64 {
2      let y = async_foo(x).await;
3      y
4  }
```

Its desugared HIR form is:

```
1  async fn async_foo(x@0: usize)
2  ->
3      / * impl Trait * / |mut _task_context: ResumeTy|
4  {
5      let x@1 = x@0;
6      {
7          let _t@0 =
8              {
```

```
 9                     let  y@0 =
10                         match into_future(async_foo(x@1)) {
11                             mut __awaitee =>
12                                 loop {
13                                     match unsafe {
14                                             poll(new_unchecked(&mut
                                                 __awaitee),
15                                                 get_context(_task_context))
16                                         } {
17                                         Ready { 0:  result  } => break result,
18                                         Pending {} => {  }
19                                     }
20                                     _task_context = (yield  ());
21                                 },
22                         };
23                     y@0
24                 };
25             _t@0
26         }
27     }
```

Do not be intimidated by the complexity. Let us walk through the HIR.

First, the return type `impl Future` is desugared into an opaque type `/*impl Trait*/|mut _task_context: ResumeTy|`. We do not need to concern ourselves with it, as we only need to gather the original logic of the function.

Second, the entire function body is wrapped in a special closure of kind `CoroutineDesugaring::Async`, which takes no input arguments:

```
 1  {
 2      let  x@1 = x@0;
 3      {
 4          let  _t@0 =
 5              {
 6                  // rest  of  the  function body
 7              };
 8          _t@0
 9      }
10  }
```

The body appears as a block, but it is actually a special closure. For simplicity and practicality, Verus should not deal with the complexity of this closure directly. Instead, we extract the original function body from the desugared HIR:

```
async fn async_foo(x@0: usize)
-> // we do not care about the return type yet
{
    let y@0 =
        match into_future(async_foo(x@1)) {
            mut __awaitee =>
                loop {
                    match unsafe {
                            poll(new_unchecked(&mut __awaitee),
                                get_context(_task_context))
                        } {
                        Ready { 0: result } => break result,
                        Pending {} => { }
                    }
                    _task_context = (yield ());
                },
        };
    y@0
}
```

However, at the beginning of the closure, all input parameters are rebound to the same name (e.g., `let x@1 = x@0;`). Recall that in HIR each variable is assigned a unique identifier (HIR id). Therefore, our second task is to resolve the input parameters to their rebound versions, so that later accesses to the rebound variables are correctly redirected to the parameters of the function, rather than to the rebound copies. After resolving, we obtain:

```
async fn async_foo(x@1: usize)
-> // we do not care about the return type yet
{
    let y@0 =
        match into_future(async_foo(x@1)) {
            mut __awaitee =>
                loop {
```

```
8                           match unsafe {
9                                   poll(new_unchecked(&mut __awaitee),
10                                      get_context(_task_context))
11                              } {
12                              Ready { 0:  result  } => break result,
13                              Pending {} => {  }
14                          }
15                          _task_context = (yield  ());
16                      },
17              };
18          y@0
19      }
```

Now the parameter `x@0` has been replaced by the rebound `x@1`.

Readers may have already guessed: the `match` and `loop` construct:

```
1   match into_future(async_foo(x@1)) {
2       mut __awaitee =>
3           loop {
4               match unsafe {
5                       poll(new_unchecked(&mut __awaitee),
6                           get_context(_task_context))
7                   } {
8                   Ready { 0:  result  } => break result,
9                   Pending {} => {  }
10              }
11              _task_context = (yield  ());
12          },
13  };
```

is the desugared form of the await expression `async_foo(x@1).await`. Verus only needs to extract the expression inside `into_future(async_foo(x@1))` to fully restore the function logic. Therefore, we get:

```
1   async fn  async_foo(x@1: usize)
2   -> // we do not care about the return type yet
3   {
4       let  y@0 = async_foo(x@1).await;
5       y@0
6   }
```

Verus then lowers the extracted HIR into VIR in a manner similar to non-async functions.

5.7 Rewriting the Postcondition

Our next step is to bind the postcondition to the prophecy variable (i.e., `future.awaited()`). For an async function with an ensures clause:

```
1  async fn async_add_one(x: &mut usize) -> (ret: ())
2      requires
3          old(x) < usize::MAX,
4      ensures
5          x == old(x) + 1,
6  {
7      // function body omitted
8  }
```

In the VIR, it is rewritten as:

```
1  async fn async_add_one(x: &mut usize) -> (
       _verus_internal_async_desugared_ret: impl Future<Output = ()>)
2      requires
3          old(x) < usize::MAX,
4      ensures
5          { let ret = _verus_internal_async_desugared_ret.view();
6            _verus_internal_async_desugared_ret.awaited() ==> x == old(x) +
                 1 },
7  {
8      // function body omitted
9  }
```

First, Verus creates a new return value `_verus_internal_async_desugared_ret` of type `Future` to represent the actual return value of the desugared async function. Second, for each expression in the ensures clause, Verus binds the return value of the function to the prophecy value `future.view()` as a placeholder, since the actual return value is only in scope when the future is awaited—yet the return

value is created when the async function returns. The expression is then connected with the prophecy value `future.awaited()`. Because `future.awaited()` remains uninterpreted before the future is awaited, we can soundly capture the undefined state of `x`. For example:

```
1  async fn foo() {
2      let mut x = 0;
3      let f = async_add_one(&mut x);
4      assert(x == 1); // fails
5      assert(x == 0); // fails
6  }
```

Both assertions fail because the state of `x` is unknown.

5.8 Encoding Await

When the future is `await`ed, the async function invocation conceptually ends. Rust's lifetime checks already correctly terminate all borrowings passed to the invocation, so Verus only needs to update the state of the prophecy variables. In the previous example, if we call `await` on `f`, the following behavior is expected:

```
1  async fn foo() {
2      let mut x = 0;
3      let f = async_add_one(&mut x);
4      assert(f.awaited() == true); // fails, f.awaited() is unknown
5      f.await;
6      assert(f.awaited() == true); // passes
7      assert(x == 1);              // passes
8  }
```

After `await` returns, `f.awaited()` is bound to `true`, thereby revealing the full postcondition to the verifier.

To achieve this, Verus introduces a new executable function for `FutureAdditionalSpecFns`:

```
pub trait FutureAdditionalSpecFns<T>: Future<Output = T> {
    fn exec_await(self) -> (ret: T)
        ensures
            self.awaited() == true,
            ret == self.view(),
        opens_invariants any;
}

impl<V, T: Future<Output = V>> FutureAdditionalSpecFns<V> for T {
    #[verifier::external_body]
    fn exec_await(self) -> (ret: V)
        ensures
            self.awaited() == true,
            ret == self.view(),
    {
        unimplemented!()
    }
}
```

First, Verus defines `exec_await` to capture the behavior of `await` as a function call that consumes ownership of the `future`. `exec_await` is marked with `opens_invariants any`, which disallows callers from calling `await` while holding any opened invariants, since `await` may yield execution to the async runtime. (We will discuss concurrency in later chapters.) The postcondition of `exec_await` sets the prophecy value `f.awaited()` to true and binds the return value of the async function to the prophecy value `f.view()`.

During the Verus VIR stage, after necessary checks, Verus replaces all `f.await` expressions with a call to `exec_await(f)`. To avoid any potential unsoundness from users accidentally calling `exec_await` in their code,

`exec_await` is left as `unimplemented!()`, so that Rustc emits an error if `exec_await` is invoked directly.

CHAPTER 6

RESEARCH CHALLENGES IN OS KERNEL VERIFICATION

6.1 Background of OS Kernel Verification

Historically, the complexity of systems software has kept it beyond the reach of formal verification. Although several projects have demonstrated the feasibility of developing verified kernels [13], [22], [24], [25], [32], [60], verification has required extensive formal expertise and significant human effort. For example, the development of the first formally verified microkernel, seL4, required 11 person-years (an additional 9 person-years were needed for the development of formal language frameworks, proof tools, and libraries [32]).

To address the complexity of verification, several projects have explored the development of programming languages with support for a high degree of proof automation [35] and even complete "push-button" verification that attempts direct translation of kernel code into a satisfiability modulo theories (SMT) expression **Push-Button-Verification-of-File-Systems-Verification-of-File-Systems**, [43], [45], [47]. While achieving nearly complete automation, such approaches have required numerous simplifying assumptions about the internal organization of the kernel and its interface [10]. For instance, Hyperkernel required all

execution paths in the kernel to be finite—the file open system call forced the user to provide an explicit file descriptor number for opening a file instead of allowing the kernel to choose an available one [45].

First, linear type systems [59] takes care of reasoning about aliasing and the heap, thereby enabling a significantly simpler SMT encoding of the program [2], [34], [37]. Second, a combination of linear types and ideas of permission-based reasoning [2], [5], [29], [34], [42] provides a practical way of reasoning about non-linear pointers that are typical in low-level kernel data structures such as linked lists and trees.

Yet, despite the impressive progress offered by recent SMT-based verifiers [2], [34], the practicality of verifying feature-rich, low-level system code remains an open debate [26]. Modern kernels implement their logic as a collection of recursive data structures with frequent pointer references and a combination of complex, non-linear lifetimes. Recursive specifications required to reason about recursive data structures are inherently difficult for SMT solvers, which do not support inductive proofs natively—a typical approach is to unroll a recursive proof a finite number of times, thereby making it inherently unsuitable for verifying unbounded data structures. While tools like Verus support recursive specifications and proofs through inductive proof functions [58], the verification complexity remains high and often becomes unmanageable for complex, feature-rich data structures typical in the kernel.

6.2 High Proof Effort

The proof-to-code ratio—the number of lines of proof script needed to verify one line of implementation code— is famously high for verified OS kernels built with interactive theorem provers (ITPs) such as seL4 ($\approx$20:1) and CertiKOS ($\approx$15:1). This reflects the immense manual effort required to mathematically prove that every line of low-level C/assembly code correctly implements a precise formal specification. The resulting verification cost (often measured in person-years) is prohibitive for most software projects because it demands rare expertise, consumes many calendar years, and does not scale easily to larger codebases.

Ironclad and NrOS are two systems projects built with SMT. Despite the automation offered by SMT solvers, the verification effort remains high—for complex theorems and properties, SMT solvers often require as many hints as (and sometimes even more than) interactive provers, resulting in high manual effort.

VeriSMo is an outlier. It uses Verus to implement a verified security module for confidential VMs on AMD SEV-SNP [61]. VeriSMo proves functional correctness of the security module as well as the correctness of its information flow. Despite the large codebase, the core of VeriSMo is semantically simple—most of the system state can be described in safe Rust with minimal use of low-level primitives (e.g., pointers).

As a result, the systems and verification communities tend to believe that although offering minimal automation, ITP-based verification frameworks remain the only viable option for building a complex, low-level system software. SMT-based verification frameworks are not considered suitable for this workload, as they require careful optimization and automation management to combat the explosion in the SMT search space, and even with care, they demand as much or even higher proof effort.

Table 6.1 shows the proof-to-code ratio of several verified projects.

6.3 Excessively Long Verification Time

In addition to the verification effort, the effect of verification time is largely overlooked by the verification community. However, for hands-on verification engineers, long verification time is far more detrimental than a high proof-to-code ratio.

Table 6.1. Proof effort for existing verification projects.

Name	Language	Spec Language	Proof-to-Code Ratio
seL4	C + Asm	Isabelle/HOL	20:1
CertiKOS	C + Asm	Coq	14.9:1
SeKVM	C + Asm	Coq	6.9:1
Ironclad	Dafny	Dafny	4.8:1
NrOS	Rust	Verus	10:1
VeriSMo	Rust	Verus	2:1
Atmosphere	Rust	Verus	3.3:1

Long verification time, for a verification project, is analogous to long compile time for developing a non-verified project. Imagine developing a new kernel module for the Linux kernel in C. Each time we finish a chunk of code (e.g., a function), the compiler takes 20 minutes to emit an error message about a syntax error. To make matters worse, unlike a syntax error in unverified code—which is typically visible only in its local scope— an error in verified code can cause proofs of other components in the system to fail. Thus, a more realistic analogy is that each time we recompile the project, the compiler takes 20 minutes to emit an error message about a syntax error caused by our recent change in another module that we have never even heard of! How frustrating would that be!

Table 6.2 shows the verification time of several recent verified projects built with Verus. Even with the help of the Rust borrow checker, verification time remains far from ideal.

Table 6.2. Verification time of different systems on CloudLab c220g5.

Name	1 thread	8 threads	Proof LoC	Code LoC
NrOS page table	1m 52s	51s	5329	400
Mimalloc	8m 12s	1m 40s	13703	3178
VeriSMo	61m 24s	12m 11s	16101	7915
Atmosphere	3m 29s	1m 7s	20098	6048

6.4 Limitations and Challenges in Linearity-Based Verification

6.4.1 Shared and Cyclic Ownership

Rust's linear type system and strict ownership model provide safety guarantees that are excellent for systems software, but they can be overly restrictive for an OS kernel—the low-level middleware that itself manages the raw physical memory of the entire system.

Typically, linear ownership falls short for:

1. Data structures with multiple components that need to reference the same data in complex ways (e.g., graphs with cycles, doubly-linked lists, observer patterns);
2. Data structures where a field points to another field within the same struct;
3. Resource pools and arenas (e.g., a memory allocator).

As introduced in Chapter 2, Verus linear ghost permissions enable shared and cyclic ownership in data structures, but at the expense of offloading the burden of managing lifetimes and proving the correctness of permissions to the proof code. Nevertheless, compared to directly proving memory safety in an unsafe system language (i.e., C), because the Rust borrow checker ensures linearity of these permissions, linear permissions do not require proofs about pointer aliasing and memory overlap, which substantially reduces verification complexity.

Since an OS kernel heavily relies on data structures beyond Rust's linear ownership—for interaction with low-level machine details and performance optimizations—the use of linear permissions is inevitable. Therefore, the sound

and efficient construction of invariants about all linear permissions across the entire kernel is critical for minimizing verification effort and verification time.

6.4.2 Memory Leak Freedom

Contrary to common misconceptions, safe Rust can leak memory. Rust's memory safety guarantees are about preventing undefined behavior (use-after-free, data races, etc.), not about preventing resource leaks.

The Rust Book explicitly states: "Rust's memory safety guarantees do not include preventing memory leaks entirely."

6.5 Inherent Limitations of SMT Solvers on Unbounded Data Structures

SMT solvers lack native support for recursive proofs. Recursion defines a function or property in terms of itself. Proving a recursive property typically requires mathematical induction. Induction is an inherently unbounded proof rule. For example: "If `P(0)` holds, and if `P(k)` implies `P(k+1)` for all `k`, then `P(n)` holds for all `n`." SMT solvers have no native mechanism to apply this universal, unbounded rule. They cannot generate the inductive hypothesis automatically. Programmers, as a result, have to define and prove the inductive hypothesis manually for each recursive proof in the project.

Typically, an OS kernel contains numerous recursive data structures, many of which are indeed unbounded. A page table is a bounded data structure: starting from the root, the page table entries recursively point to the next-level page map (i.e., each PML4 entry contains a pointer to the PML3 page map). The kernel must

then maintain invariants between adjacent page-map levels in order to prove the correctness of the page table. Thus, for a page table with 4 levels, the invariants recurse 3 times. The process tree (e.g., the process tree in the Linux kernel) is a fundamental organizational structure for process lifecycle management, signal propagation, resource accounting, and access control. It is an unbounded data structure—in theory, the process tree can grow to infinite depth, as long as system resources are not exhausted. When applying invariants between adjacent levels, the invariants must propagate from the root to every level, resulting in an unbounded recursive proof.

Writing and proving an inductive hypothesis for each recursive property of all unbounded data structures and applying it to appropriate places require significant verification effort.

6.6 Complexity of Kernel Invariants

The size of kernel invariants is massive, covering both the hardware and software sides: from memory usage of all kernel objects, address space/IOMMU table mapping of each user process, reference counting by user programs of shared resources, to access control and resource ownership in the process hierarchy; from the absence of stale TLB entries to CPU register saving and restoring. Throwing all kernel invariants and state to the SMT solver in an unshielded manner will undoubtedly cause verification timeout and state explosion.

A common solution is modular verification. That is, similar to the linked-list example in previous chapters, encapsulating the internal invariants of a data structure (a module) as closed specifications that are visible only to the member function bodies. When higher levels use low-level data structures to build more complex data structures, only the abstract state of the low-level data structure is revealed, thereby mitigating verification overhead. This approach has its limitations. Sometimes, the proofs of high-level data structures require low-level data structures to satisfy certain properties, which are often embedded in the low-level data structures as internal invariants. Manually revealing invariants from low-level data structures, especially through multiple levels of nesting, requires non-trivial manual effort and a clean organization of invariants. If the data structure involves recursive invariants, revealing nested closed specifications consequently requires introducing inductive hypotheses as mentioned in the previous subsection.

6.7 Background: Related Work

6.7.1 seL4

seL4 [32] is the world's first operating system kernel with a comprehensive, machine-checked formal proof of correctness, security, and functional properties. Partially to combat verification complexity, seL4 adopts the microkernel architecture, which provides a minimal set of abstractions: threads, address spaces (via page tables), inter-process communication (IPC), and capabilities (for

access control). Everything else (device drivers, file systems, and other OS services) runs in unprivileged user space.

seL4 performs verification in three layers; in other words, it constructs refinement proofs in three layers. All proofs are conducted in Isabelle/HOL, an interactive theorem prover. Programmers write specifications of the system and desired invariants, and then use Isabelle to construct machine-checked proofs that the models satisfy the properties.

The topmost layer is the *abstract specification*. It is the highest-level, most abstract model of the kernel's API and behavior. It describes the expected behavior of the system calls with which the user level interacts.

The second layer is the *executable specification* in Haskell. It is a more concrete, executable model of the kernel written in Haskell. It is an algorithmically detailed version of the abstract specification. In the refinement proof, it is proven that this Haskell model correctly refines (implements) the abstract specification. Every behavior of the Haskell model is a valid behavior of the abstract specification.

The third layer is the *implementation* (C code). The C code is automatically translated into a formal model, and it is proven that this formal model refines the Haskell executable specification. This step bridges the gap between a high-level language and the formal models.

Thus, through these three layers and two levels of refinement proofs, seL4 proves that the low-level C implementation is a refinement of the high-level abstract specification. In other words, the implementation correctly implements the specification.

seL4 manages resource ownership with its unique capability system. Each process owns a set of capabilities for different kernel resources. Each capability can be shared with another process or derived into different capabilities (if the capability is untyped). The kernel maintains a capability derivation tree to track sharing, similar to the process tree. Each shared or derived capability can be revoked by its parent capability, which implements seL4's resource revocation scheme. The capability tree is, of course, an unbounded data structure, which is a strong argument for why an OS kernel with the complexity of seL4 is difficult to verify using SMT-based verification frameworks.

6.7.2 Hyperkernel

Hyperkernel is a formally verified research OS kernel designed to achieve "push-button" verification—fully automated proofs with minimal manual effort [45]. Its primary goal was to radically reduce the proof burden compared to pioneering projects like seL4 and CertiKOS. Unlike conventional verification which must reason about infinite state spaces (e.g., any number of processes, unbounded loops), Hyperkernel intentionally limits the kernel's interface to make it finite. This transforms a difficult undecidable verification problem into a decidable one that computers can solve automatically.

Unlike seL4 and CertiKOS that develop the proof in interactive theorem provers (Isabelle and Coq), Hyperkernel relies on SMT (Satisfiability Modulo Theories) solving. First, LLVM lowers the source code in C to LLVM IR. Then, a custom Irpy translator (irpy) translates the LLVM IR into Python code that

preserves the exact semantics but is now suitable for symbolic execution . Finally, the Python code is executed symbolically against a high-level state-machine specification. The Z3 theorem prover automatically checks if the implementation refines the spec for all possible inputs and states. Hyperkernel demonstrated a high degree of automation through the use of SMT solvers but at the cost of severe limitations in kernel functionality. Since Hyperkernel operates on LLVM IR level and does not rely on linear ownership model to mitigate verification complexity, despite the automation offered by Z3, verification time is roughly 30 minutes on quad-core i7-7700K

6.7.3 NrOS

NrOS [4] is a research operating system kernel that tackles a major challenge in modern computing: achieving high performance on many-core systems without sacrificing development simplicity and correctness. NrOS is constructed primarily with sequential logic and sequential data structures, and it scales across cores and nodes using node replication ("NR") [9], a log-based shared-memory synchronization mechanism inspired by state machine replication in distributed systems. This approach simplifies formal verification, especially when reasoning about concurrency, as verification can focus on the sequential logic and sequential data structures. NrOS then verifies the node replication algorithm using IronSync [23], which was later ported to Verus.

The verified NrOS kernel is an ongoing open-source project [7] built with Verus. As of now, verification of the NrOS page table has been completed and

released. The NrOS page table consists of multiple layers of refinement proof similar to seL4. It has three layers—L2 (the implementation layer), L1 (the specification layer), and the topmost OS specification layer. Most of the page table logic and interactions with the MMU are verified in the L1 specification layer. In this layer, NrOS models each level of the page table as identical page map structures placed at different levels and manages the ownership of page maps with idiomatic Rust ownership model— that is, the ownership of each page map is stored alongside the upper-level page table entry that points to it. As a result, the NrOS page table uses recursive specifications to propagate invariants from the root to each level. Although this approach is elegant and produces highly readable code, the verification complexity is high. The recursive proofs require multiple induction lemmas, and the verification time is not ideal.

CHAPTER 7

ATMOSPHERE

7.1 Introduction

In this work, we argue that automated verifiers can be used for development and verification of low-level systems. Specifically, we demonstrate the possibility of using Verus [33], an SMT-based verifier for Rust, for developing a fully-verified microkernel, Atmosphere.

Atmosphere is similar to classical microkernels: it runs on multi-CPU hardware (under a big lock), supports processes and threads, dynamic memory management, inter-process communication (IPC), virtual address spaces, IOMMU, and an abstraction of containers, groups of isolated processes. Atmosphere is designed as a separation kernel that supports strict isolation in mixed criticality systems. We develop all code in Rust and prove its functional correctness, i.e., refinement of a high-level specification, with Verus. Moreover, our speed and development effort approach the commodity unverified development–while more time was spent on the design and implementation of formal specifications and proofs, a lot less effort was required for testing.

7.2 Key Design Decisions

Pointer-centric design: Linear languages do not support non-linear data structures well, e.g., Rust relies on trusted unsafe extensions to support cyclic references. This result in a range of suboptimal design choices: one can try to represent kernel state without non-linear data structures [36], or just trust unsafe smart pointer types, or implement their verified versions and then establish high-level correctness of each data structure along with leak freedom.

In Atmosphere, we make the design choice to embrace the use of raw, nonlinear pointers, arguably, deviating from canonical Rust idioms and instead use the language in a manner similar to unsafe C. In other words, we develop kernel data structures using raw pointers, but rely on Verus to establish the correctness of all pointer operations via the proof. This allows us to design performance-efficient data structures with all optimizations typical for unsafe code, e.g., use internal storage to represent pointers to tree nodes, similar to Linux, use reverse pointers to support efficient, constant-time removal from linked lists, etc.

Intuitively, such a design choice is prohibitive due to the pervasive use of raw pointers and the complexity of the proofs required to establish the correctness of pointer operations. Surprisingly, however, a combination of linear permission pointers [34] (an abstraction offered by Verus for reasoning about pointer operations) and the flat approach to permission storage–allows us to avoid complexity or recursive proofs.

Flat approach to permission storage: To keep the complexity of the proof under control, we adapt a "flat" permission design, in which permission pointers to inner objects in the hierarchy, e.g., all nodes of the page table, all threads in the system, all nodes of a tree, etc., are stored at the topmost level of the subsystem with the global view of a specific subsystem in a flat map. We build on the ideas of prior systems that advocated for flat permission storage [7], [34], [57], [61]but push it to the extreme arguing that it is a key design choice critical for the scalability of the proof. The global view of the data structure gives us the freedom to encode the properties of unbounded, recursive data structures, in a simple non-recursive manner. Moreover, it allows us to separate reasoning about the structural (e.g., the tree has no cycles, reverse pointers are correct) and non-structural (e.g., a new thread is added to a process) arguments.

Manual memory management: Rust ensures memory safety and automatic memory management via its borrow checker and a collection of trusted pointer types. Unfortunately, automatic memory management creates a semantic gap between the formal specification of the system and the state of its memory. I.e., reasoning about global, kernel-wide properties requires explicit knowledge of all memory in the system. For example, the non-interference theorem requires that one domain cannot exhaust the memory of the system. Without explicit knowledge of the state of all memory in the system, specifications of the memory-related system calls become nondeterministic. Additionally, Rust type system does not protect against memory leaks, which are possible in the kernel due to complex cyclic dependencies and intricate object lifetimes [49]. In

Atmosphere, we abandon automated memory management of Rust, i.e., we do not rely on the Rust borrow checker for reasoning about lifetimes of kernel objects and instead allocate and deallocate them explicitly (again, similar to how it is done in an unsafe language like C). We, however, establish the safety of all memory operations as well as leak freedom via a proof. Note, that Rust ownership model is still critical for establishing correctness of the code as it is used by Verus to reason about linearity of all tracked proof variables (e.g., linear permissioned pointers) as well as the heap and aliasing. This design choice allows us to support a combination of complex manual and reference counted lifetimes in the kernel without trusting any of the Rust pointer types.

7.3 System Architecture

Atmosphere is a full-featured microkernel. The microkernel supports a minimal set of mechanisms to implement address spaces, threads of execution, memory management, interrupt dispatch, inter-process communication, IOMMUs, and containers – groups of user processes with guaranteed memory and CPU reservations that are used to enforce isolation between mixed-criticality systems. Previous works have shown the value of OS-level support for isolation [11], [38], [39]. In Atmosphere, system device drivers, network stacks, file systems, etc. run as user-space processes. Atmosphere supports multiprocessor machines, but to simplify verification, we rely on a big-lock synchronization, i.e., all interrupts and system calls execute in the microkernel under one global lock and with further interrupts disabled.

Processes can communicate via endpoints. A sender thread can pass scalar data, references to memory pages, IOMMU identifiers, and references to other endpoints. Endpoints allow processes to establish regions of shared memory, which then can be used for efficient asynchronous communication [6], [8], [20].

7.3.1 Isolation and Non-interference

To support provable separation between groups of processes, Atmosphere implements an abstraction of a container. A container is a group of processes that have a guaranteed reservation of memory (*container quota*) and CPU cores, i.e., when a container is created, the parent container passes a subset of its own reservation of memory and cores to a child. The microkernel implements careful accounting of all memory allocations from any process in the container, i.e., memory allocated for the user process or allocated by the kernel to keep metadata about threads, endpoints, etc. The system call specifications are designed to support user-level proofs of functional correctness and non-interference between containers. The non-interference theorem guarantees that any arbitrary system call from any of the isolated containers cannot change the observable state of another.

7.3.2 Access Control and Revocation

In Atmosphere, containers form a tree to maintain parent-child relationships. Parents have the capability to terminate their direct and indirect children and harvest their resources. Inside each container, the processes form a separate

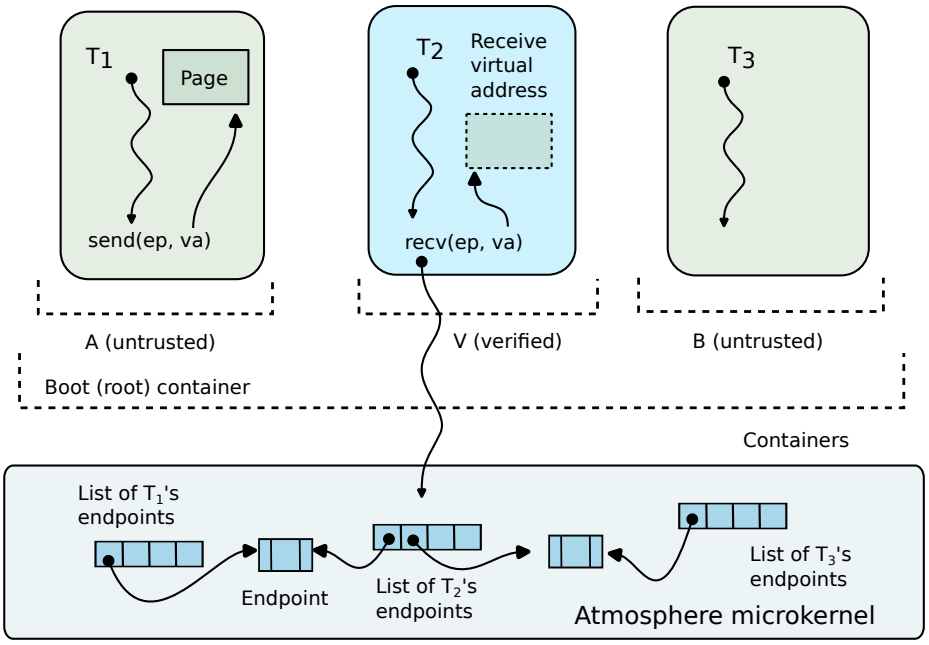


Figure 7.1. Architecture of Atmosphere.

process tree, which allows parent-child tracking of all processes in the same container. Similarly, the parent process has the capability to terminate child processes. Note that container and process hierarchies are *unbounded* – processes and nested containers can be created for as long as memory is available.

In contrast to classical capability microkernels [32], in Atmosphere we make a design choice to prohibit fine-grained revocation of resources. Resources return to the parent container when the child container is terminated (i.e., the only way to revoke resources of a container is to terminate it). This design choice simplifies verification of the user code – we have a guarantee that memory cannot be revoked while the user code that we verify is executed. Note, that it's possible to pass resources outside of the container via an endpoint and then they will not be revoked. Hence, the system has to be carefully constructed to control the flow of

resources across container boundaries and may involve trusted or verified proxy containers to enforce information and resource flow policies.

This is, however, intentional. Atmosphere is designed to support verification of user-level code that can be used for enforcing such policies. In our running example, which we discuss in section 7.4.3, we demonstrate a simple scenario of a system configuration in which multiple isolated containers can establish controlled communication with a shared container. Specifically, we show how two mistrusting containers A and B that are otherwise completely isolated by the microkernel can establish communication channel with a third container V (figure 7.1). We demonstrate how we can proof functional correctness of V and non-interference between A and B. Functional correctness of V guarantees that even without explicit revocation, resources are correctly released as V correctly implements the resource release protocol. Specifically, in our example, containers establish efficient shared-memory communication channels with V. Since we prove functional correctness of V, we have a guarantee that (1) while V exchanges shared memory with A and B it does not leak memory between them, and (2) V always releases all memory received from either A or B even if the container on the other end of the communication channel crashes.

7.4 Verification

To gain confidence that Atmosphere is operating correctly, we prove two theorems: *refinement* and *well-formedness*. The refinement theorem establishes high-level functional correctness, i.e., implementation of the microkernel is a

refinement of its high-level abstract specification. The well-formedness theorem proves that high-level correctness invariants about the system state hold after each state transition.

We model the microkernel as a state machine operating on the abstract state. We rely on a collection of high-level specifications that describe how the abstract state of the kernel is updated on each kernel invocation, and a hierarchy of invariants that describe the correctness of each kernel data structure, as well as express cross-cutting properties like safety and isolation.

7.4.1 Flat, Pointer-Centric Design

In Atmosphere, we make a design choice to embrace the use of raw, non-linear pointers. We develop recursive pointer data structures like linked lists, container and process trees, and in general, freely use pointers to implement complex logical dependencies between kernel data structures. Unrestricted use of pointers offers the freedom of unsafe development, i.e., use coding paradigms and performance optimizations typical for unsafe, unverified code, e.g., reverse pointers from children to parents for efficient lookup, internal list and tree storage, etc. Not surprisingly, establishing the correctness of pointer operations is challenging. Reasoning about recursive data structures is inherently difficult for SMT solvers as they provide only limited support for inductive proofs and can only unroll recursive proofs a finite number of times, making them unsuitable for unbounded verification.

7.4.1.1 Flat Organization of Pointer permissions

We make a key architectural choice: "flat" organization of pointer permissions for complex recursive data structures. Specifically, in all recursive pointer data structures, we use raw pointers like one would do in unsafe C. Yet permissions to access these pointers are stored as a single flat map at the topmost level of the subsystem, which provides the global view of the entire subsystem. For example, the process manager, a subsystem responsible for managing processes, IPC, and scheduling, holds the permissions to all threads, processes, containers, endpoints, etc., as a collection of flat maps (listing 7.4.1.1, lines 3–6):

```
1  pub struct ProcessManager {
2    pub root_container: ContainerPtr,
3    pub cntr_perms: Tracked<Map<CtnrPtr, PointsTo<Container>>>,
4    pub proc_perms: Tracked<Map<ProcPtr, PointsTo<Process>>>,
5    pub thrd_perms: Tracked<Map<ThrdPtr, PointsTo<Thread>>>,
6    pub edpt_perms: Tracked<Map<EdptPtr, PointsTo<Endpoint>>>,
7     ...}
8  pub struct Container {
9    parent: Option<CtnrPtr>,        // root has no parent
10   children:  StaticList <CtnrPtr>, // direct children
11   depth: usize,
12   path: Ghost<Seq<CtnrPtr>>,    // direct and indirect parents
13   subtree: Ghost<Set<CtnrPtr>>, // all reachable children
14    ...  }
```

Flat ownership provides us with a global view of the data structure and specifically allows us to:

1. convert recursive specifications into non-recursive ones;
2. decouple the proofs about the structure of the data structure (e.g., a tree is well-formed) from other non-structural proofs (e.g., object lifetimes);

3. flexible traversal of the data structure (e.g., up and down traversal is required when the tree is updated, i.e., we need to access both the parent and all children of the updated node).

7.4.1.2 Global Invariants

Availability of the global system state via the permission pointer map allows us to formulate global invariants without navigating through the object hierarchies, i.e., peering into the threads of a process within a container. For example, we can directly access the state of all the threads in the system and establish invariants about them:

```
1  // all threads in the system are well formed
2    pub spec threads_wf(pm: &ProcessManager) -> bool {
3       forall |t_ptr:ThrdPtr|
4         pm.thrd_perms@.dom().contains(t_ptr)
5       ==>
6       pm.thrd_perms@[t_ptr].value().wf()
7    }
```

Here, we iterate over the `thrd_perms` map that stores the tracked permissions of all threads in the system (listing 7.4.1.1) to establish that they all are well-formed (`wf()`).

7.4.1.3 Non-recursive Invariants

To illustrate how we use flat ownership to avoid inductive proofs about unbounded data structures, consider an example of a container tree – an unbounded data structure that can grow for as long as the system has memory (listing 7.4.1.1, lines 8–14). In Atmosphere, containers form a single recursive tree

(internally, each container has a separate tree of processes). We implement the tree in a manner similar to unsafe C: each node has a list of children and a pointer to its parent (listing 7.4.1.1, lines 9–10). However, all permissions to access the nodes of the container tree are stored at the level of the process manager that holds the non-ghost root of the container tree.

To support various aspects of the proof, we expose the parent-child relationships between nodes as two ghost variables: `path` and `subtree`. The node sequence `path` describes the path from the root to the current node (listing 7.4.1.1, line 12), and the `subtree` holds all children of the node.

The path along with the set of reachable children is critical for constructing user-level proofs about the properties of subtrees (e.g., in the isolation proof, we prove that all the address spaces in the subtree of one isolated container are disjoint from the address spaces in the subtree of another).

Without the flat storage, maintaining the `path` invariant requires recursive definitions about the parents and their direct children – only local information about immediate children is available in a non-flat, hierarchical case. For example, one first encodes that the child's `path` equals the parent's `path` plus the parent:

```
1  pub spec fn child_resolve_path_wf(&self:Container) -> bool{
2   forall |child:Container|
3     self.children@.contains(child)  ==>
4       self.get_child(child).path =~= self.path.push(self)}
```

This recursive definition can be used to reconstruct the full path from the root to any node in the tree. Verus supports both automatic unrolling of the definition and constructing a manual inductive proof. However, SMT solvers can only

unroll a recursive proof a finite number of times (due to their bounded recursion depth) and lack native support for inductive reasoning. Although the unbounded recursions can be proven in Verus with the help of inductive proof functions, the verification difficulty and manual effort are high.

With the global view of all children available in the flat case, we can instead describe the `path` invariant in a non-recursive way:

```
1  pub spec fn resolve_path_wf(&self:ProcessManager) -> bool {
2    for |c:container, d:int|
3      0 <= i < self.cntr_perms@[c].value().path.len() ==>
4      self.cntr_perms@[c].value().path.subrange(0, d)
5        =~= self.cntr_perms@[self.cntr_perms@[c].value().path[d]].value().
             path
6  }
```

I.e., for any node `n` at depth `d` on the path of container `c`, `c`'s subpath from the root (node at depth 0) to the node at `d` is equal to the path of `n`. Instead of relying on recursive reasoning about pairs of parents and children, we directly express the correctness of the `path` invariant for all nodes of the tree. Global permission map `cntr_perms` allows us to reason about any node in the tree.

7.4.1.4 Modularity and Isolation of Structural and Non-structural Proofs

Pointer-centric design of the kernel data structures requires us to maintain structural invariants that control the correctness of each data structure, e.g., the tree has no cycles besides the child-to-parent reference in each node. Maintaining such invariants is challenging as they have to be validated on any update even if it is not structural, e.g., `mmap()` changes container's quota, but does not change the structure of the tree. We leverage flat permission storage along with `closed`

specification functions to isolate structural invariants and proofs from non-structural arguments (e.g., each alive container has at least one alive process).

We define structural invariants as separate `closed` specification functions (the body of a closed function is only visible in the defining module, hence incurring no verification overhead if used in a different module).

For example, executable function `new_container()` creates a child container C_C for the given parent container C_P and transitions the state of the process manager from Φ (old state) to Φ' (new state). We separate structural and non-structural proofs into two functions. First, we use a closed spec function `container_tree_wf()` to define all the structural invariants of the container tree in a separate container tree module. Second, an open spec function `new_container_ensures()` defines how each container's state changes in `new_container()`.

The `new_container_ensures()` function only describes the changes in the kernel state, but does not check any structural invariants. Hence, it has very little verification overhead. In our example, a new container C_C is added to the container tree. We extend the set of reachable children of C_C's direct and indirect parent (`.subtree`) by C_C. In the preconditions, `new_container()` requires the container tree to be well-formed on entry and establishes that it satisfies `new_container_ensures()` (line listing 9) on exit. The proof function `new_container_preserve_tree_wf()` in the container tree module proves that if the transition from the old state to the new state satisfies the specification

`new_container_ensures()`, the new state also satisfies `container_tree_wf()`. In `new_container_preserve_tree_wf()`, we reveal the function body of `container_tree_wf()` and use a range of proof code to help Verus finish the proof. Therefore, in the `new_container()` function we can establish `container_tree_wf()` (line listing 10) without requiring the details of complex structural invariants of the container tree. Such an approach to the proof structure significantly reduces the size of the SMT search space and allows us to lower both proof complexity and verification time.

```
1  pub fn new_container(Φ:&mut PM, C_P:CntrPtr, C_C:CntrPtr,...)
2  requires
3        container_tree_wf(Φ), ...
4  ensures
5        container_tree_wf(Φ'), ...
6  {    ...
7     // at the end of the function
8     assert(container_tree_wf(Φ')) by {
9        assert(new_container_ensures(Φ, Φ', C_P, C_C));
10       new_container_preserve_tree_wf(Φ, Φ', C_P, C_C);
11 }}
12 pub open spec fn new_container_ensures(Φ:&PM, Φ':&PM,
13       C_P:CntrPtr, C_C:CntrPtr) -> bool{
14 &&&
15 // new container's direct and indirect parents' subtrees
16  forall |c_ptr:CntrPtr|
17    Φ.cntr_perms@.dom().contains(c_ptr) &&
18       (c_ptr == C_P || !Φ.cntr_perms@[C_P].value().resolve_path.contains(c_ptr
              )) ==>
19       Φ.cntr_perms@[c_ptr].value().subtree@.insert(C_C) =~=
20          Φ'.cntr_perms@[c_ptr].value().subtree@
21       // other fields
22  ...  // other containers are not changed
23 }
24 pub proof fn new_container_preserve_tree_wf(Φ:&PM, Φ':&PM,
25       C_P:CntrPtr, C_C:CntrPtr)
26 require
27       container_tree_wf(Φ),
28       new_container_ensures(Φ, Φ', C_P, C_C),
29 ensures
```

```
30          container_tree_wf(Φ'),
31      {...}
```

Listing 7.1. Isolation of structural and non-structural proofs.

7.4.2 Manual Memory Management

In Atmosphere we implement manual memory management. This design choice allows us to support reasoning about complex cross-cutting properties like isolation and non-interference (this requires knowledge of all memory in the system) as well as support nonlinear lifetimes for a variety of kernel data structures.

7.4.2.1 Memory Allocation

In Atmosphere dynamic memory allocation for kernel objects, e.g., containers, processes, threads, endpoints, is done at the granularity of 4KB, 2MB, or 1GB memory pages. While it seems wasteful, in practice, the system does not allocate a large number of objects. The simplicity of the allocator, however, allows us to reason about non-interference between containers as well as to simplify specifications that expose the internal state of the allocator required for maintaining safety and leak freedom invariants. We support allocation of 2MB and 1GB superpages to support construction of large address spaces with low TLB pressure.

The page allocator uses a page array (similar to the page array in Linux) to maintain the metadata for each physical page in the system. We ensure that each physical page is in one of the following states:

1. *free* (on the list of free pages in the memory allocator)
2. *mapped*, (mapped by one or more processes)
3. *merged*, (merged to form a 2MB or 1GB superpage)
4. *allocated* (allocated for one of the kernel data structures, e.g., a process).

The allocator uses three doubly-linked lists to store the free pages of different sizes (4KB, 2MB, 1GB). We use the page metadata array to efficiently merge 4KB pages into superpages, e.g., to form a 2MB superpage out of free 4KB pages we scan the page array and remove merged 4KB pages from the list of free 4KB pages. Each page metadata in the array maintains a pointer to the node of the linked list holding the page, which allows us to perform constant-time removal when the page is merged.

7.4.2.2 Safety

We establish safety and the correctness of all pointer operations. We define memory safety and leak freedom as a combination of:

1. type safety (each allocated region of memory is used by exactly one data structure of a correct type)
2. spatial safety (we check that data types are within allocated memory, and all memory accesses to variable-sized data structures like arrays are in-bounds)
3. temporal safety (all pointers point to live objects, i.e., memory is not deallocated or reallocated for another object).

To prove safety, we need to prove that all allocated memory is used by disjoint objects. To prove leak freedom, we need to prove that the sum of all

memory used by all objects in the system equals all allocated memory (i.e., a combination of pages in the "allocated", "mapped", and "merged" states). The most challenging part of the proof is establishing safety and leak freedom for memory allocated for kernel objects. For this, we need to establish: (1) all the objects in the kernel are pairwise disjoint in memory and (2) the sum of all memory used by all objects in the system is equal to the collection of "allocated" pages.

A straightforward approach to such proof is to establish that all the objects in the kernel are pairwise disjoint, and then prove that the sum of the memory used for storing these objects is equal to the collection of "allocated" pages. Such an approach is challenging as it requires establishing explicit invariants for *all* objects (i.e., sizes and addresses) in the kernel, making the SMT search space to grow exponentially. Instead, we leverage the fact that objects of each specific type are used in the kernel in a hierarchical manner, i.e., page tables are used in the memory subsystem; threads, processes, and containers are used by the process management subsystem, etc.

In Atmosphere, for each data structure in the kernel, we implement the `page_closure()` specification function, which returns a set of pages used by the data structure and all objects owned by it. Here, the ownership means either a direct ownership of Rust (e.g., an array of objects) or ownership via a tracked permission. For example, a page table does not own any other objects, besides the physical pages used to construct the page table.

7.4.2.3 Bottom-up Recursive Memory Reasoning

To reason about the system's memory usage, we hierarchically maintain the `page_closure()` for each kernel object, prove pairwise disjointness locally, and merge the `page_closure()` of all subsystems. For example, the virtual memory management subsystem owns the memory of all page tables and IOMMU page tables. The subsystem maintains a set of invariants to ensure that each page table and IOMMU table's `page_closure()` are pairwise disjoint, and their union is equal to the `page_closure()` of the virtual memory management subsystem. With this, we can easily infer the desired memory properties (i.e., all the objects in the kernel are pairwise disjoint) without explicitly proving them as global invariants.

7.4.2.4 Explicit Memory Allocator State

Establishing leak freedom and cross-cutting properties of the memory subsystem requires visibility of the state of the memory allocator. Specifically, explicit invocation of the allocation and deallocation APIs enable exact reasoning about the allocator's state through the API's pre- and post-conditions. We expose the internal state of the allocator as sets of free, allocated, merged, and mapped pages. Each time we allocate memory (i.e., a page), we need to establish that:

1. the page closure of the allocating subsystem is extended by the new freshly allocated page
2. the set of total "allocated" pages is extended by this page
3. the page was previously "free", i.e., not used by any other subsystem

4. other subsystems in the kernel are not changed.

```
1  pub fn alloc_page_4k(alloc:&mut Allocator)
2    ensures
3      alloc.free_pages_4k() =~=
4        old(alloc).free_pages_4k().remove(ret.0),
5      alloc.allocated_pages_4k() =~=
6        old(alloc).allocated_pages_4k().insert(ret.0),
7      ...
8
9  pub fn new_endpoint(pm:&mut ProccessManager)
10   ensures
11     pm.page_closure() =~=
12       pm(self).page_closure().insert(page_ptr),
13
14 // allocate new endpoint
15 let (p_ptr,mut p_perm) = self.page_alloc.alloc_page_4k();
16 self.proc_man.new_endpoint(p_ptr, p_perm, ...);
```

Listing 7.2. Establishing safety of allocation.

For example, allocation of a new endpoint in the `syscall_new_endpoint()` function uses the postcondition of the `alloc_page_4k()` (allocates a page and a permission to use it), and `new_endpoint()` (consumes the page and adds the new endpoint to the calling thread) functions (lines 15-16, listing 7.2). First, we establish that a newly allocated page was previously not allocated via the postconditions of `alloc_page_4k()`. This ensures that using this page to form a new endpoint object in the kernel is safe. Furthermore, the postconditions of the two functions ensure that the set of allocated pages grows by exactly one page, and process manager's `page_closure()` grows exactly by this one page as well. Since other subsystems do not change, we can prove that the memory safety and leak freedom invariants still hold.

7.4.2.5 Consistency of Page Table Updates

Even though Atmosphere operates under a big lock, threads of the same process scheduled on different cores might access the page table while it is updated by one of the threads. Floyd-Hoare reasoning only guarantees the correctness of the final state, not that intermediate states during execution are well-formed or consistent. We model each page table update (i.e., each write to both leaf and intermediate levels) as a separate step. For any step that does not modify the last-level page table entry, we prove that the address space (abstract mapping) of the page table does not change. For any step that modifies the last-level page table entry, exactly one entry in the page table abstract mapping is changed (adds a new entry or removes an existing entry).

7.4.3 Noninterference

Atmosphere is a separation kernel designed to support isolation and noninterference between containers. In most real-world scenarios, complete isolation is impractical – isolated subsystems need to communicate, but in a controlled manner (e.g., access multiplexed device drivers, etc.). Careful design of the system call interface specifications allows us to use Atmosphere for deployment of mixed-criticality systems in which users can define their custom notions of isolation and non-interference, i.e., support isolation across mission-critical domains and yet allow them to communicate in a controlled manner.

Specifically, we develop a proof of isolation and non-interference for a system that executes three containers: two untrusted, unverified, and isolated containers A and B and a verified, shared container V (we discuss how such proof can be generalized for different system configurations). A and B do not interfere with each other but are allowed to establish communication with V (Figure 7.1). Naturally, since both A and B can communicate with V, their isolation and noninterference relies on V's functional correctness, which we prove. However, we make no assumptions about A and B. They are not trusted, not verified, and thus can perform arbitrary system calls with arbitrary system call arguments.

We implement V as a container with one process that runs one thread of execution. This simplifies verification, but is arguably practical for simple containers that implement system security policies. I/O intensive shared device drivers will require proof about concurrent code, which we leave as future work (note that such proofs are feasible due to limited concurrency in modern device drivers, which separate control and data planes and avoid sharing and synchronization across threads in the data plane).

We implement V as an event-driven state machine: it executes a loop that checks for incoming IPC messages from A and B, and reacts to the actions from A and B according to its abstract specifications. V may receive pages and endpoints from A and B, but never shares them across container boundaries.

7.4.3.1 "Flat" System Call Specifications

The proof of non-interference depends on the following state of the system:

- The state of the entire kernel (Ψ).
- All subcontainers C_A, C_B, C_V that are recursively created from A, B, and V.
- All processes P_A, P_B, P_V from all containers in C_A, C_B, C_V respectively.
- All threads T_A, T_B, T_V from all containers in C_A, C_B, C_V respectively.

We leverage flat permission storage to provide direct access to the state of the kernel objects, i.e., state of all containers, threads, processes, etc. For example, to construct T_A with recursive ownership, one would need to rely on two unbounded recursive specs to first walk the container subtree to form C_A (by recursively merging the children containers level by level), second walk the process tree of each container to form P_A, and finally create a union of the threads of P_A to form T_A. With flat storage, we can maintain the sets of all reachable child containers in `.subtree` and simply construct T_A with the following bidirectional invariant to ensure T_A contains and only contains all the threads in C_A:

```
1  spec fn T_A_wf(Ψ:Kernel,A:CntrPtr,T_A:Set<ThrdPtr>) -> bool{
2    &&& forall|c_ptr: CntrPtr, t_ptr:ThrdPtr|
3      (c_ptr == A || Ψ.get_cntr(A).subtree@.contains(c_ptr))
4      && Ψ.get_cntr(c_ptr).owned_thrds@.contains(t_ptr)
5      ==> T_A.contains(t_ptr)
6    &&& forall|t_ptr:ThrdPtr| T_A.contains(t_ptr) ==>
7      (Ψ.get_thrd(t_ptr).owning_cntr == A || Ψ.get_cntr(A)
8        .subtree@.contains(Ψ.get_thrd(t_ptr).owning_cntr))}
```

7.4.3.2 Isolation

We define and prove isolation between P_A and P_B as two main invariants: memory and endpoint isolation.

```
1  pub open spec fn memory_iso(Ψ: Kernel, P_A:Set<ProcPtr>, P_B:Set<ProcPtr>)
       -> bool{
```

```
    forall |a_p_ptr: ProcPtr, a_va:VAddr,
        b_p_ptr: ProcPtr, b_va:VAddr|
      P_A.contains(a_p_ptr) && P_B.contains(b_p_ptr) &&
      Ψ.get_address_space(a_p_ptr).dom().contains(a_va) &&
      Ψ.get_address_space(b_p_ptr).dom().contains(b_va)
      ==>
      Ψ.get_address_space(a_p_ptr)[a_va].addr !=
        Ψ.get_address_space(b_p_ptr)[b_va].addr}

pub open spec fn endpoint_iso(Ψ: Kernel, T_A:Set<ThrdPtr>, T_B:Set<ThrdPtr
    >) -> bool{
    forall |a_t_ptr:ThrdPtr, a_e_idx: EdptIdx,
        b_t_ptr:ThrdPtr, b_e_idx: EdptIdx|
      T_A.contains(a_t_ptr) && T_B.contains(b_t_ptr)
      ==>
      Ψ.get_thrd_edpt_descriptors(a_t_ptr)[a_e_idx] !=
        Ψ.get_thrd_edpt_descriptors(b_t_ptr)[b_e_idx]}
```

That is, the pages mapped by the address spaces of P_A are proven not to be mapped by the address spaces of P_B. Additionally, to allow fast communication, shared endpoints can be established between T_V and T_A or T_V and T_B, but not between T_A and T_B:

To prove that A and B cannot break the memory and communication channel isolation, we show that after a random system call with random system call arguments from any thread in T_A and T_B, the memory isolation and endpoint isolation are still satisfied:

```
forall |Ψ: Kernel, Ψ': Kernel, t:ThrdPtr, a:SysCallArg|
  Step(Ψ, Ψ', t, a) && (T_A.contains(t) || T_B.contains(t)) &&
    memory_iso(Ψ, P_A, P_B) && endpoint_iso(Ψ, T_A, T_B)
  ==> memory_iso(Ψ', P_A, P_B) && endpoint_iso(Ψ', T_A, T_B)
```

7.4.3.3 Noninterference

At a high level, the non-interference theorem establishes that isolated containers cannot observe actions of each other. Similar to prior systems [19], [22], we leverage the unwinding theorem [21], [53] stating that it's sufficient to establish unwinding conditions to prove the global noninterference property. Specifically, we leverage unwinding conditions as formulated by Luke et al. [44]:

Output consistency (OC): Output consistency requires the kernel to provide the same system return value given two identical system states. In Atmosphere, the return value of system calls is deterministic and only depends on the state of the old kernel state and arguments. According to all system call postconditions, same kernel pre-states always result in same kernel post-states and return values. Thus, OC is trivial to prove.

Step consistency (SC): SC requires that the system state remains equivalent for one domain before and after any action is performed by another domain. In our model, the observable state of a container subtree C_B includes its memory quotas, address spaces, schedulers, endpoints, state of the processes, etc. We show that the observable state of B and the system call return value of each arbitrary system call from container B remains unaffected before and after an arbitrary system call from container A.

Local respect (LR): Local respect requires that an arbitrary system call executed by one container cannot affect the observable state of another distrusting

container. In our setup, since only A and B are isolated, LR is proven when we prove SC.

7.4.3.4 Discussion and Limitations

We illustrate how noninterference and isolation can be established for a system configured with three container hierarchies: A, B and V. To make a similar poof but for a different system configuration, one has to define the desired isolation and noninterference invariants. We argue that such proofs will be similar to the one we discuss here. The key observation is that this and similar noninterference proofs leverage the flat state of the kernel, which allows accessing it in a non-recursive manner. In the case, when any number of isolated containers do not communicate, the proof is a strict subset of the proof presented here.

In the non-interference proof, we only reason about the effects of system calls, but not the user code. Despite being isolated on different CPUs, user processes can introduce timing channels in shared caches and other hardware resources. Also, at the moment, Atmosphere allows execution of several long-running system calls (e.g., operations like mapping and sending large regions of memory, or terminating containers and processes), which can block other processes due to a big lock in the kernel for a long period of time and leak timing information. This limitation can be removed in the future by bounding the memory region size and by implementing iterative versions of the "kill" system calls similar to seL4.

7.5 Trusted Computing Base

Formal verification enables us to establish the correctness of the system and eliminate a range of errors that are typical in low-level code. Yet, it is important to be explicit about all software and hardware layers that form the trusted computing base (TCB) of the system, and hence can still contain flaws and vulnerabilities:

1. The Verus frontend responsible for translating Rust and Verus code into SMT verification conditions.
2. The underlying SMT solver (Z3). Verus uses Z3, and any unsoundness in it will compromise the soundness of verification.
3. The Rust compiler. We trust the compiler to correctly perform borrow checking, type checking, and code generation. We also assume that the toolchain provides correct implementations of core intrinsics. Other standard build tools (assembler, linker, etc.) are also used to create the final microkernel binary.
4. The Rust core library (`core`). The Rust standard library contains three parts: `core` implements core routines for primitive types (e.g., `u64::pow()`) and standard abstractions (e.g., `Option::is_some()`), while `alloc` and `std` build on top of it to provide heap-allocated data structures and platform-specific interfaces. Atmosphere only makes use of a minimal subset of `core` . We do not rely on `alloc` and `std` , and our code does not use many common types like vectors, strings, `Rc` and `Arc` , hash maps, mutexes, etc.

5. Specifications of core Rust data structures and routines. These specifications model the expected behavior of core data structures and routines as implemented in compiler intrinsics and `core`. For example, Atmosphere trusts that a Rust slice behaves like a `Seq` with a fixed length.
6. Axioms missing in Verus. For example, if we remove an element from a unique sequence, the result sequence is still unique (around 700 lines of spec). These axioms can be verified in the future.
7. Setter functions for mutating tracked ghost permissions and Rust core primitive data structures. Verus currently has very limited support for `&mut`. This adds 300 lines of executable code with 1900 lines of specifications.
8. Sequences of assembly and trusted Rust code. The fragments of assembly code (172 lines) implement entry and exit trampolines for system call and interrupt handlers. Trusted Rust code (total of around 3,000 lines) implementing hardware interface to configure IOMMU (465 lines), setup execution environment of the kernel (321 lines), initialize interrupt handling (293 lines), advanced programmable interrupt controller (287 lines), interrupt descriptor table (179 lines), initialize task state segment (TSS) and global descriptor table (172 lines), fast system call entry via sysenter, handle kernel command line (123 lines), initialize per-CPU data structures and application processes, etc.
9. The Atmosphere boot loader. We implement a minimal, trusted boot loader that sets up the initial runtime environment for the verified kernel (e.g.,

enumerates available physical memory, sets up stacks, initializes interrupt controllers, etc.).

10. The trampolines for system calls,
11. Finally, we trust the CPU (i.e., the silicon implementation, CPU microcode and all other firmware executing on hidden CPU cores like power management microcontroller), DRAM, and the main board including the system management mode (SMM) and Intel Management Engine (ME) that have access to physical memory of the machine. We do not trust physical devices that we can run behind an I/O Memory Management Unit (IOMMU).

7.6 Conclusions

In this work we provides a possible blueprint for practical verified development of kernel code. Our work demonstrates that a collection of careful design choices combined with modern automated verification tools enables practical development of formally verified kernels. A combination of pointer-centric design, flat permission organization and manual memory management allowed us to develop Atmosphere in a manner similar to unsafe, unverified kernels and with an unexpectedly modest development effort. While we had to be conscious of the internal organization of the system to make sure that verification is possible, in most cases Verus provides a way to move forward without significant limitations on the kernel code and with an excellent degree of proof automation. Atmosphere's low manual proof effort and rapid verification

time open the door for the development of OS kernels with more conceptual complexity. In the later chapters, we will be presenting VeriFlat, a fully-verified concurrent OS microkernel built on top of the things we learned from Atmosphere.

7.7 Evaluation

Primarily, we evaluate Atmosphere to answer two main questions: (1) How practical is the development of verified kernel code with Verus in terms of verification speed and development effort? (2) Can Atmosphere be used as a practical kernel which does not sacrifice performance for formal correctness? To ensure repeatability of experimentation, we conduct evaluation on the publicly available CloudLab [52] machines: c220g5, c220g2, and d430 [15]. We measure the verification time on c220g5, which is configured with two Intel Xeon Silver 4114 10-core CPUs running at 2.20 GHz, 192 GB RAM. Network experiments use a pair of c220g2 machines with Intel X520 10Gb network interfaces. NVMe experiments utilize d430, which are configured with PCIe-attached 400GB Intel P3700 Series SSDs. All the machines run 64-bit Ubuntu 20.04 with a 5.4.0 kernel. To reduce variance in benchmarking, we disable hyper-threading, turbo boost, CPU idle states, and frequency scaling for all the experiments.

7.7.1 Verification Time

Rapid verification is essential for interactive development.

On the c220g5 server, Atmosphere finishes full verification in about 1 minute 7 seconds with 8 threads (figure 7.2).

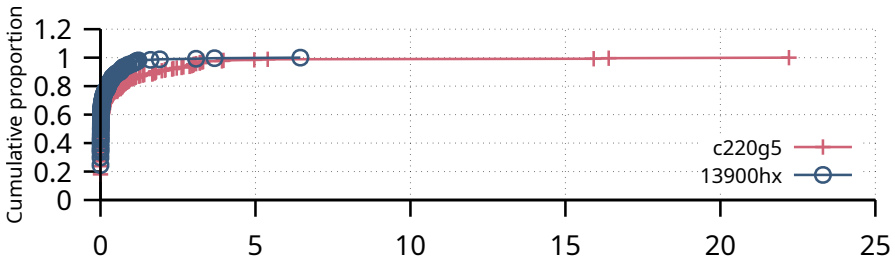


Figure 7.2. Verification time for each function.

Table 7.1. Verification time of different systems on CloudLab c220g5.

System	1 thread	8 threads	Proof	Code	P/C Ratio
NrOS page table	1m 52s	51s	5329	400	13.3:1
Atmo. page table	33s	–	2168	496	4.3:1
Mimalloc	8m 12s	1m 40s	13703	3178	4.3:1
VeriSMo	61m 24s	12m 11s	16101	7915	2.0:1
Atmosphere	3m 29s	1m 7s	20098	6048	3.3:1

On a modern laptop with a recent Intel i9-13900hx CPU, Atmosphere finishes full verification in under 20 seconds (on 32 threads), and 47 seconds (on 1 thread). Table 7.1 lists the verification time of various low-level systems projects built with Verus

7.7.2 Verification Complexity

To put the verification effort of Atmosphere in perspective with prior work, we collect the proof-to-code ratio across several recent kernel projects in Table 7.2.

Table 7.2. Proof to code ratio of different systems.

Name	Language	Spec Lang.	Proof-to-Code Ratio
seL4	C+Asm	Isabelle/HOL	20:1
CertiKOS	C+Asm	Coq	14.9:1
SeKVM	C+Asm	Coq	6.9:1
Ironclad	Dafny	Dafny	4.8:1
NrOS	Rust	Verus	10:1
VeriSMo	Rust	Verus	2:1
Atmosphere	Rust	Verus	3.3:1

Atmosphere has a proof-to-code ratio of 3.32:1, which is a good improvement compared to the existing formally verified microkernels seL4 [32] and CertiKOS [22], which have proof-to-code ratio of 20:1 and 14.9:1, respectively. Interestingly, VeriSMo has an even lower proof-to-code ratio of 2.0. Two factors contribute to the low proof-to-code ratio. First, arguably, VeriSMo is semantically less complex than Atmosphere. Second, the VeriSMo development team prioritized reducing proof effort over verification speed – by significantly relaxing the verification timeout limit, VeriSMo achieves a low proof-to-code ratio at the cost of higher verification time.

7.7.3 Impact of Flat Design

To isolate the impact of our flat approach, we compare the proof-to-code ratio and verification times for the page table subsystems in NrOS [50] and

Atmosphere – both systems use Verus to implement a comparable page table logic with similar verification goals, and a similar size of executable code.

We implement support for a 4-level page table with different page sizes: 4KiB, 2MiB, and 1GiB. The abstract state of the page table is represented by three maps (one for each page size) from virtual to physical addresses along with the permission bits. The tracked permissions of each PML level are stored at the topmost level of each page table. NrOS' page table, however, follows Rust recursive ownership.

Similar to the page allocators, semantically, we use the beginning virtual addresses and physical addresses of the pages(and hugepages) to represent the the entire region of the memory. For example, if the user maps virtual address 0x1000 - 0x2000 (one 4KiB page) to physical address 0x3000 - 0x4000, the abstract state of the `mapping_4k()` will have `mapping_4k()[0x1000].paddr = 0x3000`. By doing so, we avoid some complexity of reasoning memory region disjointness, due to the aligned nature of these physical and virtual addresses.

7.7.3.1 Refinement Proof

The page table uses a range of invariants to prove its structural invariants (e.g, each entry in any PML level only maps to the next PML level). Most importantly, it proves refinement between its concrete state and its abstract mapping. For example, in mappings of 4KiB pages, we use four-level spec

functions to simulate the address resolution of the MMU and prove that the `mapping_4k()` matches what the MMU will theoretically see.

```
forall |l4i: L4I, l3i: L3I, l2i: L2I, l1i: L2I|
  0<=l4i<512 && 0<=l3i<512 && 0<=l2i<512 && 0<=l1i<512
  ==>PT.mapping_4k().contains(index2va((l4i,l3i,l2i,l1i)))
    == PT.resolve_mapping_4k(l4i,l3i,l2i,l1i).is_Some()
forall |l4i: L4I, l3i: L3I, l2i: L2I, l1i: L2I|
  0<=l4i<512 && 0<=l3i<512 && 0<=l2i<512 && 0<=l1i<512
    && PT.resolve_mapping_4k_l1(l4i,l3i,l2i,l1i).is_Some()
  ==> PT.mapping_4k()[index2va((l4i,l3i,l2i,l1i))]
    =~= PT.resolve_mapping_4k(l4i,l3i,l2i,l1i).unwrap()
```

The first `forall` statement ensures that the domain of virtual addresses mapped in the concrete page table is equal to the domain of the abstract mapping. The second `forall` statement ensures the equivalency of the mappings.

One of the most complicated parts of the proof is to establish that if the page table adds a new virtual-to-physical mapping, the abstract mapping of other virtual addresses does not change. In NrOS, such a proof requires manual unrolling of the recursive specs through different PML levels and requires around 200 lines of proof code in `map_frame_aux()` function [50]. Since we have direct access to the states of all page table levels, no unrolling is needed. With around 30 lines (PML4 requires 7 lines) of proof code, we directly prove that all other entries in all PML levels do not change:

```
pub fn map_4k_page(PT: &mut PageTable, dst_l4i:L4I, dst_l3i: L3I, dst_l2i:
    L2I, dst_l1i: L2I, va:VAddr, ...)
{ // executable code
  // other virtual addresses at PML1,2,3 do not change
  // other virtual addresses at PML4 do not change
  assert( forall |l4i: L4I, l3i: L3I, l2i: L2I, l1i: L1I|
    0<=l4i<512 && 0<=l3i<512 && 0<=l2i<512 && 0<=l1i<512
    && ((dst_l4i, dst_l3i, dst_l2i, dst_l1i)
      != (l4i, l3i, l2i, l2i))
```

```
 9         ==> PT.resolve_mapping_l1(l4i,l3i,l2i, l1i)  =~=
10           old(PT).resolve_mapping_l1(l4i,l3i, l2i, l1i));
11   }
```

Compared to NrOS [7], Atmosphere page table has 3x lower proof to code ratio (13.3:1 and 4.4:1). Moreover, on a single thread, verification of the Atmosphere's page table is over 3x faster.

7.7.4 Development Speed and Effort

We completed development of Atmosphere in three stages, or, more specifically, we developed three versions, which were clean-slate rewrites borrowing design and implementation lessons from a previous version (figure 7.3):

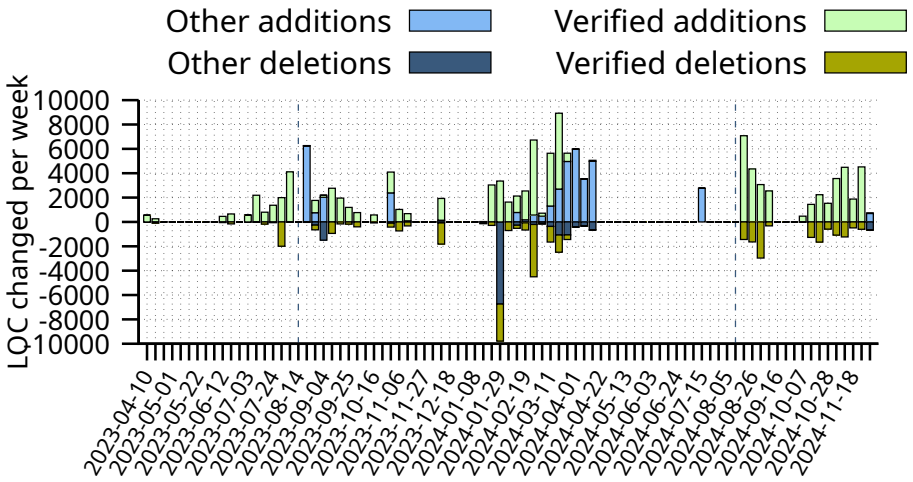


Figure 7.3. Atmosphere commit history.

1. *version 1* (2 months, one person) resulted in a simplistic kernel centered around the process manager and the page allocator, which was aimed primarily at familiarizing ourselves with Verus,
2. *version 2* (8 months two people, clean-slate rewrite) resulted in a simple but functioning kernel with initial ideas of pointer-centric design, flat permission organization and manual memory management, and
3. *version 3* (4 months, one person, 50% code reuse from the previous version) developed support for revocation via recursive container trees, superpages, practical user-level specifications and support of isolation and noninterference proofs.

The entire development of Atmosphere took around 2 person years, with roughly 14 months spent on development of verified code.

7.7.5 Microbenchmarks

We perform several microbenchmarks on a c220g5 node on CloudLab. Note that here we run experiments under the KVM hypervisor with hardware virtualization enabled. We use KVM as it allows us to disable turbo boost and frequency

Table 7.3. Latency of communication and typical system calls (cycles) on c220g5.

System call	Atmosphere	**seL4**
Call/reply	1058	1026
Map a page	1984	2650
Create a thread	2334	-
Create a process	6519	-

In table 7.3, We compare our call/reply with the synchronous IPC mechanism implemented by the seL4 microkernel (we use the IPC call test).

An IPC send/receive mechanism in Atmosphere takes around 1058 cycles, whereas seL4 takes 1026 cycles. We also measure the overhead of mapping a page which takes around 1984 cycles Atmosphere and 2650 cycles in seL4, although the system calls are not strictly equivalent. Overall, careful use of Rust allows Atmosphere to achieve performance comparable to carefully-optimized C.

7.7.6 Device Drivers

In Atmosphere, device drivers can either run as: (1) part of the user process (similar to user-level device drivers like DPDK [17] and SPDK [28]), or (2) as a separate user process that communicates with clients via fast asynchronous IPC. We develop and evaluate two device drivers: (1) an Intel 82599 10Gbps Ethernet driver (Ixgbe), and (2) an NVMe driver for PCIe-attached SSDs.

7.7.6.1 Ixgbe Network Driver

We compare the performance of Atmosphere's Ixgbe driver with a highly-optimized driver from the DPDK user-space packet processing framework [17] on Linux. Similar to DPDK, we use polling mode to achieve peak performance. We configure Atmosphere to run several configurations:

1. atmo−driver: the benchmark application is statically linked with the driver (this configuration is very similar to user-level packet frameworks like DPDK).
2. atmo−c2: the benchmark application runs in a separate process on a different core, and communicates with the driver driving on a separate core through a shared-memory ring buffer.
3. atmo−c1−b1 and atmo−c1−b32: the benchmark application runs as a separate process on the same CPU alongside the driver.

The application uses a shared ring buffer with the driver and invokes the driver through an IPC endpoint which involves a context switch. The number after `b` represents the batch size of the request, i.e., the number of requests the application sends to the ring buffer before invoking the driver. We send 64 byte UDP packets and measure the performance on two batch sizes: 1 and 32 packets (Figure 7.4). For packet receive tests, we use `Pktgen`, a packet generator that utilizes DPDK framework to generate packets at line-rate.

Linux achieves 0.89 Mpps as it uses a synchronous interface and crosses the syscall boundary for every packet and goes through layers of abstraction in the kernel (Figure 7.4). On a batch of 32 packets, both drivers achieve the line-rate performance of a 10GbE interface (14.2 Mpps).

To understand the impact of interprocess invocations, we run the benchmark application as a separate process (atmo−c1−b1) and (atmo−c1−b32). Atmosphere can send and receive packets at the rate of 2.3 Mpps per-core with one context switching per packet (atmo−c1−b1). On a batch of 32 packets, the overhead of

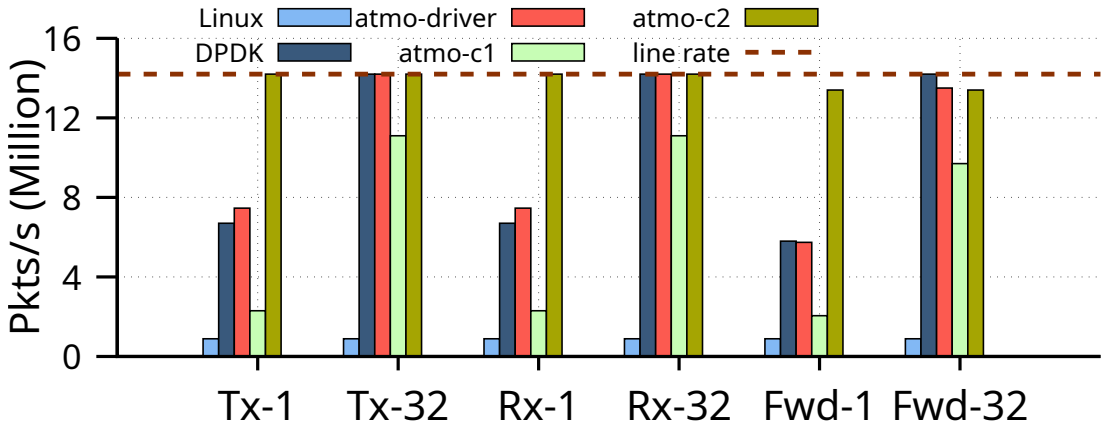


Figure 7.4. Atmosphere Ixgbe driver performance.

context switching is less pronounced and the application achieves 11.1 Mpps (atmo−c1−b32).

7.7.6.2 NVMe Driver

To understand the performance of Atmosphere's NVMe driver, we compare it with the multi-queue block driver in the Linux kernel and a well-optimized NVMe driver from the SPDK storage framework [28]. Similar to SPDK, the Atmosphere driver works in polling mode. Similar to Ixgbe, we evaluate NVMe driver under various configurations:

1. statically linked (atmo−driver);
2. application and driver run on different cores and communicate through shared ring buffer (atmo−c2);

3. application runs on the same core with the driver and uses the endpoint to invoke the driver process after sending a batch of requests (atmo−c1−b1 and atmo−c1−b32).

We perform sequential read and write tests with a block size of 4KiB on a batch size of 1 and 32 requests (Figure 7.5).

On Linux, we use `fio`, a fast I/O generator; on SPDK and Atmosphere, we develop similar benchmark applications that submit a set of requests at once, and then poll for completed requests. To set an optimal baseline for our evaluation, we chose the configuration parameters that could give us the fastest path to the device. Specifically, on Linux, we configure `fio` to use the asynchronous `libaio` library to overlap I/O submissions, and bypass the page cache with the `direct` I/O flag.

On sequential read tests, `fio` on Linux achieves 13K IOPS and 141K IOPS per-core on the batch size of 1 and 32, respectively (Figure 7.5). On a batch size of

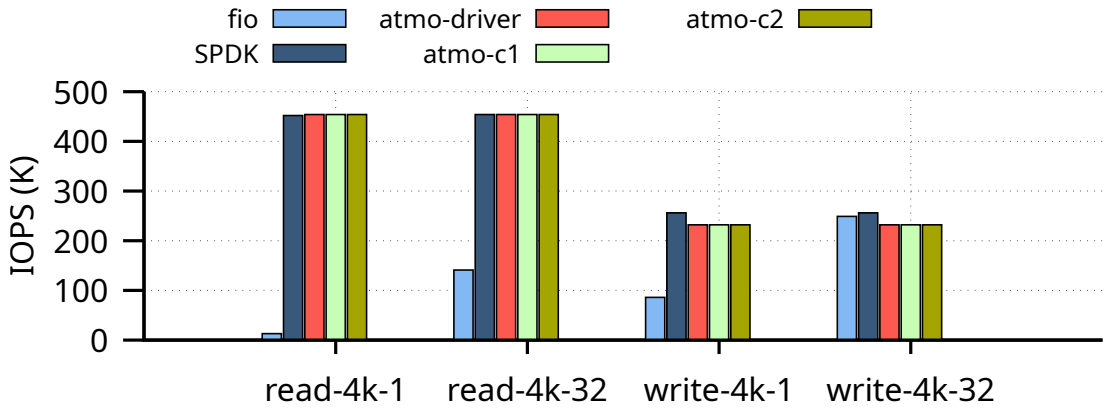


Figure 7.5. Atmosphere Performance of the NVMe driver.

1 and 32, the Atmosphere driver performs similar to SPDK, achieving maximum device read performance. On sequential write tests with a batch size of 32, Linux is within 3% of the device's maximum throughput of around 256K IOPS. Atmosphere driver incurs an overhead of 10% on all the configurations for writes (232K IOPS).

7.7.7 Application Benchmarks

To further evaluate the performance of Atmosphere under real-world scenarios, we develop three data-intensive applications on top of the device drivers: (1) the Maglev load balancer (Maglev) [18], (2) a memcached-compatible key-value store (kv−store).

Since device drivers in Atmosphere run as user space processes, we evaluate the applications in two scenarios. First, when the application itself communicates with the device driver through shared memory, and second, when the device driver runs in the same process with the application.

7.7.7.1 Maglev Load-balancer

Maglev is Google's load balancer with an algorithm that evenly distributes incoming traffic among backend servers [18]. To compare with Atmosphere we develop a normal Linux program that uses the socket interface, and a DPDK-powered application that directly accesses the network card via PCIe passthrough [17].

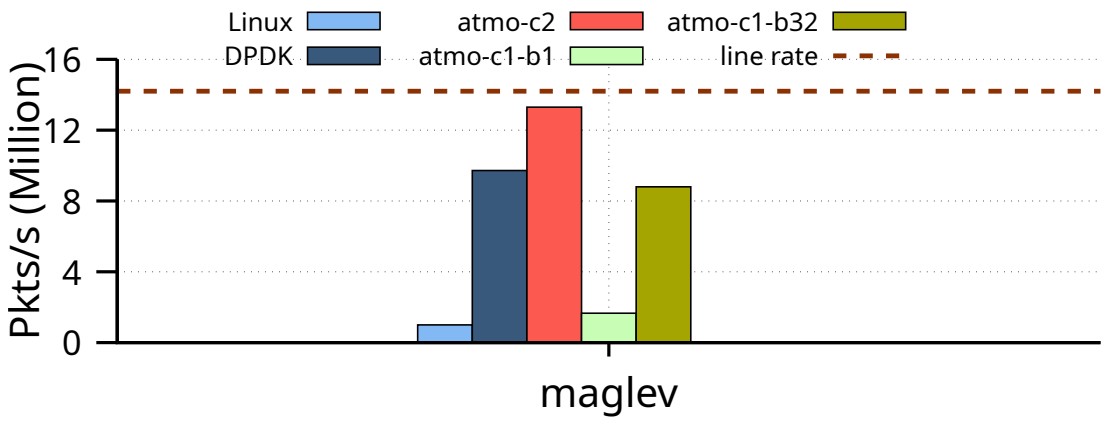


Figure 7.6. Atmosphere Performance of Maglev.

With a normal Linux socket interface, Maglev only achieves 1.0 Mpps per core because of both the overhead of the system call interface and an overly generic design of the Linux network stack (figure 7.6). The DPDK-powered application has direct access to the NIC and achieves 9.72 Mpps per core. In Atmosphere, Maglev achieves 13.3 Mpps with the device driver running on a separate core with communication established over a shared ring buffer (atmo−c2). With Maglev invoking the device driver on the same core, the load balancer runs at 8.8 Mpps with a batch size of 32 (atmo−c1−b32), and 1.66 Mpps with a batch size of 1 (atmo−c1−b1).

7.7.7.2 Key-value Store

Key-value stores are crucial building blocks for modern datacenter systems ranging from social networks [46] to key-value databases [16]. To evaluate Atmosphere's ability to meet the performance requirement of datacenter

applications, we develop a prototype of a network-attached key-value store, kv−store.

Our implementation relies on an open addressing hash table with linear probing and uses the FNV hash function. In our experiments, we compare three implementations: a C version running on Linux with the DPDK driver, an Atmosphere program that executes in a process separate from the driver but utilizes a separate core (atmo−c2) and as a separate process but on the same core with batch size of 32 (atmo−c1−b32). In figure 7.7, We evaluate two hash table sizes: 1 M and 8 M entries with three sets of key and value pairs (<8B,8B> , <16B,16B> , <32B,32B>).

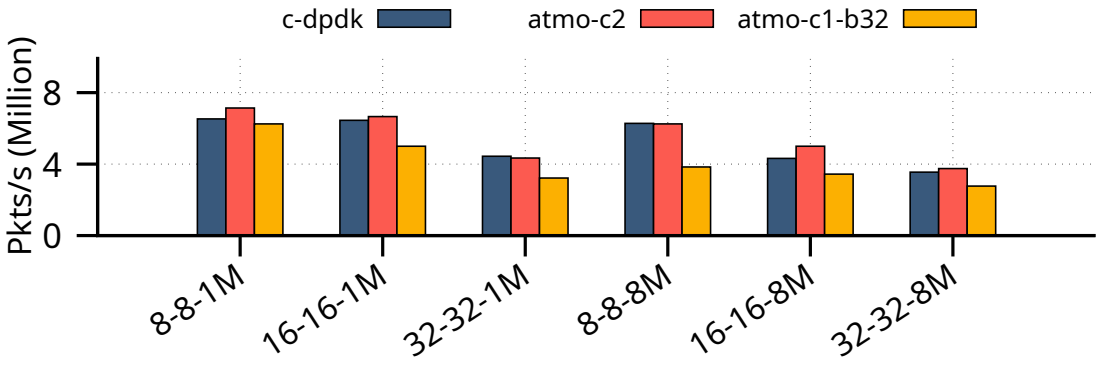


Figure 7.7. Atmosphere Performance of Key-value store.

CHAPTER 8

CHALLENGES IN VERIFYING CONCURRENT PROGRAM

Verifying concurrent program is notoriously difficult. Verifying a sequential program often involves proving a refinement theorem that the low-level implementation of the program refines the high level state machine model. For a concurrent program, the refinement theorem needs to hold true under every possible execution history (under any interleaving). However, interleaving of concurrent operations from different threads explodes the possible execution of the program, significantly increasing the complexity of the high-level state machine model because it now needs to model concurrent changes, and the refinement proof because it now needs to build forward simulation from concurrent implementation level operations to sequential transitions of the high-level state machine.

8.1 Separation Logic

Separation Logic [51] is an approach to reasoning about programs that manipulate shared mutable data structures. In SL, memory can be divided into separate, non-overlapping parts. Running a program that manipulate one part of the memory does not affect the state of other disjoint parts. Sounds familiar? This

is the fundamental theory behind the Verus linear ghost permissions. Each Rust object (generated to be disjoint by the correctness of the allocator), can be turned into a linear ghost permission, whose operations do not affect the state of other linear ghost permissions and objects in the system.

8.2 Concurrent Separation Logic

In a concurrent program, if two threads can access the same memory location and at least one writes to it, their operations can interfere with each other. This leads to race conditions and makes reasoning incredibly difficult. Traditional sequential reasoning fails since the proof cannot prove one thread's execution correct while assuming the others do not change the state, because they do. However, if we extend SL to a concurrent setting, that is, if threads operate on physically disjoint parts of memory, they cannot interfere by definition. To prove this disjointness, the program usually uses locks to grant exclusive access to a thread, or, in the case of Rust based verification, Rust's thread safety, in which the linear ownership guarantees exclusive accesses.

8.3 Challenge: Deadlocks

Deadlocks are specially hard to verify against because of two reasons. First, reasoning about deadlocks is beyond Floyd-Hoare logic, which does not have a notion of time Temporal logic is needed for reasoning about the absence of deadlocks (eventually the lock one thread is acquiring will be released), which does not integrate well with normal programming languages (in any

programming language, the code only expresses the changes to the program state, without any notion of time). Second, reasoning about deadlocks requires the knowledge of other threads' state, which defeats concurrent separation logic's principle and modular verification. As a result, reasoning about deadlocks has high conceptual complexity and verification overhead.

8.4 Background: Related Work

8.4.1 IronFleet

IronFleet [24] is an research project for proving distributed systems correct built in Dafny. It is a landmark methodology that provides end-to-end, machine-checked proofs that practical distributed systems (like key-value stores) are both safe and live.

IronFleet's core methodology consists of three layers of proof: High-level spec layer; Distributed protocol layer; And implementation layer. IronFleet aims for three verification goals: Safety of the system – Nothing bad ever happens. Liveness – Something good eventually happens. Atomicity of the state machine model in the high level spec layer – the system appears to be atomic to the customers.

IronFleet uses traditional refinement proof to build proofs between different layers. First, each local host (implementation layer) is completely sequential. The refinement proof abstract away all the low-level details in the implementation layer, and thus simplifies the protocol layer. The protocol layer models the distributed system state as a state machine consisting of all the hosts and the

network environment (events including packets being successfully delivered or dropped). Concurrency is then introduced in the protocol layer, since multiple host and the network environment can advance the distributed system state machine concurrently. Therefore, receiving and sending an packet is a single atomic action in the protocol layer state machine, since they cannot affect more than one entity in the system – receiving a packet only affects the host that receives the packet; sending a packet only affects the state of the network environment since it is not yet delivered. Since each local host is completely sequential and cannot affect the rest of the system unless it sends a packet to the network, the protocol layer can squash all the local actions of a host between any two actions of receiving or sending packets into a large atomic action in the protocol state machine model (although for better proof engineering, IronFleet can also choose to squash these actions into multiple actions, as long as they do not contain receiving or sending packets). The safety invariants of system are also implemented and checked at the protocol layer – by showing that after each atomic action in the protocol state machine, all the invariants are still preserved. The last piece is to build refinement proof between the state machines in the high-level spec and protocol layer. Specifically, the proof shows that each step in the protocol layer advances the high-level state machine by one or zero step. For example, for an distributed key-value store system, the high-level state machine is just a hash table, whereas the protocol layer includes all the hosts (the leader host for key dispatching, and worker hosts each maintaining a shard of the hash table)

and the network. When the leader host receives an update request from the user, there are a few steps in the protocol layer:

1. the leader finds out which host is holding the correct shard of the key-value store
2. the leader sends a packet containing the key and new value to the local host that maintains the key
3. the local host receives the packet
4. the local host updates the shard accordingly
5. the local host sends a packet to the leader to notify the successful update
6. the leader receives the packet
7. the leader notifies the user for the successful update.

Only step (7) updates the state of the abstract layer. The refinement proof hides the complexity of the distributed protocol and provides a clean interface to the user.

To prove liveness, IronFleet encodes the TLA rules in Dafny. Typically, the liveness proof involves demonstrating that if some condition holds then eventually another condition holds. In IronFleet, it assumes that local hosts make progress (make a step in the protocol layer) infinitely often, and the network eventually delivers all the resent packets. The liveness proof then applies the TLA rules to prove that if a client repeatedly sends request, it eventually receives a reply.

8.4.2 IronSync/VerusSync

IronSync [23] and VerusSync [33] are two verification frameworks focus on truly concurrent programs with shared memory. IronSync uses linear Dafny as it base language. VerusSync uses Verus. Both frameworks reply on linear ownerships (IronSync on linear types in Dafny and VerusSync on Rust ownership) to guarantee exclusive accesses of shared objects and therefore employs concurrent separation logic for reasoning about concurrency.

Different from IronFleet, in which the local hosts share no memory or other states and thus can use traditional sequential verification techniques, each thread in IronSync and VerusSync do share memory with other threads. However, the linear ownerships in both frameworks can ”shard” the system state into small, exclusively owned shards to each thread, therefore allowing the actions of each thread to be reasoned as if they are completely sequential. To lift the thread local proofs into the global level proofs with concurrency, IronSync and VerusSync introduce the Localized Transition System (LTS). With LTS, the global state is modeled as a state machine that can be sharded into thread local shards, and operations of each shard are modeled as individual transitions in the global state machine. This approach is similar to the protocol layer in IronFleet. The LTS then checks that all the invariants of the system are always satisfied after each transition. Even though invariants may be broken during the transitions, since the linear ownership guarantees exclusive accesses, none of these broken invariants can be observed.

IronSync and VerusSync focus on safety and correctness of the system (only good things happen), but not liveness.

8.4.3 CertiKOS

CertiKOS [22] is the world's first formally verified, concurrent operating system kernel, demonstrating that a practical kernel for multi-core processors can be mathematically proven for safety, functional correctness and deadlock freedom. CertiKOS's success stems from a novel, layered verification architecture that makes the massive proof manageable. Instead of verifying the entire 6,500-line kernel (in C (CompCert) and assembly) as a single block, it decomposes the system into a hierarchy of small, certified abstraction layers. The corner stone of this method is contextual refinement: a proof that the concrete implementation of any kernel function behaves exactly like its abstract specification, under any possible interleaving with other threads or user programs. This is a very strong guarantee that the kernel is correct in all contexts. In its essence, contextual refinement requires the implementation of one thread to always refine the high level spec under any arbitrary concurrent contexts (any possible behaviors of other threads). The key difference between contextual refinement and CSL is that in CSL, the context (what other threads do) does not matter, since the resources are exclusive to the local thread, whereas in contextual refinement, the context does matter, but no matter what other threads do, the refinement proofs still hold. However, to make verification of the sequential thread local implementation code possible, CertiKOS introduces a notion of CPU local copy (in the proof code, not

real physical copies) of each shared object, which is very similar to the linear ownerships from linear Dafny and Verus. Essentially, the rules of local copy ensures that only one CPU has right to access (write or read) a given shared object. To lift the thread local proof to the concurrent machine model, CertiKOS performs Shuffling and Merging of switch pointers (interleaving from other thread). This is very similar to IronFleet's reduction – before each local copy is released (and becomes available) to other threads, all the changes are invisible to other threads, and therefore it is OK to not consider interleaving between the local changes and changes of other thread.

8.4.4 Iris: Higher-Order Concurrent Separation Logic

Iris is a higher-order concurrent separation logic framework, implemented and mechanized in the Coq proof assistant, designed for modular verification of fine-grained concurrent programs [31]. Building on the foundations of separation logic [51], Iris introduces several key primitives that together enable reasoning about shared mutable state without sacrificing compositionality. Central to the framework is the notion of a resource algebra — a composable algebraic structure that gives precise semantics to ghost state, allowing proof authors to track logical ownership of both heap resources and abstract bookkeeping quantities that exist only at the proof level. Shared mutable state is governed through invariants, persistent propositions maintained at every program step, whose contents can be temporarily accessed within a single atomic action under the discipline of a mask — a statically-tracked set of invariant names that prevents re-entrant opening and

is enforced as a type-level constraint in Coq. For specifying the observable behavior of implementations to clients, Iris provides logically atomic triples, which assert that an operation — however complex its internal implementation — appears to take effect at a single identifiable linearization point, permitting lock-based implementations to be given the same clean atomic specifications as their lock-free counterparts. Together, these primitives form not a fixed program logic but a meta-logic: a framework upon which domain-specific logics, type systems, and reasoning principles can be built and verified once, with their soundness inherited from Iris's single mechanized proof.

However Iris has some practical limitations for building a verified concurrent OS kernel. Its default language, HeapLang, is an uncompilable lambda calculus with no notion of raw memory layout, pointer arithmetic, inline assembly, or privileged instructions, meaning any OS model written in it is a mathematical abstraction that never runs on hardware. Beyond the compilation gap, Iris's memory model assumes sequential consistency and a flat abstract heap, whereas OS kernels depend critically on weak memory models, physical versus virtual addressing, MMIO, and concurrency patterns such as interrupt handlers, RCU, and per-CPU spinlocks that do not map cleanly onto Iris's invariant and ownership discipline. Finally, Iris proofs are extremely expensive to write: verifying small concurrent data structures of a few hundred lines of HeapLang routinely requires months of expert effort, making verification at the scale of an OS kernel currently impractical.

8.4.5 CIVL

CIVL [54] (by Kragl and Qadeer, built on Boogie [3]) is a deductive verifier for concurrent programs built around the idea of *layered refinement*: rather than verifying a concurrent program in a single monolithic proof, the programmer expresses the program as a sequence of abstraction layers `P1, ..., Pn` where `P1` is the concrete low-level implementation and `Pn` is a simple high-level specification whose correctness is obvious. Each consecutive pair (`Pi` , `Pi+1`) is connected by a sound program transformation, and CIVL automatically generates and discharges the verification conditions that justify each transformation using the Z3 SMT solver.

The key mechanism that makes refinement tractable is *mover types*, drawn from Lipton's reduction theory [41]: each atomic action is classified as a right mover (it can commute rightward past any concurrently executing action), a left mover (it can commute leftward), both, or neither. A sequence of right movers followed by an arbitrary action followed by left movers can be reduced to a single atomic step, which eliminates the need to reason about the interleavings within that sequence. This allows a fine-grained lock-based implementation – in which lock acquisitions are right movers, critical section actions are both movers, and releases are left movers – to be atomized into a single abstract action at a higher layer, precisely capturing the lock–operate–unlock atomicity argument.

CIVL also supports *yield invariants*, named and parameterized noninterference specifications that express what a thread may assume about its

environment at each yield point, serving a role analogous to rely conditions in rely/guarantee reasoning. Compared to Iris, CIVL operates on a real structured imperative language compiled through Boogie and discharges proof obligations automatically via SMT, making it more amenable to automation at the cost of expressiveness: CIVL's assertion language is first-order, and it lacks Iris's higher-order ghost state and modular resource algebra infrastructure, making it less suitable for verifying algorithms whose correctness arguments depend on intricate ownership transfer or recursive data structure invariants.

CHAPTER 9

VERIFLAT

This work is currently in progress and subject to revision.

9.1 Introduction

In this work, we present techniques and methodologies to build and formally verify a general-purposed concurrent OS microkernel with favorable verification speed and overhead. Specifically, we use Verus [33] for developing a fully-verified, microkernel with fine-grained locking concurrency, VeriFlat.

VeriFlat system architecture is largely similar to Atmosphere However, VeriFlat differs from Atmosphere in several aspects: VeriFlat is not under a global kernel lock. Instead, VeriFlat supports concurrent execution of system calls on different CPUs. VeriFlat, as a result, supports better resource separation than Atmosphere – not only achieving strict isolation between different compartments (containers), but also informally ensuring performance isolation. Despite concurrency, we apply logic atomicity to specify VeriFlat system call as simple linearizable atomic operations, completely hiding the implementation details of locking and synchronization.

Similarly to Atmosphere, we develop all code in Rust and prove its functional correctness with Verus. To support custom concurrency rules in VeriFlat, We introduce minimal modifications to Verus.

9.2 Verification of Concurrency

VeriFlat, similar to Atmosphere, proves two major theorems:

1. Refinement between the implementation and the high-level atomic specification
2. Well-formedness of the kernel before and after each transition.

We model the microkernel as one kernel view state machine operating on the abstract state of all kernel objects and a user view state machine, which is a strict subset of the kernel state machine and contains only user-aware objects (e.g., schedulers, addresses spaces, etc.).

Refinement: The refinement theorem establishes high-level functional correctness and atomicity of each system call on the user view state machine. The implementation of the microkernel is a refinement of its high-level abstract specification, and each system call appears to users as completely linearizable and atomic operation that occurs instantaneously, in its entirety, or not at all. For several long-running system calls that kill active containers, processes, threads (and their recursive child objects), the system calls contain a fixed number of atomic operations with total ordering.

Well-formedness: The well-formedness theorem proves that high-level correctness invariants about the system state hold after each state transition.

Different from the system calls' atomicity, each system call contains multiple logically atomic operations to the kernel view state machine, and therefore, invariants are proven to hold before and after each atomic operation on the kernel view. Since each system call contains only complete atomic operations on the kernel view, the invariants hold for all the user-level programs too.

9.2.1 Locks and LocalContext

RwLock: VeriFlat supports concurrency with fine-grained locking, i.e., each object in the kernel is protected by at least one lock. The locks' internal implementation is completely trusted. The reader-writer(Rw) lock tracks the number of readers and writers and ensures only one thread has write access or multiple threads have read access. After successfully acquiring the lock, the lock returns a tracked lock permission `LockPerm`, which is similar to the `PointsTo<T>` in Verus. `LockPerm` tracks the identify of the lock as well as the reads/writes right of the acquired lock.

Lock ID: Each lock maintains a pure ghost lock id consisting of the following elements

1. Container Depth: the depth of the container the object behind the lock is assigned to, if applicable.
2. Process Depth: the depth of the process the object is assigned to, if applicable.
3. Lock Major ID: the id based on the type of the object. May change based on the state of the object.

4. Lock Minor ID: the id based on the physical address of the object. Remains constant throughout the lifetime of the object.
5. Killed state: if a kernel thread issues kill to the object, it changes the state of the lock to `Kill`. Only containers, processes, and threads have `kill` state.

LocalContext: At each system call's entrance, the TCB grants the kernel thread a ghost, unforgeable tracked object of type `LocalContext`, which tracks the acquired locks and locking state of the current thread. Through out the entire execution of the system call, `LocalContext` needs to be passed as a mutable reference when acquiring and releasing locks.

`LocalContext` tracks the following states:

1. Record all acquired locks to avoid acquiring the same lock twice or releasing a lock without acquiring.
2. Record the `lock id` of all acquired locks and the ordering
3. Track the state of the current execution in `Acquire` or `Release` mode.

Later sections introduces how these states alongside with lock id aid to verifying deadlock freedom and logic atomicity.

Locking with LocalContext: Below code shows how to acquire a write lock of type `T` without `kill` state.

```
1  pub fn wlock(&mut self, Tracked(lctx): Tracked<&mut LocalContext>,
       lock_id: Ghost<LockId>) -> (ret:Tracked<LockPerm>)
2      requires
3          old(self)@.container_depth() == lock_id@.container,
4          old(self)@.process_depth() == lock_id@.process,
5          old(self)@.lock_major_sat(lock_id@.major),
6          old(self)@.lock_minor() == lock_id@.minor,
7
8          wlock_requires(*old(self), old(lctx)),
```

```
 9            old(lctx).lock_id_acyclic(lock_id@),
10        ensures
11            self.locking_thread() == RwLockState::Write { thread_id: lctx.
                  thread_id(), lock_id: self.lock_id() },
12            self@ == old(self)@,
13            ret.state() is WriteLock,
14            ret.lock_id() == self.lock_id()
15
16            lctx.lock_seq() =~= old(lctx).lock_seq().push(self.lock_id())
17            wlock_ensures(lctx),
18    {
19        self.lock.wlock();
20        Tracked::assume_new()
21    }
```

The trusted function takes a mutable reference of `LocalContext`, a precomputed `lock_id`, and returns a tracked `LockPerm`. In lines 3 to lines 6, the spec requires the precomputed `lock_id` match the lock id of the object (old(self)@) and its current state. Line 8 requires `lctx` to not hold the lock to `self` before invoking `wlock()`. Line 9 requires the caller to prove that the lock ids of acquired locks plus `self` remains acyclic. Line 11 records the locking kernel thread of `self` is `lctx`, line 12 ensures no changes in the object under the lock, line 13,14 grant write access to `ret`, the lock permission, and finally line 16 adds the current lock id to the end of `lctx.lock_seq()`, which tracks the ordering of all the lock ids.

9.2.2 Deadlock Prevention

VeriFlat formally guarantees the absence of deadlock throughout all possible execution paths in all system calls with the classic lock ordering algorithm.

Recall that each `Lock` maintains a `Lock Id` and `LocalContext.lock_seq()` records a sequence of `Lock Id` in the order of which the locks are acquired. Upon

each acquire, the code must prove that the lock ids in the sequence are still monotonically increasing after acquiring the lock.

9.2.2.1 Dynamic Lock Major ID

The lock id of each lock consists of 5 fields (in the table), and the lock ids are compared based on the order in the previous table. That is, if a process `P1` belongs to a container with higher depth than a process `P2` or `P1` has the same container depth as `P2` but higher process depth (meaning `P1` is a child process of `P2`), `P1` can be locked after `P2`, This primarily enables implementing `kill` system calls, in which the user processes higher in the hierarchy have the right to recursively terminate child processes. Threads, for example, have dynamic lock major id based on the state when they were unlocked. The primary reason for the dynamic lock major is because of some unavoidable circular lock dependencies in `send()` and `receive()` system calls. In a `send()` system call, locks are acquired in the following order:

1. Lock the thread issuing the `send`
2. Lock the endpoint the thread is sending to
3. Lock the first thread in the endpoint queue

This forms an obvious circular dependency – `thread` -> `endpoint` -> `thread`. A key observation to break this circle is that the threads in `Running` state (so they could invoke the `send()` system call in the first place) are always locked before threads in `Blocked` state (threads blocked on the endpoint queue). Therefore, by assigning different lock ids to threads in different states, we breaks this circular

dependency. Specifically in this case, `Running thread` < `endpoint` < `Blocked thread`.

9.2.2.2 Kill State

A different possible circular dependency occurs in the `kill` system call, in which a container or a process terminates its direct or indirect children. In a `kill_process(Proc_A)` system call, locks are acquired in the following order:

1. Lock all child processes of `Proc_A` by their depth in the process tree.
2. Lock all the threads of child processes of `Proc_A`.
3. Some locked threads are blocked on endpoints, lock these endpoints.

Apparently, (3) causes circular dependency: `Running thread` -> `Blocked thread` -> `endpoint` which we just tried to solve above. To combat this as well as to enable a smoother process for `kill ()` system calls, we introduce a kill state in the locks. For a normal reader/writer lock, when threads are acquiring the locks, they first try to enter a critical section by performing a CAS (compare and swap) operation on a guard variable on the lock, once in the critical section, they check the number of readers/writers, update the number if allowed, or exit the critical section and try again. We add a second variable `killed(thread_id)`, (in which the `thread_id` is completely ghost) alongside the readers/writers counter to indicate that the object is being actively killed by a kernel thread with `thread_id`. Before checking the number of readers/writers, the acquire function first checks `Killed(thread_id)` field to make sure liveness of the object, if it is killed, the system call will return to user-level and report that the object is currently being

killed. When an object is marked as `kill`, its container depth, process depth, and lock major id change accordingly. Therefore, the circular dependency disappears, as all the threads will be marked `kill` before the kernel thread locks any endpoint, and any other kernel threads that lock the endpoint will return to user-level if they try to lock a killed thread.

9.2.3 Logic Atomicity

9.2.3.1 Logic Atomicity in Kernel View

A classical approach to achieving atomic behavior in concurrent systems is the *lock–operate–unlock* pattern: a thread acquires a range of mutual exclusion locks, performs a sequence of reads and writes over the protected state, and releases the locks upon completion. Since the locking discipline is enforced through the Rust type system (the read/write permission is granted after a successful acquire and taken upon a release), there exist either multiple readers or exclusively one writer simultaneously, so the protected state transitions from one consistent state to another without any intermediate state being observable by a concurrent thread.

In this sense, the operation is *serially atomic*: contending threads are totally ordered by their lock acquisitions, and each observes only the pre- or post-state of every other thread's critical section. However, serial atomicity is weaker than *logical atomicity* in the sense of linearizability [27]: the lock–operate–unlock pattern has observable duration, in that any thread not contending on the same

lock may observe the passage of time while the lock is held, and the operation cannot be treated as an instantaneous event on a global timeline.

In the Iris program logic [31], this distinction is made precise: a lock-based implementation satisfies a *logically atomic triple* if and only if the locks are entirely internal to the module — never exposed through the interface — so that clients have no means of observing the lock's held state. In that case, since the system call interference never exposes the state of the locks in the system, each *lock–operate–unlock* pattern can be seen as a logically atomic transition to the kernel view state machine. In that case, the proof identifies a single *linearization point* inside the critical section, typically the moment of the final lock release.

9.2.3.2 Logic Atomicity in User View

For the user view state machine, some *lock–operate–unlock* patterns can be merged to form a bigger atomic transition, if and only if the observable objects form only a single *lock–operate–unlock* pattern in the transition. In practice, all system calls except for `kill()` satisfy the above constraint.

9.2.3.3 Rust ownership in Logic Atomicity

In CIVL [54], operations are classified into different movers – `acquire` is a right mover, `release` is a left mover, and operations on locked objects are both movers. We can then stitch a *right–both–left* pattern into a large logically atomic step. Rust ownership model further simplifies this mechanism. Since all objects are thread-safe in safe Rust, all operations except `acquire` and `release` are

naturally both movers – Rust guarantees that objects operated on the local thread are either the read-only or exclusively owned. Therefore, we can completely avoid reasoning about the majority of the operations for logical atomicity – all we need to reason about are `acquire` and `release`.

9.2.3.4 Enforce Logic Atomicity with LocalContext

Recall that `LocalContext` maintains the state of the current execution as `Acquire` or `Release`. `LocalContext` changes the state from `Acquire` to `Release` whenever the local thread releases a lock, and denies any further attempts to acquire locks if the state is in `Release`.

Specifically, Line 8 in wlock() requires the `LocalContext` to be in `Acquire` state, and `wunlock()` transitions the state to `Release` state.

To restore the state back to `Acquire`, `LocalContext` has to invoke a range of proof functions that check global invariants and model the possible interleaving from other threads.

9.2.3.5 Model Interleaving with LocalContext

VeriFlat relies on a simple, auditable trusted proof function `interleave()` that transitions the system state with *all* the possible concurrent actions from other threads. Specifically, the system state after interleaving can be in any state, except for objects that are locked by the current thread. Object lifetimes and states are all advanced into new unknown states, and new objects may be added to the new

system state. This models the maximum possible state changes from any number of threads after any possible number of random transitions.

9.2.4 System Invariants

In a system that guarantees logical atomicity, the correctness of shared data structure invariants need only be established at *linearization points* rather than at every intermediate program step [27].

Intuitively, because a logically atomic operation appears to take effect instantaneously from the perspective of all concurrent threads, the system's observable states form a sequence of consistent snapshots separated by atomic transitions; any state that arises between the start and the linearization point of an operation is internal to that operation and is never witnessed by an external observer. It therefore suffices to verify that every invariant I holds in the pre-state immediately before each linearization point and is re-established in the post-state immediately after, without requiring I to hold continuously throughout the operation's physical execution. This significantly reduces the proof burden in highly concurrent systems.

9.2.5 Restoring LocalContext State after Linearization point

Linearization points exist at the end of each *lock–operate–unlock* pattern, which can be spread out at random places in the system call execution paths. Therefore, to ensure sound and sufficient invariant checks, similar to `interleave()`, we bound the obligation to check global invariants to `LocalContext`.

Specifically, the `Kernel` struct (the top level struct that governs all kernel objects) allows `LocalContext` to switch back to `Acquire` after verifying all global invariants and invoking the `interleave()`.

9.2.6 System Call Specifications

In VeriFlat, the approach to specifying the interface of the concurrent kernel module is to require that every exported system call is logically atomic, presenting to the user-level programs a clean sequential specification that abstracts away entirely the existence of locks in the system. Each system call is associated with an atomic transition on the *user view* abstract state — covering the state of the following items in the system:

- Containers and the container tree
- Processes and the process tree
- Threads
- Each CPU's execution status
- Endpoints
- Schedulers
- Address Spaces
- Each containers' resource quota (e.g., memory, PCIe devices, etc.).

The rest of kernel objects and all the locks are completely invisible to the user level, such that their states have no effect on the observable behavior (the system call return value) of the system calls. The implementation may internally employ any combination of fine-grained locks, IPIs (inter-process-interrupts) to issue TLB

flushes or updates on the user-level address spaces to achieve correctness, but all such mechanisms are entirely hidden behind the interface boundary; user programs neither acquire any lock nor observe their held state, and the system calls make no reference to them.

This design makes user-level reasoning straightforward, since a user program concurrently invoking system calls may treat each system call as a single indivisible step in a sequential proof, without needing to reason about the internal states the implementation traverses between the call and return. The user-level program may only reason about the possible interleaving at the granularity of a system call, rather than each internal step, which drastically lowers the complexity of using the system call specification for developing verified user-level program.

9.2.7 Automation Management

9.2.7.1 Invariants Isolation

VeriFlat, as the name suggested, takes the flat approach in Atmosphere to the extreme. A key observation to the system global invariants is that the most of the invariants are orthogonal to each other. Therefore, it is more favorable, towards both the automation and verification speed, to choose more aggressive triggers for each invariant, and isolate them from each other. To achieve more freedom on the openness of system's invariants, VeriFlat keeps a flat layout for *all* kernel objects, directly storing them at top level, under `struct Kernel`:

```
1
```

```
2  pub struct Kernel{
3      pub pagetable_map: LockedMap<RwLockPageTableRoot, PageTable<
           PT_TYPE>, PAGE_TABLE_HAS_KILL_STATE>,
4      pub page_array: LockedArray<Page, NUM_PAGES, NO_KILL_STATE>,
5      pub cpu_array: LockedArray<Cpu, NUM_CPUS,
           CPU_HAS_KILL_STATE>,
6      pub cpu_tlb: CpuTLB,
7
8      pub root_container: RwLockContainerPtr, // Never dies
9      pub container_map: LockedMap<RwLockContainerPtr, Container,
           CONTAINER_HAS_KILL_STATE>,
10     pub number_containers: RwLock<NumContainers, NO_KILL_STATE>,
11     pub scheduler_map: LockedMap<RwLockSchedulerPtr, Scheduler,
           SCHEDULER_HAS_KILL_STATE>,
12     pub process_map: LockedMap<RwLockProcessPtr, Process,
           PROCESS_HAS_KILL_STATE>,
13     pub thread_map: LockedMap<RwLockThreadPtr, Process,
           THREAD_HAS_KILL_STATE>,
14     pub endpoint_map: LockedMap<RwLockEndpointPtr, Endpoint,
           ENDPOINT_HAS_KILL_STATE>,
15     pub allocator_4k_map: LockedMap<RwLockPageAllocatorPtr,
           PageAllocator, ALLOCATOR_HAS_KILL_STATE>,
16     pub allocator_2m_map: LockedMap<RwLockPageAllocatorPtr,
           PageAllocator, ALLOCATOR_HAS_KILL_STATE>,
17     pub allocator_1g_map: LockedMap<RwLockPageAllocatorPtr,
           PageAllocator, ALLOCATOR_HAS_KILL_STATE>,
18
19     //rest of the fields
20 }
```

Each invariant spec function is by default an open spec function wrapped in a closed spec function, which can be temporarily opened with a proof function that reveals the context of the open spec function:

```
1  pub open spec fn container_pages_wf_inner(&self) -> bool{
2      &&&
3      forall |page_index:PageIndex|
4      page_index_wf(page_index)
5      ==>
6      {
7          self.page_array.spec_index(page_index).view().view().state
               matches PageState::Allocated2m{state: Allocated2MPageState::
               AsContainer}
8          ==>
```

```
 9              self.container_map.dom().contains(page_index2page_ptr(
                    page_index))
10          }
11
12          &&&
13          forall |c_ptr:RwLockContainerPtr|
14          self.container_map.dom().contains(c_ptr)
15          ==>
16          page_ptr_2m_valid(c_ptr)
17          &&
18          {
19              self.page_array.spec_index(page_ptr2page_index(c_ptr)).view().
                    view().state matches PageState::Allocated2m{state:
                    Allocated2MPageState::AsContainer}
20          }
21
22  }
23  pub closed spec fn container_pages_wf(&self) -> bool {
24          &&&
25          self.container_pages_wf_inner()
26  }
27  pub proof fn container_pages_wf_proof()
28          ensures
29              forall |s:Self|
30              s.container_pages_wf() <==> s.container_pages_wf_inner()
31  {}
```

`container_pages_wf_inner` (line 1) is the spec function that encodes bidirectional invariants ensuring memory safety and leak freedom for 2MiB hugepages allocated and retyped as containers. `container_pages_wf` (line 23)is the actual spec function that is exposed to the verifier (by since it is a closed spec function, its body is not exposed). When needed, we manually invoke `container_pages_wf_proof` (line 27) to reveal `container_pages_wf` to the rest of the world.

A typical flow for verifying the invariants in VeriFlat is to prove the existence of target kernel objects, lock, operate and unlock kernel objects, and finally prove

invariants one by one by temporarily opening the closed spec function to reveal the invariant to the verifier:

```
pub fn kernel_add_mapping_4k(&mut self, ...)
    requires
        old(self).inv(),
        ...
{
    let vaddr = index2va((target_l4i, target_l3i, target_l2i, target_l1i));
    let Tracked(page_lock_perm) = self.page_array.wlock(page_index);
    let mut page = self.page_array.take(page_index, &page_lock_perm);
    page.ref_count = page.ref_count + 1;
    page.mappings_4k = Ghost(page.mappings_4k@.insert((pagetable_root, vaddr)));
    self.page_array.put(page_index, Tracked(lctx), &page_lock_perm, page);
    self.page_array.wunlock(page_index, Tracked(lctx), page_lock_perm);

    let mut pagetable = self.pagetable_map.take(pagetable_root, Tracked(lctx), pagetable_lock_perm);
    pagetable.map_4k_page(target_l4i, target_l3i, target_l2i, target_l1i, target_l1_p, target_entry);
    self.pagetable_map.put(pagetable_root, Tracked(lctx), pagetable_lock_perm, pagetable);

    assert(self.inv()) by {
        assert(self.container_pages_wf()) by {
            Self::container_pages_wf_proof();
        };
        assert(self.process_pages_wf()) by {
            Self::process_pages_wf_proof();
        };
        assert(hugepage_2m_wf(self.page_array)) by {
            page_ptr_lemma1();
            page_ptr_2m_lemma();
            page_ptr_1g_lemma();
            page_index_lemma();
            page_ptr_page_index_truncate_lemma();
            hugepage_2m_wf_proof();
        };

        assert(tlb_wf_spec(self.cpu_tlb, self.pagetable_map, self.cpu_array)) by {
            process_cpu_wf_proof();
            cpu_not_in_dirty_map_imply_not_in_tlb_proof();
```

```
37                  tlb_wf_spec_proof();
38                  cpu_dirty_map_contains_pagetable_pcid_match_proof();
39              };
40              //rest of proofs
41          };
42
43          return;
44      }
```

For example, `container_pages_wf` (line 19 to 21) is completely orthogonal to other invariants since proving it only requires opening its own closed spec function. On the other hand, `tlb_wf_spec` (line 34 to 39) is one of the most demanding invariants and involves more invariants in the system. Verifying it requires opening three other closed invariants. Even the complex invariant such as `tlb_wf_spec` presents a relatively small SMT search space for the verifier, and because of invariant isolation, we are able to pick more aggressive triggers for these invariants, which allows the verification to finish without any additional hints.

9.3 System Architecture

9.3.1 Container, Process, Thread, and Endpoint

Containers, processes, threads and endpoints are the central elements for VeriFlat's process management subsystem.

9.3.1.1 Container

A container consists of a group of processes, a set of endpoints that could be referenced by processes of child containers, an memory allocator, a scheduler, a

PCID/IOMMU ID allocator, a set of reserved memory pages, PCIe devices, and CPU cores. All memory allocations and scheduling are managed by the container's local memory allocator and scheduler, hence have no impact on the performance of memory allocations or scheduling from other containers. And each container's threads are always scheduled to run on the reserved CPU cores.

9.3.1.2 Process

Each process contains one or more threads, is assigned with a PCID and optionally an IOMMU ID, and owns a page table and optionally an IOMMU table.

9.3.1.3 Container and Process Tree

Similar to Atmosphere and Linux, to enable resource revocation and access right control all containers form a doubly-linked tree that tracks the parental relationships, and processes under each container form a similar doubly-linked tree to track the relationships within the tree. Containers and processes have the right to terminate a child container or process and all of the recursive children too.

9.3.2 Memory Allocator

In VeriFlat, each container contains three page allocators, one for each size of the physical page (4KiB, 2MiB hugepage, and 1GiB superpage). Each allocator contains a physical counter, named `Quota`, which tracks the number of available pages of a given size in the container, a free page doubly-linked list, and a per-CPU free page doubly-linked list as cache. Both the free page list and

per-CPU caches are invisible to the user-level; only the `Quota` is visible to the user-level. The proof guarantees:

1. memory allocations of size smaller than the `Quota` always succeed and decrement `Quota` exactly by the allocation size.
2. freeing memory always increments `Quota` by the freed size.

The benefits of this design are two-fold. First, `Quota` needs to be locked by only a short period of time. An allocation of a large number of pages requires checking and updating `Quota` only once. The actual allocation can happen gradually after unlocking `Quota` without modifying the user-aware states, by popping the per-CPU caches and the free page list. Second, the contention on the free page list is drastically reduced, as threads first consult their respective CPU cache, and only occasionally, move a batch of free pages from or to the local cache.

9.3.3 Virtual PCID and TLB

In VeriFlat, since CPUs are exclusively owned by containers, the CPU TLB (translation lookaside buffer) only needs to sync with the address spaces within the container, and each container can assign up to all 4096 unique PCIDs to its processes. To eliminate the system limitation on the total number of live processes, VeriFlat supports PCID virtualization. That is, processes within a container might be assigned with the same PCID, but the kernel performs a TLB flush on the PCID before scheduling a different process with the same PCID to ensure memory safety. Specifically, each CPU maintains a dirty map that tracks the last address space run on the CPU for each PCID. The invariant ensures that

for each PCID in the CPU TLB, the TLB is a strict submap of the corresponding address space. The kernel maintains the invariant by issuing TLB flushes when an address space is killed (as a part of a process), a new address space is scheduled to run on the CPU when the dirty bit is set, or an address space that is in the dirty map performs `munmap`.

9.3.4 Performance Isolation

VeriFlat's system architecture enables a high level of resource and performance isolation.

The strict division of CPU cores and hardware resource (e.g., physical pages, PCID/IOID virtualization, PCIe devices, etc.) guarantees resource isolation between containers – one container cannot, through any means, exhaust or even occupy system resources of another container, without an explicit ownership transfer, which requires consent from both containers.

Informally, VeriFlat also achieves container (partition) level performance isolation. Specifically, system calls from different containers operate on completely disjoint sets of kernel objects, hence eliminating performance penalty caused by lock contentions from different containers. There are a few exceptions. By design, Kill system calls allow one container to interfere with its children containers, and IPC system calls across containers allow containers to contend for overlapping kernel objects. Freeing shared resources may cause system calls invoked from one container to operate on kernel objects belonging to another container. For example, if container `C_A` shares a physical page to container

C_B, when C_B frees the page and the reference counter becomes zero, C_B write-locks C_A's page allocator to return the page to C_A's free list. Although we argue that all these exceptions exist by design, and could only happen when the containers are involved in shared resources with other containers. If a container never shares any resources nor establishes communication channels outside of its own, it has full performance isolation. Note that our implementation does not consider any possible performance penalty caused by memory bandwidth usage, CPU TLB or CPU caches.

9.4 Trusted Computing Base

VeriFlat reuses much of Atmosphere's infrastructure and thus the TCB.

VeriFlat assumes the x86-64 total store order(TSO) memory model for read/writes not protected by locks (e.g., a page table walk issued by a user-space program). However, for vast majority of concurrent read/write on locked protected kernel objects, VeriFlat follows the Rust memory model (the C++11/C++20 memory model), which completely abstracts away the underlying machine memory model.

In addition to the TCB of Atmosphere, to enable verification of concurrency, we introduce the following items to VeriFlat's TCB

1. The implementation of locks and Rust atomic primitives (memory order)
2. The soundness of the lock ordering algorithm
3. The use of logic atomicity and reduction

4. Modeling of possible concurrent actions at interleaving points (although we model the maximum possible actions).

9.5 Current Status

We have completed a significant portion of this ongoing work, including:

1. All primitive data structures (e.g., page tables, linked lists, etc.)
2. Concurrency-related infrastructure (e.g., locks, local contexts, etc.)
3. Approximately 80% of system invariants
4. Several internal kernel-level function calls (e.g., `kernel_add_mapping_4k`).

VeriFlat's primitive data structures are further optimized compared to those in Atmosphere, enabling improved proof scalability and performance. As of now, the entire kernel completes a full verification run in under 10 seconds on a 1300HX processor, whereas Atmosphere takes approximately 20 seconds under similar conditions.

We anticipate an additional six to twelve months of development before VeriFlat is ready for submission to a major conference.

9.6 Conclusions

In this work, we present ongoing progress toward practical verification of concurrent systems code, including the development of techniques and concurrency rules that support scalable verification.

Our preliminary results suggest that a combination of relatively simple yet expressive concurrency rules, together with modern automated verification tools, can enable the practical development of formally verified concurrent kernels.

In particular, the use of logical atomicity, permission-based locking, and lock ordering has allowed us to build VeriFlat with minimal modifications to an automatic deductive program verifier, in a manner similar to the single-threaded kernel Atmosphere, while reusing much of its optimized verified primitives.

This work remains in progress. We plan to further explore the integration of high-performance lock-free data structures into VeriFlat and to redesign the page table with fine-grained locking for improved performance. Additionally, we aim to study potential proof scalability challenges as system complexity continues to grow.

CHAPTER 10

CONCLUSION

We present how to build formally verified operating system microkernels.

Verus, the combination of SMT solvers and safe language, Rust, provides a practical foundation for formal verification of systems code. In the first half of the document, we introduce the basics of formal verification, Verus semantics, encoding from Rust to SMT queries, and a real world example of how to extend Verus to support opaque types and async functions. In the second half of the document, we present Atmosphere and VeriFlat, two microkernels built and verified with Verus.

Our successes and preliminary result in Atmosphere and VeriFlat prove a point: *Formal verification of a complete, multi-threaded microkernel with a SMT based verification framework for its functional correctness, safety, and deadlock freedom is feasible and practical.*

BIBLIOGRAPHY

[1] M. Abadi and L. Lamport, "The existence of refinement mappings," *Theor. Comput. Sci.*, vol. 82, no. 2, pp. 253–284, May 1991. DOI: https://doi.org/10.1016/0304-3975(91)90224-P.

[2] V. Astrauskas, P. Müller, F. Poli, and A. J. Summers, "Leveraging Rust types for modular specification and verification," in *Proc. ACM SIGPLAN Int. Conf. Object-Oriented Program. Syst. Lang. Appl. (OOPSLA)*, vol. 3, ACM, Oct. 2019, 147:1–147:30. DOI: 10.1145/3360573.

[3] M. Barnett, B.-Y. E. Chang, R. DeLIne, B. Jacobs, and R. Leino, "Boogie: A modular reusable verifier for object-oriented programs," in *FMCO 2005*, Springer Berlin Heidelberg, Nov. 2005. [Online]. Available: https://www.microsoft.com/en-us/research/publication/boogie-a-modular-reusable-verifier-for-object-oriented-programs/.

[4] A. Bhardwaj, C. Kulkarni, R. Achermann, I. Calciu, S. Kashyap, R. Stutsman, A. Tai, and G. Zellweger, "Nros: Effective replication and sharing in an operating system," in *Proc. 15th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '21)*, USENIX Association, Jul. 2021, pp. 295–312. [Online]. Available: https://www.usenix.org/conference/osdi21/presentation/bhardwaj.

[5] R. Bornat, C. Calcagno, P. O'Hearn, and M. Parkinson, "Permission accounting in separation logic," in *Proc. 32nd ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL '05)*, ser. POPL '05, Long Beach, California, USA: ACM, Jan. 2005, pp. 259–270. DOI: 10.1145/1040305.1040327.

[6] W. de Bruijn and H. Bos, "Beltway buffers: Avoiding the os traffic jam," in *Proc. 27th IEEE Int. Conf. Comput. Commun. (INFOCOM)*, Apr. 2008, pp. 136–140. DOI: 10.1109/INFOCOM.2008.36.

[7] M. Brun, R. Achermann, T. Chajed, J. Howell, G. Zellweger, and A. Lattuada, "Beyond Isolation: OS Verification as a Foundation for Correct Applications," in *Proc. 19th Workshop Hot Top. Oper. Syst. (HotOS '23)*, ser. HOTOS '23, Providence, RI, USA, Jun. 2023, pp. 158–165. DOI: 10.1145/3593856.3595899.

[8] A. Burtsev, K. Srinivasan, P. Radhakrishnan, L. N. Bairavasundaram, K. Voruganti, and G. R. Goodson, "Fido: Fast inter-virtual-machine communication for enterprise appliances," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC '09)*, Jun. 2009, pp. 313–326.

[9] I. Calciu, S. Sen, M. Balakrishnan, and M. K. Aguilera, "Black-box concurrent data structures for numa architectures," in *Proc. 22nd Int. Conf. Archit. Support Program. Lang. Oper. Syst. (ASPLOS '17)*, ser. ASPLOS '17, Xi'an, China: ACM, Apr. 2017, pp. 207–221. DOI: 10.1145/3037697.3037721.

[10] C. Cebeci, Y. Zou, D. Zhou, G. Candea, and C. Pit-Claudel, "Practical verification of system-software components written in standard c," in *Proc. 30th ACM SIGOPS Symp. Oper. Syst. Princ. (SOSP '24)*, ser. SOSP '24, Austin, TX, USA: ACM, Oct. 2024, pp. 455–472. DOI: 10.1145/3694715.3695980.

[11] X. Chen, Z. Li, T. Jain, V. Narayanan, and A. Burtsev, "Limitations and opportunities of modern hardware isolation mechanisms," in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC '24)*, Santa Clara, CA: USENIX Association, Jul. 2024, pp. 349–368.

[12] X. Chen, Z. Li, L. Mesicek, V. Narayanan, and A. Burtsev, "Atmosphere: Towards practical verified kernels in rust," in *Proc. 1st Workshop Kernel Isolation, Safety Verification (KISV '23)*, ser. KISV '23, Koblenz, Germany: ACM, Oct. 2023, pp. 9–17. DOI: 10.1145/3625275.3625401.

[13] X. Chen, Z. Li, J. Zhang, and A. Burtsev, "Veld: Verified linux drivers," in *Proc. 2nd Workshop Kernel Isolation, Safety Verification (KISV '24)*, ser. KISV '24, Austin, TX, USA: ACM, Oct. 2024, pp. 23–30. DOI: 10.1145/3698576.3698766.

[14] X. Chen, Z. Li, J. Zhang, V. Narayanan, and A. Burtsev, "Atmosphere: Practical verified kernels with rust and verus," in *Proc. 31st ACM SIGOPS Symp. Oper. Syst. Princ. (SOSP '25)*, ser. SOSP '25, Lotte Hotel World, Seoul, Republic of Korea: ACM, Oct. 2025, pp. 752–767. DOI: 10.1145/3731569.3764821.

[15] *CloudLab hardware info*. [Online]. Available: https://docs.cloudlab.us/hardware.html.

[16] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Oper. Syst. Princ. (SOSP '07)*, ser. SOSP '07, Oct. 2007, pp. 205–220. DOI: 10.1145/1294261.1294281.

[17] *DPDK: Data Plane Development Kit*. [Online]. Available: https://www.dpdk.org.

[18] D. E. Eisenbud, C. Yi, C. Contavalli, C. Smith, R. Kononov, E. Mann-Hielscher, A. Cilingiroglu, B. Cheyney, W. Shang, and J. D. Hosein, "Maglev: A fast and reliable software network load balancer," in *Proc. 13th USENIX Symp. Netw. Syst. Des. Implement. (NSDI '16)*, ser. NSDI '16, Mar. 2016, pp. 523–535. [Online]. Available: https://www.usenix.org/conference/nsdi16/technical-sessions/presentation/eisenbud.

[19] A. Ferraiuolo, A. Baumann, C. Hawblitzel, and B. Parno, "Komodo: Using verification to disentangle secure-enclave hardware from software," in *Proc. 26th ACM Symp. Oper. Syst. Princ. (SOSP '17)*, ser. SOSP '17, Shanghai, China: ACM, Oct. 2017, pp. 287–305. DOI: 10.1145/3132747.3132782.

[20] K. Fraser, S. Hand, R. Neugebauer, I. Pratt, A. Warfield, M. Williamson, *et al.*, "Safe hardware access with the Xen virtual machine monitor," in *Proc. 1st Workshop Oper. Syst. Archit. Support Demand IT Infrastruct. (OASIS)*, ser. OASIS, 2004, pp. 1–1.

[21] J. A. Goguen and J. Meseguer, "Unwinding and inference control," in *Proc. IEEE Symp. Secur. Privacy (SP)*, 1984, pp. 75–75. DOI: 10.1109/SP.1984.10019.

[22] R. Gu, Z. Shao, H. Chen, X. (Wu, J. Kim, V. Sjöberg, and D. Costanzo, "CertiKOS: An extensible architecture for building certified concurrent OS kernels," in *Proc. 12th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '16)*, Savannah, GA: USENIX Association, Nov. 2016, pp. 653–669.

[23] T. Hance, Y. Zhou, A. Lattuada, R. Achermann, A. Conway, R. Stutsman, G. Zellweger, C. Hawblitzel, J. Howell, and B. Parno, "Sharding the state machine: Automated modular reasoning for complex concurrent systems," in *Proc. 17th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '23)*, Boston, MA: USENIX Association, Jul. 2023, pp. 911–929. [Online]. Available: https://www.usenix.org/conference/osdi23/presentation/hance.

[24] C. Hawblitzel, J. Howell, M. Kapritsos, J. R. Lorch, B. Parno, M. L. Roberts, S. Setty, and B. Zill, "Ironfleet: Proving practical distributed systems correct," in *Proc. 25th ACM Symp. Oper. Syst. Princ. (SOSP '15)*, ser. SOSP '15, Monterey, California: ACM, Oct. 2015, pp. 1–17. DOI: 10.1145/2815400.2815428.

[25] C. Hawblitzel, J. Howell, J. R. Lorch, A. Narayan, B. Parno, D. Zhang, and B. Zill, "Ironclad apps: End-to-End security via automated Full-System verification," in *Proc. 11th USENIX Symp. Oper. Syst. Des. Implement. (OSDI*

'14), ser. OSDI '14, Broomfield, CO: USENIX Association, Oct. 2014, pp. 165–181. [Online]. Available: https://www.usenix.org/conference/osdi14/technical-sessions/presentation/hawblitzel.

[26] G. Heiser, *Benchmarking crimes meet formal verification*, Apr. 27, 2025. [Online]. Available: https://microkerneldude.org/2025/04/27/benchmarking-crimes-meet-formal-verification/.

[27] M. P. Herlihy and J. M. Wing, "Linearizability: A correctness condition for concurrent objects," *ACM Trans. Program. Lang. Syst.*, vol. 12, no. 3, pp. 463–492, Jul. 1990. DOI: 10.1145/78969.78972.

[28] Intel Corporation, *Storage Performance Development Kit (SPDK)*. [Online]. Available: https://spdk.io.

[29] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer, "RustBelt: Securing the foundations of the Rust programming language," *Proc. ACM Program. Lang.*, vol. 2, 66:1–66:34, POPL Dec. 27, 2017. DOI: 10.1145/3158154.

[30] R. Jung, B. Kimock, C. Poveda, E. S. Muñoz, O. Scherer, and Q. Wang, "Miri: Practical undefined behavior detection for rust," *Proc. ACM Program. Lang.*, vol. 10, no. POPL, Jan. 2026. DOI: 10.1145/3776690.

[31] R. JUNG, R. KREBBERS, J.-H. JOURDAN, A. BIZJAK, L. BIRKEDAL, and D. DREYER, "Iris from the ground up: A modular foundation for higher-order concurrent separation logic," *J. Funct. Program.*, vol. 28, e20, Nov. 2018. DOI: 10.1017/S0956796818000151.

[32] G. Klein, J. Andronick, K. Elphinstone, G. Heiser, D. Cock, P. Derrin, D. Elkaduwe, K. Engelhardt, R. Kolanski, M. Norrish, T. Sewell, H. Tuch, and S. Winwood, "Sel4: Formal verification of an operating-system kernel," *Commun. ACM*, vol. 53, no. 6, pp. 107–115, Jun. 2010. DOI: 10.1145/1743546.1743574.

[33] A. Lattuada, T. Hance, J. Bosamiya, M. Brun, C. Cho, H. LeBlanc, P. Srinivasan, R. Achermann, T. Chajed, C. Hawblitzel, J. Howell, J. R. Lorch, O. Padon, and B. Parno, "Verus: A practical foundation for systems verification," in *Proc. 30th ACM SIGOPS Symp. Oper. Syst. Princ. (SOSP '24)*, ser. SOSP '24, Austin, TX, USA: ACM, Oct. 2024, pp. 438–454. DOI: 10.1145/3694715.3695952.

[34] A. Lattuada, T. Hance, C. Cho, M. Brun, I. Subasinghe, Y. Zhou, J. Howell, B. Parno, and C. Hawblitzel, "Verus: Verifying rust programs using linear ghost types," *Proc. ACM Program. Lang.*, vol. 7, no. OOPSLA1, Apr. 2023. DOI: 10.1145/3586037.

[35] R. Leino, "Dafny: An automatic program verifier for functional correctness," in *Proc. 16th Int. Conf. Logic Program. Autom. Reason. (LPAR-16)*, Springer Berlin Heidelberg, Apr. 2010, pp. 348–370. [Online]. Available: https://www.microsoft.com/en-us/research/publication/dafny-automatic-program-verifier-functional-correctness-2/.

[36] A. Levy, B. Campbell, B. Ghena, D. B. Giffin, P. Pannuto, P. Dutta, and P. Levis, "Multiprogramming a 64kb computer safely and efficiently," in *Proc. 26th ACM Symp. Oper. Syst. Princ. (SOSP '17)*, ser. SOSP '17, Shanghai, China: ACM, Oct. 2017, pp. 234–251. DOI: 10.1145/3132747.3132786.

[37] J. Li, A. Lattuada, Y. Zhou, J. Cameron, J. Howell, B. Parno, and C. Hawblitzel, "Linear types for large-scale systems verification," *Proc. ACM Program. Lang.*, vol. 6, no. OOPSLA1, Apr. 2022. DOI: 10.1145/3527313.

[38] Z. Li, V. Narayanan, X. Chen, J. Zhang, and A. Burtsev, "Rust for linux: Understanding the security impact of rust in the linux kernel," in *Proc. Annu. Comput. Secur. Appl. Conf. (ACSAC '24)*, Dec. 2024, pp. 548–562. DOI: 10.1109/ACSAC63791.2024.00054.

[39] Z. Li, J. Zhang, J. Tlatelpa-Agustin, X. Chen, and A. Burtsev, "Understanding the security impact of cheri on the operating system kernel," in *Proc. Annu. Comput. Secur. Appl. Conf. (ACSAC '25)*, Dec. 2025, pp. 212–227. DOI: 10.1109/ACSAC67867.2025.00031.

[40] Z. Li, J. Wang, M. Sun, and J. C. Lui, "Mirchecker: Detecting bugs in rust programs via static analysis," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur. (CCS '21)*, ser. CCS '21, Virtual Event, Republic of Korea: ACM, Nov. 2021, pp. 2183–2196. DOI: 10.1145/3460120.3484541.

[41] R. J. Lipton, "Reduction: A method of proving properties of parallel programs," *Commun. ACM*, vol. 18, no. 12, pp. 717–721, Dec. 1975. DOI: 10.1145/361227.361234.

[42] P. Müller, M. Schwerhoff, and A. J. Summers, "Viper: A verification infrastructure for permission-based reasoning," in *Proc. Int. Conf. Verification, Model Checking, Abstract Interpret. (VMCAI)*, B. Jobstmann and K. R. M. Leino, Eds., Berlin, Heidelberg: Springer, Jan. 2016, pp. 41–62. DOI: 10.1007/978-3-662-49122-5_2.

[43] L. Nelson, J. Bornholt, R. Gu, A. Baumann, E. Torlak, and X. Wang, "Scaling symbolic evaluation for automated verification of systems code with serval," in *Proc. 27th ACM Symp. Oper. Syst. Princ. (SOSP '19)*, ser. SOSP

'19, Huntsville, Ontario, Canada: ACM, Oct. 2019, pp. 225–242. DOI: 10.1145/3341301.3359641.

[44] L. Nelson, J. Bornholt, A. Krishnamurthy, E. Torlak, and X. Wang, "Noninterference specifications for secure systems," *SIGOPS Oper. Syst. Rev.*, vol. 54, no. 1, pp. 31–39, Aug. 2020. DOI: 10.1145/3421473.3421478.

[45] L. Nelson, H. Sigurbjarnarson, K. Zhang, D. Johnson, J. Bornholt, E. Torlak, and X. Wang, "Hyperkernel: Push-button verification of an os kernel," in *Proc. 26th ACM Symp. Oper. Syst. Princ. (SOSP '17)*, ser. SOSP '17, Shanghai, China: ACM, Oct. 2017, pp. 252–269. DOI: 10.1145/3132747.3132748.

[46] R. Nishtala, H. Fugal, S. Grimm, M. Kwiatkowski, H. Lee, H. C. Li, R. McElroy, M. Paleczny, D. Peek, P. Saab, D. Stafford, T. Tung, and V. Venkataramani, "Scaling memcache at Facebook," in *Proc. 10th USENIX Symp. Netw. Syst. Des. Implement. (NSDI '13)*, ser. NSDI '13, Apr. 2013, pp. 385–398. [Online]. Available: https://www.usenix.org/conference/nsdi13/technical-sessions/presentation/nishtala.

[47] O. Padon, K. L. McMillan, A. Panda, M. Sagiv, and S. Shoham, "Ivy: Safety verification by interactive generalization," *SIGPLAN Not.*, vol. 51, no. 6, pp. 614–630, Jun. 2016. DOI: 10.1145/2980983.2908118.

[48] S. Pernsteiner, I. S. Diatchki, R. Dockins, M. Dodds, J. Hendrix, T. Ravich, P. Redmond, R. Scott, and A. Tomb, *Crux, a precise verifier for rust and other languages*, Oct. 2024. arXiv: 2410.18280 [cs.PL].

[49] T. R. Project, *Reference cycles can leak memory*. [Online]. Available: https://doc.rust-lang.org/book/ch15-06-reference-cycles.html.

[50] *Recursive proof in Verified NrOS page table*, Accessed: 2024-12-10. [Online]. Available: https://github.com/matthias-brun/verified-nrkernel/blob/8f30cf0f910e7606c3a0f633821acfdfde410cf4/page-table/impl_u/l2_impl.rs#L947-L1562.

[51] J. Reynolds, "Separation logic: A logic for shared mutable data structures," in *Proc. 17th Annu. IEEE Symp. Log. Comput. Sci. (LICS)*, Jul. 2002, pp. 55–74. DOI: 10.5555/645683.664578.

[52] R. Ricci, E. Eide, and The CloudLab Team, "Introducing CloudLab: Scientific infrastructure for advancing cloud architectures and applications," *USENIX ;login:*, vol. 39, no. 6, Dec. 2014. [Online]. Available: https://www.usenix.org/publications/login/dec14/ricci.

[53] J. Rushby, *Noninterference, transitivity, and channel-control security policies*. SRI International, Computer Science Laboratory Menlo Park, Dec. 1992. [Online]. Available: http://www.csl.sri.com/papers/csl-92-2/.

[54] S. F. Siegel, M. Zheng, Z. Luo, T. K. Zirkel, A. V. Marianiello, J. G. Edenhofner, M. B. Dwyer, and M. S. Rogers, "Civl: The concurrency intermediate verification language," in *Proc. Int. Conf. High Perform. Comput. Netw. Storage Anal. (SC '15)*, ser. SC '15, Austin, Texas: ACM, Nov. 2015. DOI: 10.1145/2807591.2807635.

[55] X. Sun, W. Ma, J. T. Gu, Z. Ma, T. Chajed, J. Howell, A. Lattuada, O. Padon, L. Suresh, A. Szekeres, and T. Xu, "Anvil: Verifying liveness of cluster management controllers," in *Proc. 18th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '24)*, Santa Clara, CA: USENIX Association, Jul. 2024, pp. 649–666. [Online]. Available: https://www.usenix.org/conference/osdi24/presentation/sun-xudong.

[56] A. VanHattum, D. Schwartz-Narbonne, N. Chong, and A. Sampson, "Verifying dynamic trait objects in rust," in *Proc. 44th IEEE/ACM Int. Conf. Softw. Eng. Softw. Eng. Pract. (ICSE-SEIP)*, Oct. 2022, pp. 321–330. DOI: 10.1145/3510457.3513031.

[57] Verus-Lang, *A verified doubly-linked list example in Verus*. [Online]. Available: https://github.com/verus-lang/verus/blob/main/examples/doubly_linked.rs.

[58] Verus-Lang, *Recursive exec and proof functions, proofs by induction*. [Online]. Available: https://verus-lang.github.io/verus/guide/induction.html.

[59] P. Wadler, "Linear types can change the world!" In *Proc. IFIP Work. Conf. Program. Concepts Methods*, vol. 3, Apr. 1990, p. 5.

[60] J. Yang and C. Hawblitzel, "Safe to the last instruction: Automated verification of a type-safe operating system," *SIGPLAN Not.*, vol. 45, no. 6, pp. 99–110, Jun. 2010. DOI: 10.1145/1809028.1806610.

[61] Z. Zhou, Anjali, W. Chen, S. Gong, C. Hawblitzel, and W. Cui, "Verismo: A verified security module for confidential vms," in *Proc. 18th USENIX Symp. Oper. Syst. Des. Implement. (OSDI '24)*, Awarded Best Paper! Jul. 2024. [Online]. Available: https://www.microsoft.com/en-us/research/publication/verismo-a-verified-security-module-for-confidential-vms/.